Design and Structure of a CAS: Symbolic Programming in LISP

Jose Daniel Fernandez Santos (@dfsantos1)

Part I — Learning LISP through MyLISP

Reference implementations: MyLISP/QL (Prospero Pro Pascal · Sinclair QL) and MyLISP/Next (Z88DK C · ZX Spectrum Next)

Table of Contents

- Appendix E — Complete Source Code of `CAS.LSP`

\newpage

Chapter 1 — What Is MYLISP?

1.1 A Brief History of LISP

The history of LISP is, at bottom, the story of how a piece of mathematical notation on paper turned almost by accident into a programming language — and of how that translation sparked a debate about language design that is still alive today.

1.1.1 The Origin: From the Lambda Calculus to the Machine (1958)

In 1958, John McCarthy, a mathematician at the Massachusetts Institute of Technology (MIT), published a paper titled *"Recursive Functions of Symbolic Expressions and Their Computation by Machine."* His purpose was not to create an executable language: he was after a **mathematical notation**, based on Alonzo Church's *lambda calculus*, for reasoning about artificial-intelligence algorithms. He defined S-expressions (symbolic expressions) and a universal `eval` function capable of interpreting them.

The leap to real computing was made by **Steve Russell**, one of McCarthy's students. Russell read the paper and realized he could implement `eval` in machine code for an IBM 704. McCarthy, as the legend goes, told him that `eval` was theoretical mathematics, not real code. Russell programmed it anyway. That is how the first LISP interpreter was born.

What made LISP radically different from the languages of its era (FORTRAN, COBOL, aimed at numerical computation or data processing) was the accumulation of ideas no one had put together before: **automatic garbage collection**, **dynamic typing**, **functions as first-class values**, and, above all, **homoiconicity**: code and data share exactly the same structure. A LISP program can create, analyze, and run other LISP programs because both are simply lists of symbols.

```
(+ 2 3)          ; this is executable code at the same time...
(QUOTE (+ 2 3))  ; ...and data: the three-element list +, 2, and 3
```

Throughout the sixties and seventies, LISP became the instrument with which MIT researchers built the first programs capable of proving mathematical theorems, playing chess, translating languages, and solving symbolic integrals. **Macsyma**, the Computer Algebra System — perhaps the most ambitious software project to come out of MIT in that era — was written almost entirely in LISP.

1.1.2 The Split: LISP-1 versus LISP-2

As LISP grew during the 60s and 70s, multiple dialects appeared — MacLISP, InterLISP, ZetaLISP — and with them a deep design question that split the LISP family into two branches that coexist to this day: the matter of **how many namespaces** the language should have.

The question is simple: when the interpreter sees the symbol `SUM`, does it always look it up in the same dictionary, or does it keep separate dictionaries depending on whether the symbol appears in function position or variable position?

LISP-1 has a single namespace. Functions and variables live in the same dictionary. A symbol like `LIST` can only point to one thing at a time. The champion of this approach was **Scheme**, created by Guy L. Steele and Gerald Jay Sussman in 1975, which additionally formalized strict **lexical scope** (functions capture the environment where they were defined, not where they are called). Mathematically, LISP-1 is faithful to the lambda calculus: functions are first-class citizens exactly like numbers. If you have a function, you pass it to another exactly as if it were a piece of data, with no special syntax.

LISP-2 has separate namespaces. A symbol has an independent *value cell* and *function cell*. When the interpreter sees `(SUM A B)`, it looks up `SUM` in the function cell; when it sees `SUM` as an argument, it looks it up in the value cell. The champion of this model was **Common LISP**, standardized in 1984 to unify the industrial fragmentation of 1970s dialects. Common LISP is pragmatic: you can have a variable named `list` that holds data and still call the function `(list 1 2 3)` without the

interpreter getting confused. The price is verbosity in functional programming: to pass a function as an argument you must use the `#'` operator, and to call one stored in a variable you must use `funcall`.

```
;; In Common LISP (LISP-2): the syntax gives away the namespace
(mapcar #'double '(1 2 3))

;; In Scheme / MYLISP (LISP-1): a function is a value like any other
(MY-MAP DOUBLE '(1 2 3))
```

Common LISP's choice of the LISP-2 model was, to a large extent, pragmatic: it kept compatibility with the enormous body of MacLISP code already running on the LISP machines of MIT's labs and companies such as Symbolics. Common LISP ended up being a massive, multi-paradigm language — it includes CLOS, one of the most powerful object systems ever designed — aimed at building large-scale industrial software.

Scheme took the opposite path: the smallest possible number of primitives and rules, maximum conceptual purity. It was designed to teach and to reason about computation. Its authors' stated goal was to show that "a programming language can be powerful and expressive with very few rules."

1.1.3 Why MYLISP Follows Scheme's Path

MYLISP is a LISP-1. This is not a minor implementation decision: it defines the mental model the programmer works with. If the goal is to teach LISP on a resource-constrained platform — the Sinclair QL — and to build a CAS from first principles, understanding every abstraction barrier along the way, the LISP-1 approach is the architecturally correct one.

It removes the cognitive friction between "data" and "code." When, in Chapter 10, we pass a differentiation function as an argument to another function, or return closures that capture algebraic expressions, we won't need any special syntax or extra operators. Everything — functions, numbers, lists, symbolic expressions — is simply a value, and is treated the same way.

1.2 The Platforms: Sinclair QL and ZX Spectrum Next

MYLISP exists in two independent implementations that share the same language and the same philosophy, but run on completely different hardware. Knowing both helps you understand why some limitations belong to the language and others to the platform.

1.2.1 The Sinclair QL

The Sinclair QL (*Quantum Leap*) was introduced by Sir Clive Sinclair in January 1984. Equipped with a Motorola 68008 processor at 7.5 MHz and 128 KB of RAM in its base configuration, the QL was the first 32-bit personal computer accessible to the general public. Its price was notably lower than that of the professional systems of the time.

The QL shipped with its own multitasking operating system — QDOS — and two microdrive units for storage. Microdrives were looped magnetic-tape cartridges that behaved like disks, though with lower speed and reliability. Files on the QL are named with a path such as `mdv1_name`, where `mdv1` indicates the physical drive.

To run MYLISP/QL you need a memory expansion of at least **640 KB**. With that RAM available, the interpreter has a heap of **24,000 cells**, can hold up to 200 symbols, and manages its own string and real-number tables. The implementation is written in **Prospero Pro Pascal** and compiled directly for the 68008 processor.

The QL was a machine ahead of its time. Although it never came to dominate the home market, it built a loyal community of programmers who saw in it a serious platform for ambitious projects. MYLISP is one of those projects.

1.2.2 The ZX Spectrum Next

The ZX Spectrum Next is a modern reimagining of the original ZX Spectrum, launched through crowdfunding in 2017. It runs an enhanced Z80 processor capable of running at 3.5, 7, or **28 MHz**, has **2 MB of RAM** organized into 8 KB pages, and runs **NextZXOS** as its operating system, which includes a FAT-compatible filesystem (esxdos) accessible from an SD card.

MYLISP/Next is written in **C with the Z88DK compiler** and takes advantage of the Next's paged memory to offer a heap of **32,000 cells** — larger than on the QL — without consuming any of the Z80's base 64 KB flat address space. Files are stored on the SD card and named with Unix-style paths: `cas.lsp`, `utils.lsp`, and so on.

The most important architectural difference between the two versions is the **evaluator**. The QL version uses a classic recursive evaluator: every call to a LISP function adds a frame to Pascal's call stack. The Next version uses an **iterative evaluator** with an explicit task stack stored in paged memory; user recursion consumes no CPU stack at all, which allows recursions several hundred levels deep with no trouble.

1.2.3 Convention Used in This Book

When MYLISP's behavior is identical on both platforms — which is true in most cases — the text does not distinguish between them. When there is a difference relevant to the programmer, a note like this appears:

> **QL:** behavior or limit on the Sinclair QL. **Next:** behavior or limit on the ZX Spectrum Next.

Code examples are valid on both platforms unless stated otherwise.

1.3 What MYLISP Is

MYLISP is a LISP interpreter available in two implementations: **MYLISP/QL**, written in Prospero Pro Pascal for the Sinclair QL, and **MYLISP/Next**, written in C with Z88DK for the ZX Spectrum Next. In both cases the executable is self-contained and boots directly from the platform's storage medium.

Its main purpose is to serve as the **engine for a Computer Algebra System** (CAS). A CAS is a program capable of manipulating mathematical expressions symbolically: it can differentiate, simplify, expand, or factor expressions without ever converting them to decimal numbers. LISP's strength for these tasks lies precisely in the fact that an expression such as `(+ x (* 2 x))` is, in LISP, an ordinary piece of data — a list — that can be inspected and transformed by any function.

MYLISP follows the **Lisp-1** model: functions and variables share the same namespace. This puts it closer to Scheme than to Common LISP, and considerably simplifies the mental model needed to program in it.

MYLISP's main features are:

- **Exact arithmetic with integers and rationals.** Integers are not silently converted to floating point when they exceed a limit. Instead, the system produces an explicit error. Fractions such as `1/2` or `3/7` are a native data type and are automatically reduced by their greatest common divisor.
- **A complete recursive evaluator.** MYLISP implements the fundamental LISP special forms: `QUOTE`, `IF`, `COND`, `AND`, `OR`, `PROGN`, `LET`, `LAMBDA`, `DEFINE`, and `DEFUN`.
- **Lexical closures.** Functions capture the environment in which they were defined, which enables advanced functional-programming techniques.
- **Automatic garbage collection.** It uses the *mark-and-sweep* algorithm. The user never needs to manage memory by hand.
- **Loading programs from storage.** The special form `LOAD` lets you load and run code files: from the QL's microdrives or from the Next's SD card.

1.4 The MYLISP Dialect: Its Place in the LISP Family

MYLISP is a fascinating hybrid: it has **Scheme's architectural engine** — Lisp-1 and lexical scope — but wears the **syntactic clothing of MacLISP and Common LISP**: `T`/`NIL`, `DEFUN`, `COND`, `PROGN`. Someone coming from Scheme will notice the familiar Common LISP syntax; someone coming from Common LISP will miss quite a few primitives. This section maps out exactly where MYLISP stands relative to both standards, so there are no surprises.

1.4.1 What MYLISP Shares with Scheme (and Not with Common LISP)

The kinship with Scheme is structural: a single namespace for functions and variables (Lisp-1), and **lexical scope** — functions capture the environment in which they were defined, not the one in which they are called. This is what makes the closures of Chapter 7 and the higher-order functional programming of Chapter 9 possible. Common LISP also has lexical scope by default, but its MacLISP heritage adds complexities that Scheme — and MYLISP — deliberately avoid.

MYLISP is also functionally **purer than Scheme itself** in one specific respect: there are no destructive primitives. Scheme includes `set!`, `set-car!`, and `set-cdr!` to mutate variables and lists in place. In MYLISP, once a value is bound to a symbol, the only way to change it is to redefine it globally with `DEFINE`. There is no way to modify an existing list; every list operation builds new structures. This is not an accidental limitation: it is a design decision that favors clarity in an educational context and eliminates an entire class of bugs.

1.4.2 What MYLISP Takes from MacLISP and Common LISP

MYLISP's **surface syntax** comes from the MacLISP branch. In Scheme, the booleans are `#t` and `#f`, and the empty list `()` is a distinct object from the false boolean. In MYLISP, as in Common LISP, `NIL` does both jobs at once: it is the empty list *and* the false value. `T` is the canonical true value, though any non-`NIL` value is considered true in a condition. Scheme predicates such as `boolean?` do not exist in MYLISP.

Likewise, the names of the special forms follow the MacLISP tradition: `DEFUN` instead of Scheme's `define`, `COND` instead of `cond` with different syntax, `PROGN` instead of `begin`.

1.4.3 Scheme Features Absent from MYLISP

There are three structural Scheme features that MYLISP does not implement, either by design or because of the limitations of the target hardware:

Continuations (`call-with-current-continuation` or `call/cc`). This is Scheme's most powerful and peculiar primitive: it lets you capture the exact state of execution and return to it later. With continuations you can implement exceptions, coroutines, and generators within the language itself. MYLISP does not implement them; instead, `LOAD` plays an analogous role at the session level: it lets you load, from microdrive, any set of definitions and data that reconstitutes exactly the desired working state, in the style of a batch file. It is a simpler mechanism, and a more appropriate one for the QL environment.

Hygienic macros (`define-syntax` / `syntax-rules`). Scheme lets you redefine the language itself by safely adding new syntactic forms, with no risk of variable collisions between the macro and the code that uses it. MYLISP has no macro system; the language is fixed to the special forms the interpreter implements.

Tail-call optimization (TCO). The R5RS standard requires Scheme to optimize tail-position calls so that they consume no stack. MYLISP does not implement TCO in either of its versions.

> **QL:** every recursive call adds a frame to Pascal's stack. Very deep recursions can exhaust the stack; in practice, the integer and heap limits tend to show up first. **Next:** the evaluator is iterative, and user recursion consumes no CPU stack. The practical limit on recursion depth on the Next is not the stack but the 32,000-cell heap — much more generous for pure functional programs.

1.4.4 Missing LET Variants

MYLISP has `LET` with parallel binding (all values are evaluated in the outer environment before any new variable becomes visible). Scheme additionally defines `let*` — sequential binding, where each variable can see the previous ones — and `letrec` — needed to define mutually recursive local closures. In MYLISP, `let*` is simulated by nesting `LET`s, and mutually recursive functions are defined with global `DEFINE` (which works perfectly, because global definitions are visible at call time, not at definition time).

1.4.5 Integers: Fixed Precision vs. Bignums

Scheme (from R5RS onward) requires unlimited exact arithmetic precision: integers can be as large as memory allows. MYLISP, by contrast, works with fixed-precision, 32-bit signed integers (range ±2,147,483,647) and produces an explicit error instead of silently converting to floating point. This is a design decision that prioritizes **algebraic correctness** over numeric range: for a symbolic CAS, an integer that silently "lies" by turning into a float would be worse than a visible error.

1.4.6 Missing Primitives That Can Be Defined in Pure LISP

MYLISP does not implement as primitives: `ABS`, `MAX`, `MIN`, `SQRT`, `EXPT`, `REVERSE`, `LENGTH`, `MAP`, `FILTER`, `REDUCE`, and `APPLY`. Nor does it have the Common LISP abbreviations `CADR`, `CADDR`, etc., or the search predicates `ASSOC` and `MEMBER`. All of them can be defined in a few lines of pure LISP; Appendix A.6 and Chapter 9 show how.

> **Note:** if future versions of the interpreter add primitives that today must be defined by hand (such as `EXPT` or `ABS`), this manual will be updated to reflect it.

1.4.7 The Syntax MYLISP Shares with Both

Differences aside, MYLISP and both standards share the essentials: prefix notation with parentheses, the S-expression structure, `LAMBDA` for creating functions, `QUOTE` or `'` to suppress evaluation, `CONS` / `CAR` / `CDR` for working with pairs, and the fundamental rule that any value other than `NIL` (or `#f` in Scheme) is true.

1.4.8 Symbol Names Limited to 8 Characters

MYLISP's namespace stores symbol names in slots of exactly **8 characters**. Any longer name is **silently truncated**. This limitation does not exist in any standard LISP dialect and is exclusive to the Sinclair QL environment, which the Next inherits.

```
MYLISP> (DEFINE FACTORIAL (LAMBDA (N) ...))
```

The name `FACTORIAL` has 9 characters. MYLISP stores it as `FACTORIA`. If we later try to call `FACTORIAL`, the interpreter won't find it — unless we write `FACTORIA`:

```
MYLISP> (FACTORIA 5)
120
```

The practical rule is: **always use names of 8 characters or fewer**. The examples in this manual follow this convention strictly.

1.4.9 Exiting the Interpreter: BYE

To end a MYLISP session, you type:

```
MYLISP> BYE
```

No parentheses. `BYE` is not a function; it is a special REPL keyword. If we wrote `(BYE)`, the interpreter would try to evaluate `BYE` as a function and would produce an undefined-symbol error.

1.4.10 `=` and `EQUAL` Are Not the Same

MYLISP has two equality predicates for numbers:

- `=` compares numerically, converting the operands to floating point. That's why `(= 1/2 0.5)` returns `T`.
- `EQUAL` compares structurally, type by type. `(EQUAL 1/2 0.5)` returns `NIL` because one is a rational and the other a real.

We'll look at this in detail in Chapter 4.

1.5 MYLISP as a Tool for Symbolic Algebra

One of MYLISP's reasons for existing is to demonstrate that an 8-bit computer — or nearly: the 68008 has an 8-bit data bus even though its internal architecture is 32-bit — can perform symbolic algebra without resorting to floating point.

Consider the problem of differentiating the expression `x² + 2x + 1` with respect to `x`. In a conventional language, this would require representing the expression in some structured form (perhaps a syntax tree) and walking it while applying differentiation rules. In LISP, the mathematical expression `x² + 2x + 1` can be represented directly as the list:

```
(+ (* x x) (* 2 x) 1)
```

And that list is an ordinary piece of data. A `DERIV` function in LISP can inspect it, recognize that the first element is `+`, and apply the rule that the derivative of a sum is the sum of the derivatives:

```
(DEFINE DERIV
  (LAMBDA (EXP VAR)
    (COND
      ((EQUAL EXP VAR) 1)
      ((ATOM EXP) 0)
      ((EQUAL (CAR EXP) (QUOTE +))
       (LIST (QUOTE +)
             (DERIV (CADR EXP) VAR)
             (DERIV (CADDR EXP) VAR)))
      ...)))
```

Chapter 10 develops a complete symbolic differentiator step by step, starting from the most basic concepts. But the example above already illustrates the key idea: in LISP, **code and data have the same structure**, and that makes symbolic manipulation feel natural.

1.6 Structure of This Book

This book is meant to be read chapter by chapter, though each one is self-contained enough to be consulted on its own.

Chapter 2 gets you started: how to boot MYLISP on the QL, how the REPL works, and how to type your first expressions. It also covers the most common mistakes of the first few minutes.

Chapter 3 explains the fundamental mental model: what S-expressions, atoms, and lists are, and the rules of evaluation. This chapter is the foundation for everything that follows.

Chapter 4 dives into numbers: integers, rationals, reals, the arithmetic operators, and the philosophy of exact arithmetic that sets MYLISP apart.

Chapter 5 covers working with lists: `CAR`, `CDR`, `CONS`, `LIST`, `APPEND`, and the equality predicates. Lists are the soul of LISP.

Chapter 6 explains control flow: `IF`, `COND`, `AND`, `OR`, `NOT`, and `PROGN`.

Chapter 7 shows how to define your own functions with `LAMBDA`, `DEFINE`, and `DEFUN`, introduces local variables with `LET`, explains recursion, and devotes a full section to lexical closures.

Chapter 8 covers text strings, the `PRINT` function, and loading programs from microdrive with `LOAD`.

Chapter 9 is a gallery of complete programs: factorial, Fibonacci, search, sorting, and pure-LISP implementations of `MAP`, `FILTER`, and `REDUCE`.

The **appendices** gather a quick reference for every function, the interpreter's internal architecture and limits, and the complete list of error messages.

1.7 Typographical Conventions

Throughout the manual the following conventions are used:

Text in `monospace font` represents LISP code or commands that must be typed literally. Interactive sessions are shown with the interpreter's prompt:

```
MYLISP> (+ 1 2)
3
```

The first line is what the user types. The second is the interpreter's response.

Comments in LISP code are preceded by a semicolon:

```
(+ 1 2)   ; this adds 1 and 2
```

When an example includes a definition that will be used later, it is marked with a header:

```
; --- Definition ---
(DEFINE SQUARE (LAMBDA (X) (* X X)))

; --- Usage ---
MYLISP> (SQUARE 5)
25
```

Warning! boxes flag limitations or behaviors that differ from other LISP dialects and could cause confusion.

> **Warning!** Symbol names are truncated to 8 characters. `FACTORIAL` becomes `FACTORIA`.

Note boxes provide supplementary information that isn't essential to follow the manual.

> **Note:** MYLISP is a Lisp-1, like Scheme. Functions and variables share the same namespace.

With all this in mind, you're ready to begin. In the next chapter we'll boot the interpreter on the Sinclair QL and write our first expressions.

Continues in Chapter 2: First Steps on the Sinclair QL

Chapter 2 — First Steps

2.1 What You Need Before You Start

2.1.1 On the Sinclair QL

- A Sinclair QL with at least **640 KB of RAM**. The base 128 KB configuration is not enough. Memory expansions from various manufacturers (Trump Card, Miracle Systems, etc.) exist that extend the QL to 640 KB or more.
- A **microdrive** with the MYLISP executable file. By convention the file is called `MYLISP` and lives on drive `mdv1_` or `mdv2_`, depending on where you copied it.
- A monitor or television connected to the QL.

If you don't have real hardware, Sinclair QL emulators exist for PC, Mac, and Linux (such as **QemuLator** or **sQLux**) that let you run MYLISP with full fidelity. On emulators, microdrives are emulated as disk images on the host system; check the emulator's documentation to learn how to transfer the MYLISP file to the image.

2.1.2 On the ZX Spectrum Next

- A ZX Spectrum Next (any edition: Issue 2, 2A, or 2B) with an up-to-date **NextZXOS**. MYLISP/Next requires no additional RAM expansion: the Next's paged memory is more than enough.
- The Next's **SD card** with the `MYLISP.BIN` file (or whatever name you gave it) copied to the root or to any folder.

If you don't have real hardware, the **CSpect** emulator (Windows/Mac/Linux) emulates the Next with high fidelity, including the SD card. It's the recommended option for development and testing.

2.2 Booting MYLISP

2.2.1 On the Sinclair QL

Turn on the QL. You'll see the QDOS boot menu. To launch MYLISP from SuperBASIC, type at the BASIC prompt:

```
EXEC_W mdv1_MYLISP
```

If the executable is on the second microdrive:

```
EXEC_w mdv2_MYLISP
```

After a moment the screen will clear and the interpreter's welcome message will appear, followed by the prompt:

```
MYLISP - LISP Interpreter for QL
Prospero Pro Pascal

>
```

> **Warning!** If the QL doesn't have enough memory, MYLISP may fail to boot or behave erratically. Check that the RAM expansion is correctly installed and recognized by QDOS before launching the interpreter.

2.2.2 On the ZX Spectrum Next

From the NextZXOS browser, navigate to the `MYLISP.NEX` file and press it to run it.

The leading dot is NextZXOS's prefix for running programs from the filesystem. After a moment the prompt will appear:

```
MYLISP - LISP Interpreter for NEXT
Version 1.0

>
```

The `>` symbol is the **REPL prompt**. It indicates that the interpreter is waiting for you to type an expression. It is identical on both platforms.

2.3 The REPL: Read, Eval, Print

MYLISP works in interactive mode through a **REPL** (*Read-Eval-Print Loop*). The cycle is as follows:

1. **Read:** the interpreter waits for the user to type an expression and press ENTER.

2. **Eval:** MYLISP evaluates the expression according to the rules of the language.
3. **Print:** the result is shown on screen.
4. **Repeat:** the prompt appears again.

Try the simplest possible expression: adding two numbers.

```
MYLISP> (+ 1 2)
3
MYLISP>
```

MYLISP evaluates `(+ 1 2)` and shows `3`. Notice the notation: the operator `+` comes **before** the operands, all inside parentheses. This is called **prefix notation** or **Polish notation**. It is LISP's universal syntax.

Another example, a multiplication:

```
MYLISP> (* 6 7)
42
MYLISP>
```

And a nested expression:

```
MYLISP> (+ (* 2 3) (* 4 5))
26
MYLISP>
```

MYLISP first evaluates `(* 2 3)` = 6 and `(* 4 5)` = 20, then adds the two results: 26.

2.4 Limits of the REPL

MYLISP's REPL has a few restrictions you should know about. The most important limit varies between platforms.

2.4.1 Line-Accumulation Limit

MYLISP lets you write expressions across **several lines**. If you open a parenthesis and press ENTER without closing it, the interpreter accumulates the input and waits for more:

```
MYLISP> (DEFINE SQUARE
         (LAMBDA (X)
           (* X X)))
```

The total accumulated length of the expression has a limit that depends on the platform:

> **QL:** the limit is **80 characters** total (the sum of all lines of a multi-line expression). If it's exceeded, the interpreter shows `ERROR: expression too long`. Longer expressions must be saved to a file and loaded with `LOAD`. **Next:** the limit is **500 characters**, both in the REPL and when loading with `LOAD`. This allows longer functions to be typed directly at the prompt.

On both platforms, once the buffer fills up the interpreter shows the error and discards the incomplete expression. Programs longer than a few lines should be saved to a file and loaded with `LOAD` regardless.

2.5 Your First Expressions

Let's explore the interpreter's basic capabilities with a series of progressive examples.

2.5.1 Basic Arithmetic

```
MYLISP> (+ 10 20 30)
60
```

MYLISP's arithmetic operators are **variadic**: they accept any number of arguments.

```
MYLISP> (* 2 3 4 5)
120
```

```
MYLISP> (- 100 25 10)
65
```

Subtraction with more than two arguments subtracts all the following ones from the first: `100 - 25 - 10 = 65`.

```
MYLISP> (/ 10 2)
5
```

```
MYLISP> (/ 1 3)
1/3
```

Division between integers produces a **rational** when it isn't exact. The result `1/3` is not a decimal approximation; it's the exact fraction one third.

2.5.2 Defining a Variable

```
MYLISP> (DEFINE PI 3.14159)
PI
MYLISP> (* PI (* 5 5))
78.53975
```

`DEFINE` associates a symbol with a value. After the definition, `PI` evaluates to the number `3.14159`.

2.5.3 Defining a Simple Function

> **Note:** the examples that follow use `LAMBDA`, `CONS`, `CAR`, `CDR`, and `NIL`. Don't worry if the syntax isn't entirely clear yet — Chapter 3 breaks down exactly how each piece works. For now, copy the commands, observe the result, and hold on to the general intuition of what they do.

```
MYLISP> (DEFINE SQUARE (LAMBDA (X) (* X X)))
SQUARE
MYLISP> (SQUARE 7)
49
MYLISP> (SQUARE 12)
144
```

`LAMBDA` creates an anonymous function. `DEFINE` gives it a name. We'll see this in detail in Chapter 7; for now, it's enough to understand the pattern.

2.5.4 Working with Lists

```
MYLISP> (CONS 1 (CONS 2 (CONS 3 NIL)))
(1 2 3)
```

`CONS` builds pairs. A chain of `CONS`es ending in `NIL` (the empty list) forms a list. Don't worry about the `NIL` for now: Chapter 3 explains exactly what it is and why it appears here.

```
MYLISP> (LIST 10 20 30)
(10 20 30)
```

`LIST` is a more convenient shorthand for building lists.

```
MYLISP> (CAR (LIST 10 20 30))
10
MYLISP> (CDR (LIST 10 20 30))
(20 30)
```

`CAR` returns the first element. `CDR` returns the rest.

2.6 Common Errors and How to Read Them

In the first few minutes with the interpreter it's common to make certain mistakes. Here are the most frequent ones and what they mean.

2.6.1 Undefined Symbol

```
MYLISP> FACTORIAL
ERROR: undefined symbol: FACTORIA
```

This happens when you type a symbol that hasn't been defined before. Notice that MYLISP shows `FACTORIA`, not `FACTORIAL` — because it truncated the name to 8 characters before looking it up.

2.6.2 Unclosed Parentheses

```
MYLISP> (+ 1 2

```

If you press ENTER with open parentheses, the interpreter waits for more input. The prompt changes (`..` appears) to indicate the expression is incomplete. Type the closing parenthesis and press ENTER:

```
MYLISP> (+ 1 2
..         )
3
```

2.6.3 Division by Zero

```
MYLISP> (/ 5 0)
ERROR: division by zero
```

2.6.4 Applying CAR to NIL

```
MYLISP> (CAR NIL)
ERROR: CAR requires a non-empty list
```

2.6.5 Integer Overflow

```
MYLISP> (* 1000000 1000000)
ERROR: integer overflow.
Use real numbers (e.g. 1.0) if you need a larger range.
```

MYLISP does not silently convert integers to floating point. If the result doesn't fit in an integer, it produces this error. To work with large numbers, use real-number literals:

```
MYLISP> (* 1000000.0 1000000.0)
1.0E12
```

2.7 How Internal Memory Works

MYLISP manages memory through a **cell table** (*heap*). Each cell can store a pair of values (LISP's basic unit, called a *cons cell*). Lists, trees, closures, and evaluation environments are all built out of these cells.

> **QL:** the heap has **24,000 cells**. **Next:** the heap has **32,000 cells**, stored in paged memory outside the Z80's 64 KB address space.

Besides the heap, the interpreter maintains a symbol table (200 entries, names truncated to 8 characters) and a string table (50 entries, 36 characters maximum). The QL version additionally has a table of reals for up to 100 floating-point numbers; in the Next version, floats are packed directly inside each cell.

When the heap fills up, the interpreter automatically runs the **garbage collector** (GC), using the *mark-and-sweep* algorithm. The GC is completely silent; the user doesn't notice it. The trigger threshold varies:

> **QL:** 80% in the REPL and 50% during `LOAD`. **Next:** 80% in the REPL. During `LOAD`, the GC **does not trigger automatically**: the 32,000-cell heap is generous, and it's assumed a single file won't exhaust it. If you want to force a collection between loads, use `(CLEAN)` from the REPL — not from inside a file — between one load and the next.

> **Note:** the GC never activates in the middle of an evaluation, only between complete expressions. This guarantees the collector never corrupts structures that are being built at that moment.

2.8 Exiting the Interpreter

To end the session, type:

```
MYLISP> BYE
```

No parentheses. Control returns to the QDOS operating system.

> **Warning!** Don't type `(BYE)`. MYLISP will try to evaluate `BYE` as a function and will produce the error `ERROR: undefined symbol: BYE`.

> **BYE erases everything — no exceptions:** when you type `BYE`, MYLISP completely destroys the session's memory. Every function defined in the REPL, every variable, all accumulated work disappears with no way to recover it. MYLISP does not save state automatically. The only way to preserve your work is to **always write it to a file** and load it with `LOAD` at the start of each session. The REPL is for experimenting; files are for saving. Build this habit from day one.

2.9 A Complete Example Session

To close this chapter, here's a complete session showing the typical MYLISP workflow: defining a few functions, testing them, and exiting.

```
MYLISP - LISP Interpreter for Sinclair QL
Prospero Pro Pascal
```

```
MYLISP> (DEFINE SQUARE (LAMBDA (X) (* X X)))
SQUARE
MYLISP> (DEFINE CUBE (LAMBDA (X) (* X X X)))
CUBE
MYLISP> (SQUARE 5)
25
MYLISP> (CUBE 3)
27
MYLISP> (+ (SQUARE 3) (SQUARE 4))
25
MYLISP> (DEFINE HYPOTENUSE
          (LAMBDA (A B)
            (SQUARE (+ (SQUARE A) (SQUARE B)))))
HYPOTENUSE
MYLISP> (HYPOTENUSE 3 4)
625
```

Wait — `625` is not the hypotenuse of the 3-4-5 triangle. We squared it twice! The correct function should use the square root. But `SQRT` is not implemented in MYLISP. For the CAS this isn't a problem — we work with exact symbolic expressions, not decimal approximations — but if you need the numeric square root, you'll have to approximate it with Newton's method, which we'll see in Chapter 9.

The correct session would have been:

```
MYLISP> (DEFINE HYPOT2
          (LAMBDA (A B)
            (+ (* A A) (* B B))))
HYPOT2
MYLISP> (HYPOT2 3 4)
25
```

`25` is the square of the hypotenuse. The hypotenuse is `√25 = 5`, but we know that; the interpreter gives us the exact value with no approximation.

```
MYLISP> BYE
```

Continues in Chapter 3: The LISP Data Model

Chapter 3 — The LISP Data Model

3.1 Everything Is an S-Expression

In LISP, **everything** — both data and code — has the same shape: the **S-expression** (*Symbolic Expression*). This concept, introduced by McCarthy in 1958, is the single most important idea in the whole language. Whoever understands it well understands LISP.

An S-expression is, recursively, one of two things:

1. An **atom**: the indivisible unit of data.
2. A **pair**: two S-expressions joined together, written as `(A . B)`.

Lists — LISP's fundamental data structure — are built out of nested pairs.

3.2 Atoms

An atom is any value that cannot be split into smaller parts. MYLISP has five kinds of atom:

Symbols. Identifiers that represent names. Used to name variables, functions, and constants.

```
X
SUM
SQUARE
NIL
T
```

Integers. Whole numbers with no decimal part.

```
0
42
-17
1000
```

Rationals. Exact fractions represented as numerator/denominator.

```
1/2
3/7
-5/3
```

MYLISP automatically reduces fractions: `(/ 4 6)` returns `2/3`, not `4/6`.

Reals. Floating-point (approximate) numbers.

```
3.14159
-2.718
1.0E10
```

Strings. Text between double quotes (limited to 36 characters).

```
"hello"
"result:"
"MYLISP 1.0"
```

The two most special atoms are `NIL` and `T`:

- `NIL` represents both the **empty list** and the value **false**. It is the only value LISP treats as false in conditions.
- `T` represents the value **true**. However, in MYLISP, **any value other than `NIL` is true** in a conditional context.

3.3 Pairs and Lists

The pair `(A . B)` — called a *cons cell* or *dotted pair* — is the building block of lists. The left-hand element is called `CAR` and the right-hand one `CDR` (names inherited from the IBM 704, the machine the first LISP was implemented on).

A list `(1 2 3)` is, in reality, a chain of nested pairs:

```
(1 . (2 . (3 . NIL)))
```

Which can be pictured like this:

```
  [1 | •]──>[2 | •]──>[3 | NIL]
```

Each box represents a cons cell. The left side holds the value (`1`, `2`, `3`) and the right side points to the next cell. The last cell points to `NIL`, marking the end of the list.

When MYLISP prints a list, it shows the abbreviated notation `(1 2 3)` instead of `(1 . (2 . (3 . NIL)))`. Internally, however, both representations are identical.

We can build pairs and lists explicitly with `CONS`:

```
MYLISP> (CONS 1 NIL)
(1)
MYLISP> (CONS 1 (CONS 2 NIL))
(1 2)
MYLISP> (CONS 1 (CONS 2 (CONS 3 NIL)))
(1 2 3)
```

Or more conveniently with `LIST`:

```
MYLISP> (LIST 1 2 3)
(1 2 3)
```

3.3.1 Nested Lists

Lists can contain other lists as elements:

```
MYLISP> (LIST (LIST 1 2) (LIST 3 4))
((1 2) (3 4))
```

Internally, this is:

```
((1 2) . ((3 4) . NIL))
```

A nested list is simply a pair whose `CAR` is another list.

3.3.2 The Empty List

`NIL` and `()` are the same thing: the empty list.

```
MYLISP> NIL
NIL
MYLISP> (LIST)
NIL
```

3.4 How MYLISP Evaluates Expressions

When you type an expression at the REPL, MYLISP follows a set of rules to evaluate it. These rules are simple but powerful.

3.4.1 Rule 1: Self-Evaluating Atoms

Numeric literals and strings evaluate to themselves:

```
MYLISP> 42
42
MYLISP> 3.14
3.14
MYLISP> 1/3
1/3
MYLISP> "hello"
"hello"
```

`NIL` and `T` are also self-evaluating:

```
MYLISP> NIL
NIL
MYLISP> T
T
```

3.4.2 Rule 2: Symbols Look Up Their Value in the Environment

A symbol evaluates to the value bound to it in the current environment. If it has no value, that's an error.

```
MYLISP> (DEFINE X 10)
X
MYLISP> X
10
MYLISP> Y
ERROR: undefined symbol: Y
```

3.4.3 Rule 3: Lists Are Function Calls (or Special Forms)

A list `(F A B C ...)` is evaluated by first evaluating `F`, then the arguments `A`, `B`, `C`... and finally applying the resulting function to the evaluated arguments.

```
MYLISP> (+ 1 2)
```

1. `+` is evaluated → the primitive addition function.
2. `1` is evaluated → `1`.
3. `2` is evaluated → `2`.
4. The addition is applied: `1 + 2 = 3`.

```
MYLISP> (+ (* 2 3) (- 10 4))
```

1. `+` is evaluated → addition.
2. `(* 2 3)` is evaluated → `6`.
3. `(- 10 4)` is evaluated → `6`.
4. The addition is applied: `6 + 6 = 12`.

3.4.4 Exception: Special Forms Don't Follow Rule 3

Some language constructs — called **special forms** — do not automatically evaluate all of their arguments. For example, `IF` only evaluates the branch that applies:

```
MYLISP> (IF T 1 (/ 1 0))
1
```

The expression `(/ 1 0)` is never evaluated (which would otherwise cause a division-by-zero error) because the condition `T` is true and only the "true" branch is evaluated. MYLISP's special forms are: `QUOTE`, `IF`, `COND`, `AND`, `OR`, `PROGN`, `LET`, `LAMBDA`, `DEFINE`, `DEFUN`, and `LOAD`.

3.5 QUOTE: Stopping Evaluation

Sometimes we want MYLISP to treat an expression as a **piece of data** rather than evaluate it. That's what `QUOTE` is for:

```
MYLISP> (QUOTE (+ 1 2))
(+ 1 2)
```

Without `QUOTE`, `(+ 1 2)` would evaluate to `3`. With `QUOTE`, it returns the list as-is: the three elements `+`, `1`, and `2`.

`QUOTE` has a shorthand form: the apostrophe `'`:

```
MYLISP> '(+ 1 2)
(+ 1 2)
MYLISP> '(a b c)
(A B C)
MYLISP> 'hello
HELLO
```

Notice that symbols are shown in uppercase — MYLISP converts every symbol to uppercase internally.

`QUOTE` is essential for building literal data:

```
MYLISP> (DEFINE COLORS '(RED GREEN BLUE))
COLORS
MYLISP> COLORS
(RED GREEN BLUE)
MYLISP> (CAR COLORS)
RED
MYLISP> (CDR COLORS)
(GREEN BLUE)
```

Without `QUOTE`, `(DEFINE COLORS (RED GREEN BLUE))` would try to evaluate `RED` as a function, which would produce an error.

3.6 The Principle of Homoiconicity

The deepest consequence of LISP's data model is that **code has the same structure as data**. The expression `(+ 1 2)` is, at the same time:

- A program that computes `3` when evaluated.
- A three-element list `(+, 1, 2)` that can be inspected and manipulated like any other list.

This means we can write programs that **generate** and **run** other programs. The `EVAL` function applies the evaluation rules to any list:

```
MYLISP> (EVAL '(+ 1 2))
3
MYLISP> (EVAL (LIST '+ 10 20))
30
```

In the second example, we build the list `(+ 10 20)` at run time with `LIST` and `QUOTE`, and then evaluate it with `EVAL`. The result is exactly the same as if we had written `(+ 10 20)` directly.

This capability — building code as data and then running it — is the foundation of the CAS. The mathematical expression `x² + 2x + 1` is represented as the list `(+ (* X X) (* 2 X) 1)`, and a differentiation function transforms it into `(+ (* 2 X) 2)` by applying the rules of calculus to the structure of the list.

3.7 Type Predicates

MYLISP includes functions for asking what type a value is. They're called **predicates** because they return `T` or `NIL`.

```
MYLISP> (ATOM 42)
T
MYLISP> (ATOM '(1 2 3))
NIL
MYLISP> (ATOM NIL)
T
```

`ATOM` returns `T` for any atom, including `NIL`. It returns `NIL` for pairs (lists).

```
MYLISP> (NUMBERP 42)
T
MYLISP> (NUMBERP 3.14)
T
MYLISP> (NUMBERP 1/2)
T
MYLISP> (NUMBERP 'X)
NIL
```

`NUMBERP` returns `T` for any number (integer, rational, or real).

```
MYLISP> (SYMBOLP 'HELLO)
T
MYLISP> (SYMBOLP 42)
NIL
```

```
MYLISP> (LISTP '(1 2 3))
T
MYLISP> (LISTP NIL)
T
MYLISP> (LISTP 42)
NIL
```

`LISTP` returns `T` for lists, including the empty list `NIL`.

```
MYLISP> (NULL NIL)
T
MYLISP> (NULL '(1 2))
NIL
MYLISP> (NULL 0)
NIL
```

`NULL` returns `T` only for `NIL`. Note that `0` is not `NIL`: in MYLISP, as in every LISP dialect, the only false value is `NIL`.

3.8 Equality: EQ, =, and EQUAL

MYLISP has three ways to compare values, and it's important to understand the differences.

3.8.1 EQ: Symbol Identity

`EQ` checks whether two objects are **the exact same symbol**, or both are `NIL`. It's the most primitive comparison.

```
MYLISP> (EQ 'A 'A)
T
MYLISP> (EQ 'A 'B)
NIL
MYLISP> (EQ NIL NIL)
T
MYLISP> (EQ '(1 2) '(1 2))
NIL
```

The last example returns `NIL` because the two lists `(1 2)`, although equal in content, are distinct objects in memory. `EQ` checks identity, not content equality.

3.8.2 =: Numeric Equality

`=` compares numeric values by converting both operands to floating point internally. That's why it can compare integers with reals and with rationals:

```
MYLISP> (= 1 1)
T
MYLISP> (= 1 1.0)
T
MYLISP> (= 1/2 0.5)
T
MYLISP> (= 2 3)
NIL
```

Useful when we want numeric equality without worrying about the specific type.

3.8.3 EQUAL: Structural Equality

`EQUAL` recursively compares the structure and type of two expressions. It's the most precise comparison.

```
MYLISP> (EQUAL '(1 2 3) '(1 2 3))
T
MYLISP> (EQUAL '(1 2) '(1 3))
NIL
MYLISP> (EQUAL 1 1.0)
NIL
MYLISP> (EQUAL 1/2 0.5)
NIL
```

The last two examples return `NIL` because, although numerically equal, `1` is an integer and `1.0` is a real; `1/2` is a rational and `0.5` is a real. `EQUAL` distinguishes types.

> **Summary:** use `EQ` to compare symbols; use `=` to compare numbers regardless of type; use `EQUAL` to compare lists, or whenever type matters.

3.9 Building and Taking Apart Lists

The fundamental list operations are `CAR`, `CDR`, and `CONS`. Everything else is built out of them.

3.9.1 CAR and CDR

`CAR` returns the first element of a list (the left-hand element of the first pair):

```
MYLISP> (CAR '(10 20 30))
10
MYLISP> (CAR '((A B) C D))
(A B)
```

`CDR` returns the rest of the list (everything except the first element):

```
MYLISP> (CDR '(10 20 30))
(20 30)
MYLISP> (CDR '(10))
NIL
```

Combining `CAR` and `CDR` we can reach any element:

```
MYLISP> (CAR (CDR '(10 20 30)))
20
MYLISP> (CAR (CDR (CDR '(10 20 30))))
30
```

`(CAR (CDR list))` reaches the second element; `(CAR (CDR (CDR list)))` the third. The combination is so common it's abbreviated as `CADR` and `CADDR`, though in MYLISP these abbreviations aren't predefined as primitives — you'll have to define them yourself if you need them:

```
(DEFINE CADR (LAMBDA (L) (CAR (CDR L))))
(DEFINE CADDR (LAMBDA (L) (CAR (CDR (CDR L)))))
```

3.9.2 CONS

`CONS` builds a new pair. Given an element `A` and a list `L`, it returns a new list with `A` as the first element followed by the elements of `L`:

```
MYLISP> (CONS 0 '(1 2 3))
(0 1 2 3)
MYLISP> (CONS 'X '(Y Z))
(X Y Z)
MYLISP> (CONS '(1 2) '(3 4))
((1 2) 3 4)
```

`CONS` does not modify the original list; it creates a new structure. This property — the **immutability** of LISP structures — is what makes functional programs safe: sharing structures between variables produces no side effects.

3.9.3 APPEND

`APPEND` concatenates two lists:

```
MYLISP> (APPEND '(1 2) '(3 4))
(1 2 3 4)
MYLISP> (APPEND '(A B) '(C D) )
```

> **Note:** in MYLISP, `APPEND` accepts exactly two lists. It is not variadic.

```
MYLISP> (APPEND '(A B) '(C D))
(A B C D)
```

3.10 Lists as a Universal Structure

Lists are flexible enough to represent practically any data structure:

Stack: the front of the list is the top.

```
(DEFINE STACK NIL)
(DEFINE PUSH (LAMBDA (ELEM S) (CONS ELEM S)))
(DEFINE POP  (LAMBDA (S) (CDR S)))
(DEFINE TOP  (LAMBDA (S) (CAR S)))
```

Key-value pair (association list, or a-list): a list of `(key . value)` pairs.

```
MYLISP> (DEFINE ADDRESS-BOOK
          '((ANN . 555-1234)
            (LOU . 555-5678)
            (MARY . 555-9012)))
ADDRESS-BOOK
```

Tree: a list of nested lists.

```
MYLISP> (DEFINE TREE '(ROOT (LEFT A B) (RIGHT C D)))
TREE
MYLISP> (CAR TREE)
ROOT
MYLISP> (CADR TREE)   ; needs CADR defined
(LEFT A B)
```

In the coming chapters we'll see how these structures are used in real programs.

Continues in Chapter 4: Numbers and Exact Arithmetic

Chapter 4 — Numbers and Exact Arithmetic

4.1 MYLISP's Three Numeric Types

MYLISP distinguishes three kinds of numbers, and each has well-defined behavior. Understanding the differences is essential, especially if you use MYLISP for symbolic algebra, where exactness matters more than speed.

4.1.1 Integers

Whole numbers with no decimal part, positive or negative. In MYLISP, integers are **fixed-precision, 32-bit signed**, with a range from −2,147,483,647 to +2,147,483,647.

```
MYLISP> 0
0
MYLISP> 42
42
MYLISP> -1000
-1000
MYLISP> 32767
32767
```

If the result of an operation exceeds this range, MYLISP produces an error instead of silently converting the value to floating point. This is a deliberate design decision, explained in section 4.4.

4.1.2 Rationals

A rational is an exact fraction expressed as `numerator/denominator`. MYLISP accepts rationals directly as literals:

```
MYLISP> 1/2
1/2
MYLISP> 3/4
3/4
MYLISP> -2/5
-2/5
```

Rationals are **automatically** reduced to their greatest common divisor. There's no need to reduce them by hand:

```
MYLISP> (/ 4 6)
2/3
MYLISP> (/ 10 5)
2
MYLISP> (+ 1/3 1/6)
1/2
```

Notice that `(/ 10 5)` returns the integer `2`, not the rational `2/1`. MYLISP simplifies the result to the simplest possible type.

4.1.3 Reals

Reals are single-precision floating-point numbers. They're written with a decimal point or in scientific notation:

```
MYLISP> 3.14159
3.14159
MYLISP> -2.718
-2.718
MYLISP> 1.0E10
1.0E10
MYLISP> 0.001
0.001
```

Reals are approximate: `(+ 0.1 0.2)` may not come out to exactly `0.3`. For symbolic and algebraic computation, always use integers and rationals.

> **Note:** MYLISP stores up to 100 distinct real numbers in an internal table. This table is shared among every real value created during the session. If you need to create many different real values, you can exhaust this table.

4.2 Arithmetic Operators

The four basic operators are `+`, `-`, `*`, and `/`. All of them are **variadic**: they accept one or more arguments.

4.2.1 Addition

```
MYLISP> (+ 1 2)
3
MYLISP> (+ 1 2 3 4 5)
15
MYLISP> (+ 1/2 1/3)
5/6
MYLISP> (+ 1 0.5)
1.5
```

When the operands are of mixed types, MYLISP applies the following promotion hierarchy: integer → rational → real. The result takes on the "widest" type present among the operands.

```
MYLISP> (+ 1 1/2)
3/2
MYLISP> (+ 1/2 0.5)
1.0
```

4.2.2 Subtraction

With two or more arguments, it subtracts the second and following ones from the first:

```
MYLISP> (- 10 3)
7
MYLISP> (- 10 3 2)
5
```

With a **single argument**, it returns the negation (sign flip):

```
MYLISP> (- 5)
-5
MYLISP> (- -3)
3
MYLISP> (- 1/4)
-1/4
```

4.2.3 Multiplication

```
MYLISP> (* 3 4)
12
MYLISP> (* 2 3 4 5)
120
MYLISP> (* 2 1/3)
2/3
MYLISP> (* 1/2 1/3)
1/6
```

4.2.4 Division

With two or more arguments, it divides the first by the second and following ones:

```
MYLISP> (/ 10 2)
5
MYLISP> (/ 1 3)
1/3
MYLISP> (/ 12 4 3)
1
MYLISP> (/ 3.0 2)
1.5
```

With a **single argument**, it returns the reciprocal:

```
MYLISP> (/ 2)
1/2
MYLISP> (/ 4)
1/4
MYLISP> (/ 3.0)
0.33333...
```

> **Warning!** Division between integers always produces a rational when it isn't exact. To get a real result, use at least one real operand: `(/ 1.0 3)` returns `0.33333...`; `(/ 1 3)` returns `1/3`.

4.2.5 Division by Zero

Any attempt to divide by zero produces an immediate error:

```
MYLISP> (/ 5 0)
ERROR: division by zero
MYLISP> (/ 1 0)
ERROR: division by zero
```

4.3 Numeric Comparisons

MYLISP includes the usual six comparison operators. All of them return `T` or `NIL`.

```
MYLISP> (= 3 3)
T
MYLISP> (= 3 4)
NIL
MYLISP> (< 2 5)
T
MYLISP> (> 10 3)
T
MYLISP> (<= 4 4)
T
MYLISP> (>= 5 3)
T
```

All of them accept a mix of numeric types, converting internally to real for the comparison:

```
MYLISP> (< 1/3 0.5)
T
MYLISP> (= 1/2 0.5)
T
```

4.4 The Philosophy of Exact Arithmetic

MYLISP's most striking design decision is its **refusal to automatically promote integers to reals**. In most languages, when an operation between integers produces a result out of range, the system silently converts the value to floating point. Not in MYLISP:

```
MYLISP> (* 1000 1000)
ERROR: integer overflow.
Use real numbers (e.g. 1.0) if you need a larger range.
```

Why this behavior? The reason is MYLISP's calling as a CAS engine. In symbolic algebra, the difference between an integer and a real is not trivial. Consider the fraction `1/3`:

- As a rational: `1/3` (exact, with no loss of information).
- As a real: `0.33333...` (a finite-precision approximation).

If MYLISP automatically converted `1/3` to `0.33333...`, subsequent algebraic operations would accumulate rounding errors and the CAS would stop being trustworthy. Exact arithmetic is the price of algebraic correctness.

What do you do when you need large numbers? If the problem requires numbers that exceed the range of 32-bit integers (more than ±2.147 million), use reals from the start. But be aware that you'll lose exactness:

```
MYLISP> (* 1000.0 1000.0)
1.0E6
```

For most symbolic-algebra computations (derivatives, simplifications, manipulating polynomials with small coefficients), 32-bit integers are more than enough.

4.5 Rationals: The Exact Fraction

Rationals are perhaps MYLISP's most useful feature for algebra. They let you represent any rational number with **total exactness**, with no approximation.

Let's see how they behave in operations:

```
MYLISP> (+ 1/4 1/4)
1/2
MYLISP> (+ 1/3 1/6)
1/2
MYLISP> (* 2/3 3/4)
1/2
MYLISP> (- 1 1/3)
2/3
```

All operations between rationals produce rationals (or integers when the result is exact).

Automatic reduction guarantees the result is always in its simplest form:

```
MYLISP> (+ 1/6 1/6 1/6)
1/2
MYLISP> (* 6/7 7/6)
1
```

Rationals compare correctly with the numeric operators:

```
MYLISP> (< 1/3 1/2)
T
MYLISP> (= 2/4 1/2)
T
```

4.6 The Crucial Difference Between `=` and `EQUAL`

The full distinction between `=`, `EQ`, and `EQUAL` is explained in section 3.8 with detailed examples. In short: `=` compares numerically (converting to floating point), `EQUAL` compares structure and type, and `EQ` compares object identity in memory.

4.7 Mixed Operations and the Type Hierarchy

When an operation mixes types, MYLISP applies the following rule: the result takes on the type of the "widest" operand according to the hierarchy:

```
integer < rational < real
```

Some examples:

```
MYLISP> (+ 1 1/2)
3/2        ; integer + rational → rational
MYLISP> (+ 1 0.5)
1.5        ; integer + real → real
MYLISP> (+ 1/2 0.5)
1.0        ; rational + real → real
MYLISP> (+ 1 1/2 0.1)
1.6        ; the real dominates over everything
```

This hierarchy reflects "loss of information": once a real appears, the result can no longer be exact.

4.8 Practical Examples

4.8.1 Computing an Exact Average

```
MYLISP> (/ (+ 1 2 3 4 5) 5)
3
```

The average of 1 through 5 is exactly 3.

```
MYLISP> (/ (+ 1 2 3 4) 4)
5/2
```

The average of 1 through 4 is exactly `5/2`. No rounding.

4.8.2 Verifying an Algebraic Identity

Does `(a/b) * (b/a) = 1` hold for concrete values?

```
MYLISP> (* 3/7 7/3)
1
MYLISP> (* 5/11 11/5)
1
```

Exact rational arithmetic confirms the identity with no margin of error.

4.8.3 A Polynomial with Rational Coefficients

Evaluate `P(x) = x² + (1/2)x − 3/4` at `x = 1/2`:

```
(DEFINE P
  (LAMBDA (X)
    (+ (* X X)
       (* 1/2 X)
       (- 3/4))))

MYLISP> (P 1/2)
```

Let's compute: `(1/2)² + (1/2)(1/2) − 3/4 = 1/4 + 1/4 − 3/4 = −1/4`

```
MYLISP> (P 1/2)
-1/4
```

The exact result: `-1/4`. Not `-0.25` or `-0.24999...`.

4.8.4 Summing a Series

Approximating $\pi/4$ with the Leibniz series: $\pi/4 = 1 - 1/3 + 1/5 - 1/7 + \ldots$

```
MYLISP> (+ 1 (- 1/3) 1/5 (- 1/7) 1/9 (- 1/11))
341/429
```

$341/429 \approx 0.7948\ldots$, while $\pi/4 \approx 0.7854$. The approximation improves with more terms. Rational arithmetic guarantees the sum of the fractions is exact.

4.9 Integer Division: MOD and DIV

When working with integers, we sometimes need the **quotient** and the **remainder** of a division, not the exact result. For that, MYLISP offers two dedicated primitives that only operate on integers (`TINT`).

4.9.1 DIV: Integer Quotient

`DIV` returns the integer part of the quotient, truncating toward zero:

```
MYLISP> (DIV 17 5)
3
MYLISP> (DIV 10 3)
3
MYLISP> (DIV 15 5)
3
MYLISP> (DIV -7 3)
-2
```

Note that `(DIV -7 3)` returns `-2` (it truncates toward zero), not `-3`.

4.9.2 MOD: Integer Remainder

`MOD` returns the remainder of the integer division. The sign of the result matches the sign of the dividend:

```
MYLISP> (MOD 17 5)
2
MYLISP> (MOD 10 3)
1
MYLISP> (MOD 15 5)
0
MYLISP> (MOD -7 3)
-1
```

The relationship between `/`, `DIV`, and `MOD` is always: `A = (+ (* (DIV A B) B) (MOD A B))`.

4.9.3 DIV and MOD Only Accept Integers

If you pass a rational or a real, an error is produced. To compute the remainder of a rational division, use `/` directly:

```
MYLISP> (DIV 1/2 3)
ERROR: ...
MYLISP> (/ 1 2)
1/2        ; this is the correct thing to do for rationals
```

4.9.4 Immediate Applications

Checking whether a number is even or odd:

```
(DEFUN EVENP (N) (= 0 (MOD N 2)))
(DEFUN ODDP  (N) (NOT (EVENP N)))
```

```
MYLISP> (EVENP 4)
T
MYLISP> (ODDP 7)
T
```

Computing the greatest common divisor (Euclid's algorithm):

```
(DEFUN GCD (A B)
  (IF (= B 0) A (GCD B (MOD A B))))
```

```
MYLISP> (GCD 48 18)
6
MYLISP> (GCD 100 75)
25
```

Continues in Chapter 5: Working with Lists

Chapter 5 — Working with Lists

5.1 Your First Library: The Utilities File

Before diving into list operations, we need to solve a practical problem: MYLISP doesn't include, as primitives, some convenience functions we'll use constantly — `CADR`, `CADDR`, `LENGTH`, `ABS`, and others. Every time you start MYLISP you begin with a completely empty environment: the previous session's definitions aren't kept.

The solution is to create a **utilities file** that you load at the start of every working session. This is the standard technique in every resource-constrained LISP environment: you build your own "standard library" and keep it ready on the storage medium.

5.1.1 Creating the Utilities File

The file's name depends on your platform:

> **QL:** save it on the microdrive as `mdv1_utils` (using Quill or another QL editor). It's loaded with `(LOAD "mdv1_utils")`. **Next:** save it on the SD card as `utils.lsp` (or whatever name you prefer). It's loaded with `(LOAD "utils.lsp")`.

The code is identical on both platforms:

```
; utils — Utility functions for MYLISP
; QL:   (LOAD "mdv1_utils")
; Next: (LOAD "utils.lsp")

; List element access
(DEFUN CADR   (L) (CAR (CDR L)))
(DEFUN CADDR  (L) (CAR (CDR (CDR L))))
(DEFUN CADDDR (L) (CAR (CDR (CDR (CDR L)))))

; List length
(DEFUN LENGTH (L)
  (IF (NULL L) 0 (+ 1 (LENGTH (CDR L)))))

; Nth element (0-based)
(DEFUN NTH (L N)
  (IF (= N 0) (CAR L) (NTH (CDR L) (- N 1))))

; Absolute value
(DEFUN ABS (X) (IF (< X 0) (- X) X))
```

```
; Symbol comparators
(DEFUN SYM< (A B) (STR< A B))

; Maximum and minimum
(DEFUN MAXI (A B) (IF (> A B) A B))
(DEFUN MINI (A B) (IF (< A B) A B))

; GCD (Euclid)
(DEFUN GCD (A B) (IF (= B 0) A (GCD B (MOD A B))))
(DEFUN LCM (A B) (/ (* A B) (GCD A B)))

(PRINT "utils loaded")
```

5.1.2 Loading It at Startup

Every time you start a working session, load the file for your platform:

```
MYLISP> (LOAD "mdv1_utils")   ; QL
MYLISP> (LOAD "utils.lsp")    ; Next
utils loaded
```

From that point on, `CADR`, `CADDR`, `LENGTH`, `ABS`, and the rest will be available for the whole session. **The examples from this chapter onward assume you've loaded the utilities file.**

> **Core habit:** always write your code in files, not just at the REPL. When you type `BYE` to quit, MYLISP **erases all of memory's work** without saving anything. If you define functions directly at the REPL and then exit, you lose them forever. The right workflow is: edit in a file → load with `LOAD` → test at the REPL → go back to the editor to fix things.

5.2 Lists Are the Heart of LISP

The name LISP stands for *LISt Processing*. That's no accident. Lists are the language's fundamental data structure: they're used to represent programs, data, algebraic expressions, derivation trees, variable environments... practically everything.

This chapter explores in depth how to build, take apart, inspect, and transform lists in MYLISP.

5.3 Building Lists

We already know the three basic ways to build lists. Let's look at them in more detail.

5.3.1 CONS

`CONS` (from *CONStruct*) creates a new pair made up of an element and an existing list. The element becomes the new first element of the resulting list.

```
MYLISP> (CONS 1 NIL)
(1)
MYLISP> (CONS 1 '(2 3))
(1 2 3)
MYLISP> (CONS 'A '(B C D))
(A B C D)
MYLISP> (CONS '(1 2) '(3 4))
((1 2) 3 4)
```

`CONS` never modifies its second argument. It always creates a new structure that shares the inside of the existing list. This is both efficient and safe.

We can build any list by chaining `CONS`:

```
MYLISP> (CONS 1 (CONS 2 (CONS 3 (CONS 4 NIL))))
(1 2 3 4)
```

5.3.2 LIST

`LIST` is a shortcut for building lists out of known elements:

```
MYLISP> (LIST 1 2 3)
(1 2 3)
MYLISP> (LIST 'A 'B 'C)
(A B C)
MYLISP> (LIST 1 '(2 3) 4)
(1 (2 3) 4)
MYLISP> (LIST)
NIL
```

`(LIST)` with no arguments returns `NIL`, the empty list.

5.3.3 APPEND

`APPEND` concatenates two lists:

```
MYLISP> (APPEND '(1 2) '(3 4))
(1 2 3 4)
MYLISP> (APPEND '(A B C) '(D))
(A B C D)
MYLISP> (APPEND NIL '(1 2))
(1 2)
MYLISP> (APPEND '(1 2) NIL)
(1 2)
```

`APPEND` doesn't modify either of the original lists; it builds a new list containing all the elements of the first followed by those of the second.

> **Warning!** In MYLISP, `APPEND` accepts exactly **two** lists. It is not variadic. To concatenate three lists, nest the calls: `(APPEND '(1 2) (APPEND '(3 4) '(5 6)))` → `(1 2 3 4 5 6)`.

5.4 Accessing Elements

5.4.1 CAR and CDR

`CAR` returns the first element; `CDR` returns the rest.

```
MYLISP> (CAR '(10 20 30))
10
MYLISP> (CDR '(10 20 30))
(20 30)
MYLISP> (CAR '((A B) C D))
(A B)
MYLISP> (CDR '((A B) C D))
(C D)
```

Both functions raise an error if applied to `NIL`:

```
MYLISP> (CAR NIL)
ERROR: CAR requires a non-empty list
MYLISP> (CDR NIL)
ERROR: CDR requires a non-empty list
```

5.4.2 Combinations of CAR and CDR

To reach elements beyond the first, we combine `CAR` and `CDR`:

Expression	Meaning
`(CAR L)`	first element
`(CAR (CDR L))`	second element
`(CAR (CDR (CDR L)))`	third element
`(CDR (CDR L))`	list from the third element onward

Example with the list `(A B C D)`:

```
MYLISP> (CAR '(A B C D))
A
MYLISP> (CAR (CDR '(A B C D)))
B
MYLISP> (CAR (CDR (CDR '(A B C D))))
C
```

For convenience, we can define the missing abbreviations:

```
(DEFINE CADR   (LAMBDA (L) (CAR (CDR L))))
(DEFINE CADDR  (LAMBDA (L) (CAR (CDR (CDR L)))))
(DEFINE CADDDR (LAMBDA (L) (CAR (CDR (CDR (CDR L))))))
```

Now:

```
MYLISP> (CADR '(A B C D))
B
MYLISP> (CADDR '(A B C D))
C
MYLISP> (CADDDR '(A B C D))
D
```

> **Warning!** `CADDDR` has 7 characters: it just fits within the 8-character limit. But `CADDDDR` would have exactly 8 characters. Be careful with long combination names.

5.5 List Predicates

5.5.1 NULL

`NULL` returns `T` if its argument is the empty list (`NIL`), and `NIL` otherwise. It's essential for controlling termination of recursion over lists.

```
MYLISP> (NULL NIL)
T
MYLISP> (NULL '())
T
MYLISP> (NULL '(1 2))
NIL
MYLISP> (NULL 0)
NIL
```

> **Note:** `NULL` and `NOT` produce the same result for `NIL`, but conceptually they're different: `NULL` asks "is this the empty list?"; `NOT` negates a boolean value. In practice, in MYLISP they're equivalent because the only false value is `NIL`.

5.5.2 ATOM

`ATOM` returns `T` for any atom (number, symbol, string, `NIL`, `T`), and `NIL` for pairs.

```
MYLISP> (ATOM 42)
T
MYLISP> (ATOM 'X)
T
MYLISP> (ATOM NIL)
T
MYLISP> (ATOM '(1 2))
NIL
```

5.5.3 LISTP

`LISTP` returns `T` for lists (including `NIL`) and `NIL` for any non-`NIL` atom.

```
MYLISP> (LISTP '(1 2 3))
T
MYLISP> (LISTP NIL)
T
MYLISP> (LISTP 'A)
NIL
MYLISP> (LISTP 42)
NIL
```

5.6 Recursively Traversing Lists

The single most important pattern in LISP programming is **recursive list traversal**. The structure is almost always the same:

```
(DEFINE MY-FUN
  (LAMBDA (LIST)
    (COND
      ((NULL LIST) ...)           ; base case: empty list
      (T (... (CAR LIST)          ; process the first element
              (MY-FUN (CDR LIST)))))))) ; and recurse on the rest
```

Let's look at a few concrete examples.

5.6.1 Summing the Elements of a List

```
(DEFINE SUM-LIST
  (LAMBDA (L)
    (COND
      ((NULL L) 0)
      (T (+ (CAR L) (SUM-LIST (CDR L)))))))
```

```
MYLISP> (SUM-LIST '(1 2 3 4 5))
15
MYLISP> (SUM-LIST '(1/3 1/3 1/3))
1
MYLISP> (SUM-LIST NIL)
0
```

5.6.2 Searching for an Element in a List

```
(DEFINE MEMBER
  (LAMBDA (ELEM L)
    (COND
      ((NULL L) NIL)
      ((EQUAL (CAR L) ELEM) T)
      (T (MEMBER ELEM (CDR L))))))
```

```
MYLISP> (MEMBER 3 '(1 2 3 4))
T
MYLISP> (MEMBER 9 '(1 2 3 4))
NIL
MYLISP> (MEMBER 'B '(A B C))
T
```

5.6.3 Counting the Elements of a List

Since `LENGTH` isn't implemented, we define it:

```
(DEFINE LENGTH
  (LAMBDA (L)
    (COND
      ((NULL L) 0)
      (T (+ 1 (LENGTH (CDR L)))))))
```

```
MYLISP> (LENGTH '(A B C D E))
5
MYLISP> (LENGTH NIL)
0
```

5.6.4 Reversing a List

```
(DEFINE REVERSE
  (LAMBDA (L)
    (COND
      ((NULL L) NIL)
      (T (APPEND (REVERSE (CDR L))
                 (LIST (CAR L)))))))
```

```
MYLISP> (REVERSE '(1 2 3 4 5))
(5 4 3 2 1)
MYLISP> (REVERSE '(A B C))
(C B A)
```

> **Note:** this implementation of `REVERSE` is simple but O(n²) in time. For an efficient accumulator-based version, see Chapter 9.

5.7 Transforming Lists

5.7.1 Applying a Function to Each Element

```
(DEFINE MY-MAP
  (LAMBDA (F L)
    (COND
      ((NULL L) NIL)
      (T (CONS (F (CAR L))
               (MY-MAP F (CDR L)))))))
```

```
MYLISP> (MY-MAP (LAMBDA (X) (* X X)) '(1 2 3 4 5))
(1 4 9 16 25)
MYLISP> (MY-MAP (LAMBDA (X) (+ X 10)) '(1 2 3))
(11 12 13)
```

5.7.2 Filtering Elements That Meet a Condition

```
(DEFINE FILTER
  (LAMBDA (PRED L)
    (COND
      ((NULL L) NIL)
      ((PRED (CAR L))
       (CONS (CAR L) (FILTER PRED (CDR L))))
      (T (FILTER PRED (CDR L))))))
```

```
MYLISP> (FILTER (LAMBDA (X) (> X 3)) '(1 2 3 4 5 6))
(4 5 6)
MYLISP> (FILTER (LAMBDA (X) (= 0 (MOD X 2)))
              '(1 2 3 4 5 6 7 8))
```

The last example filters out the even numbers using `MOD`, which is a MYLISP primitive.

5.8 Association Lists (A-Lists)

An association list (*a-list*) is a list of `(key . value)` pairs. It's the simplest structure for implementing a table or dictionary.

```
(DEFINE ADDRESS-BOOK
  '((ANN . 555-0100)
    (LOU . 555-0200)
    (EVE . 555-0300)))
```

To look up an element in an a-list, we can define:

```
(DEFINE ASSOC
  (LAMBDA (KEY ALIST)
    (COND
      ((NULL ALIST) NIL)
      ((EQUAL (CAR (CAR ALIST)) KEY)
       (CAR ALIST))
      (T (ASSOC KEY (CDR ALIST))))))
```

```
MYLISP> (ASSOC 'LOU ADDRESS-BOOK)
(LOU . 555-0200)
MYLISP> (ASSOC 'EVE ADDRESS-BOOK)
(EVE . 555-0300)
MYLISP> (ASSOC 'PETE ADDRESS-BOOK)
NIL
```

To get just the value (not the whole pair):

```
MYLISP> (CDR (ASSOC 'ANN ADDRESS-BOOK))
555-0100
```

A-lists are very useful for representing variable environments, algebraic substitutions, or any key-value mapping.

5.8.1 An A-List as an Algebraic Substitution Environment

```
(DEFINE ENVIRONMENT
  '((X . 3) (Y . 5) (Z . 7)))

(DEFINE LOOKUP
  (LAMBDA (VAR ENV)
    (COND
      ((NULL ENV) NIL)
      ((EQUAL (CAR (CAR ENV)) VAR)
       (CDR (CAR ENV)))
      (T (LOOKUP VAR (CDR ENV))))))

MYLISP> (LOOKUP 'X ENVIRONMENT)
3
MYLISP> (LOOKUP 'Y ENVIRONMENT)
5
MYLISP> (LOOKUP 'W ENVIRONMENT)
NIL
```

This pattern is exactly how an evaluator of algebraic expressions substitutes variables with values. It will be the starting point for the CAS in Part II.

5.9 Lists as Trees

Nested lists naturally represent **trees**. A binary tree can be represented as a three-element list `(VALUE LEFT RIGHT)`:

```
(DEFINE TREE
  '(5
    (3 (1 NIL NIL) (4 NIL NIL))
    (8 (7 NIL NIL) (9 NIL NIL))))
```

This represents the tree:

```
        5
       / \
      3   8
     / \ / \
    1  4 7  9
```

Functions for accessing the tree:

```
(DEFINE ROOT  (LAMBDA (A) (CAR A)))
(DEFINE LEFT  (LAMBDA (A) (CADR A)))
(DEFINE RIGHT (LAMBDA (A) (CADDR A)))
```

```
MYLISP> (ROOT TREE)
5
MYLISP> (ROOT (LEFT TREE))
3
MYLISP> (ROOT (RIGHT TREE))
8
```

To traverse the tree in order (left-root-right):

```
(DEFINE INORDER
  (LAMBDA (TREE)
    (COND
      ((NULL TREE) NIL)
      (T (APPEND
           (INORDER (LEFT TREE))
           (LIST (ROOT TREE))
           (INORDER (RIGHT TREE)))))))
```

```
MYLISP> (INORDER TREE)
(1 3 4 5 7 8 9)
```

The in-order traversal of a binary search tree returns the elements in sorted order.

Continues in Chapter 6: Control Flow

Chapter 6 — Control Flow

6.1 Special Control Forms

In MYLISP, control flow isn't handled through statements (as in Pascal or BASIC) but through **special forms**: language constructs that control when and how their subexpressions are evaluated.

Unlike ordinary functions, special forms **do not automatically evaluate all of their arguments**. This is what makes them useful for control flow: if `IF` always evaluated both branches, it would be useless.

MYLISP's control special forms are: `IF`, `COND`, `AND`, `OR`, `NOT`, and `PROGN`.

6.2 IF: The Simplest Branch

`IF` evaluates a condition and runs one of two branches:

```
(IF condition true-expression false-expression)
```

```
MYLISP> (IF T 1 2)
1
MYLISP> (IF NIL 1 2)
2
MYLISP> (IF (> 5 3) "greater" "lesser")
"greater"
```

Only the matching branch is evaluated. The other branch **is never touched**:

```
MYLISP> (IF T 42 (/ 1 0))
42
```

The expression `(/ 1 0)` is never evaluated because the condition is true. If it were evaluated, it would produce a division-by-zero error.

6.2.1 IF with a Single Branch

The "false" branch is optional. If it's omitted and the condition is false, `IF` returns `NIL`:

```
MYLISP> (IF NIL 99)
NIL
MYLISP> (IF T 99)
99
```

6.2.2 Practical Examples

Absolute value (simulated without `ABS`):

```
(DEFINE ABS
  (LAMBDA (X)
    (IF (< X 0) (- X) X)))
```

```
MYLISP> (ABS -5)
5
MYLISP> (ABS 3)
3
MYLISP> (ABS 0)
0
```

Maximum of two numbers (without `MAX`):

```
(DEFINE MAXI
  (LAMBDA (A B)
    (IF (> A B) A B)))
```

```
MYLISP> (MAXI 7 3)
7
MYLISP> (MAXI 3 7)
7
MYLISP> (MAXI 5 5)
5
```

6.3 COND: The Multi-Case Branch

`COND` generalizes `IF` to multiple conditions. Its structure is:

```
(COND
  (condition-1 expression-1)
  (condition-2 expression-2)
  ...
  (T           default-expression))
```

MYLISP evaluates the conditions **from top to bottom** and runs the expression tied to the **first true condition**. The remaining conditions are never evaluated.

```
MYLISP> (COND
          ((= 1 2) "one equals two")
          ((= 1 1) "one equals one")
          (T       "none of the above"))
"one equals one"
```

The final `(T ...)` clause acts as an "otherwise": since `T` is always true, it runs if none of the earlier conditions did.

6.3.1 Classifying a Number

```
(DEFINE SIGN
  (LAMBDA (N)
    (COND
      ((> N 0) "positive")
      ((< N 0) "negative")
      (T       "zero"))))
```

```
MYLISP> (SIGN 5)
"positive"
MYLISP> (SIGN -3)
"negative"
MYLISP> (SIGN 0)
"zero"
```

6.3.2 COND with No Default Clause

If no condition is true and there's no `T` clause, `COND` returns `NIL`:

```
MYLISP> (COND
          ((= 1 2) "never"))
NIL
```

6.3.3 COND in the Symbolic Differentiator

`COND` is ideal for implementing functions that behave differently depending on the kind of expression — exactly what a symbolic differentiator needs:

```
(DEFINE DERIV
  (LAMBDA (EXP VAR)
    (COND
      ((EQUAL EXP VAR)          1)
      ((ATOM EXP)               0)
      ((EQUAL (CAR EXP) '+)
       (LIST '+ (DERIV (CADR EXP) VAR)
                (DERIV (CADDR EXP) VAR)))
      ((EQUAL (CAR EXP) '*)
       (LIST '+
         (LIST '* (CADR EXP)
                  (DERIV (CADDR EXP) VAR))
         (LIST '* (DERIV (CADR EXP) VAR)
                  (CADDR EXP))))
      (T (LIST 'D EXP VAR)))))  ; unknown derivative
```

Chapter 10 develops this differentiator in depth.

6.4 AND: Short-Circuit Conjunction

`AND` evaluates its arguments **from left to right** and stops as soon as one is `NIL` (false). It returns the first false value found, or the last argument if all are true.

```
MYLISP> (AND T T T)
T
MYLISP> (AND T NIL T)
NIL
MYLISP> (AND 1 2 3)
3
MYLISP> (AND 1 NIL 3)
NIL
```

Short-circuiting means arguments to the right of a `NIL` are never evaluated:

```
MYLISP> (AND NIL (/ 1 0))
NIL
```

The division by zero never runs because `AND` sees `NIL` and stops.

6.4.1 Typical Uses of AND

Checking several conditions at once:

```
(DEFINE IN-RANGE?
  (LAMBDA (X MIN MAX)
    (AND (>= X MIN) (<= X MAX))))
```

```
MYLISP> (IN-RANGE? 5 1 10)
T
MYLISP> (IN-RANGE? 15 1 10)
NIL
```

Validating before computing:

```
(DEFINE SAFE-DIV
  (LAMBDA (A B)
    (AND (NOT (= B 0)) (/ A B))))
```

```
MYLISP> (SAFE-DIV 10 2)
5
MYLISP> (SAFE-DIV 10 0)
NIL
```

6.5 OR: Short-Circuit Disjunction

`OR` evaluates its arguments from left to right and returns the **first true value** found, or `NIL` if all are false.

```
MYLISP> (OR NIL NIL T)
T
MYLISP> (OR NIL NIL NIL)
NIL
MYLISP> (OR NIL 42 T)
42
```

Like `AND`, `OR` short-circuits:

```
MYLISP> (OR T (/ 1 0))
T
```

The division by zero never runs because the first argument is already true.

6.5.1 Typical Uses of OR

Providing a default value:

```
(DEFINE DEFAULT
  (LAMBDA (VALUE FALLBACK)
    (OR VALUE FALLBACK)))
```

```
MYLISP> (DEFAULT NIL 99)
99
MYLISP> (DEFAULT 42 99)
42
```

Searching across several sources:

```
(DEFINE GET
  (LAMBDA (KEY ENV1 ENV2)
    (OR (LOOKUP KEY ENV1)
        (LOOKUP KEY ENV2))))
```

6.6 NOT: Logical Negation

`NOT` returns `T` if its argument is `NIL`, and `NIL` otherwise.

```
MYLISP> (NOT NIL)
T
MYLISP> (NOT T)
NIL
MYLISP> (NOT 42)
NIL
MYLISP> (NOT '(A B))
NIL
```

Any non-`NIL` value is treated as true: `NOT` returns `NIL` for all of them.

6.6.1 Combining NOT with Predicates

```
(DEFINE NOT-NULL?
  (LAMBDA (X)
    (NOT (NULL X))))  ; true if x is NOT empty
```

```
MYLISP> (NOT-NULL? '(1 2))
T
MYLISP> (NOT-NULL? NIL)
NIL
```

```
(DEFINE DIFFERENT?
  (LAMBDA (A B)
    (NOT (EQUAL A B))))
```

```
MYLISP> (DIFFERENT? 'A 'B)
T
MYLISP> (DIFFERENT? 'A 'A)
NIL
```

6.7 PROGN: A Sequence of Expressions

`PROGN` evaluates a sequence of expressions from left to right and returns the value of the **last** one. It's used when you need to run several expressions where normally only one would fit.

```
MYLISP> (PROGN
          (DEFINE X 10)
          (DEFINE Y 20)
          (+ X Y))
30
```

`PROGN` is especially useful in the "true" or "false" branch of an `IF` when you want to do more than one thing:

```
(IF (> X 0)
  (PROGN
    (PRINT "positive")
    (* X 2))
  (PROGN
    (PRINT "negative or zero")
    (- X)))
```

Without `PROGN`, `IF` could only run one expression per branch.

6.8 Combining the Control Forms

The control forms combine naturally to build complex logic.

6.8.1 Triangle Classifier

```
(DEFINE TRIANGLE
  (LAMBDA (A B C)
    (COND
      ((NOT (AND (> A 0) (> B 0) (> C 0)))
       "invalid sides")
      ((AND (= A B) (= B C))
       "equilateral")
      ((OR (= A B) (= B C) (= A C))
       "isosceles")
      (T
       "scalene"))))
```

```
MYLISP> (TRIANGLE 3 3 3)
"equilateral"
MYLISP> (TRIANGLE 5 5 3)
"isosceles"
MYLISP> (TRIANGLE 3 4 5)
"scalene"
MYLISP> (TRIANGLE -1 2 3)
"invalid sides"
```

6.8.2 Finding the Maximum in a List

```
(DEFINE MAX-LIST
  (LAMBDA (L)
    (COND
      ((NULL L) NIL)
      ((NULL (CDR L)) (CAR L))
      (T (IF (> (CAR L) (MAX-LIST (CDR L)))
             (CAR L)
             (MAX-LIST (CDR L)))))))
```

```
MYLISP> (MAX-LIST '(3 1 4 1 5 9 2 6))
9
MYLISP> (MAX-LIST '(7))
7
```

6.8.3 Checking Whether a List Is Sorted

```
(DEFINE SORTED?
  (LAMBDA (L)
    (OR (NULL L)
        (NULL (CDR L))
        (AND (<= (CAR L) (CADR L))
             (SORTED? (CDR L))))))
```

```
MYLISP> (SORTED? '(1 2 3 4 5))
T
MYLISP> (SORTED? '(1 3 2 4))
NIL
MYLISP> (SORTED? NIL)
T
```

6.9 Truth Tables for AND, OR, and NOT

For quick reference:

A	B	(AND A B)	(OR A B)
T	T	T	T
T	NIL	NIL	T
NIL	T	NIL	T
NIL	NIL	NIL	NIL

A	(NOT A)
T	NIL
NIL	T
42	NIL
"hello"	NIL

Remember: in MYLISP, any value other than `NIL` is true.

Continues in Chapter 7: Defining Functions and Lexical Closures

---

Chapter 7 — Defining Functions and Lexical Closures

7.1 Functions in LISP: Everything Is a Value

In MYLISP, functions are **first-class values**: they can be created, stored in variables, passed as arguments to other functions, and returned as the result of an evaluation. This property, shared with Scheme and modern functional languages, is what makes LISP such a powerful tool.

This chapter covers the four ways of defining functions in MYLISP — `LAMBDA`, `DEFINE`, `DEFUN`, and `LET` — as well as the concepts of recursion and lexical closures.

7.2 LAMBDA: The Anonymous Function

`LAMBDA` is the only form MYLISP has for *creating* a function. `DEFINE` and `DEFUN`, which we'll see next, don't create anything new: they simply put a label on a function that, underneath, was always built with `LAMBDA`. Understanding `LAMBDA` well means understanding where LISP's power to treat code as data really comes from.

7.2.1 Where the Name Comes From, and Why It Matters

The name is not a typographical whim. It comes directly from Alonzo Church's **lambda calculus** (1930s), the mathematical notation McCarthy used as his starting point for designing LISP in 1958 (§1.1.1). Church needed a way to write "the function that does such-and-such" without having to christen it first — just as in mathematics we write `x ↦ x²` without needing to call it `f`. `(LAMBDA (X) (* X X))` is exactly that idea carried over into S-expressions: "the function that, given `X`, returns `X * X`," with no name at all.

This is what distinguishes `LAMBDA` from `DEFUN`: `DEFUN` answers the question *"what is this function called?"*; `LAMBDA` doesn't even admit the question — the function exists and is usable without anyone ever having named it. This ability to create "disposable" functions is the foundation of the higher-order functional programming exploited in full in Chapter 9 and in the CAS itself (from Chapter 10 onward).

7.2.2 Anatomy of the Expression

Its general structure is:

```
(LAMBDA (parameter-1 parameter-2 ...) body)
```

Two parts, both mandatory:

- **The parameter list**, in parentheses. It can have zero parameters (`(LAMBDA () ...)`, a function with no arguments), one, or several — MYLISP imposes no practical limit beyond whatever the working environment itself imposes.
- **The body**: a single expression that's evaluated when the function is called, with the parameters already substituted by the arguments received. If at some point you need to run several steps, the solution is to wrap them with `PROGN` (§7.5) — but the rule for `LAMBDA` itself is always "one function, one expression."

```
MYLISP> ((LAMBDA (X) (* X X)) 5)
25
```

Here `(LAMBDA (X) (* X X))` is the "square it" function. By writing it as the first element of a list that gets evaluated, it is **applied immediately** to the argument `5`: evaluation rule 3 itself (§3.4) makes no distinction between `(SQUARE 5)` and `((LAMBDA (X) (* X X)) 5)` — in both cases, the first element is evaluated until a function is obtained, and that function is applied to the rest.

```
MYLISP> ((LAMBDA (A B) (+ (* A A) (* B B))) 3 4)
25
```

This two-parameter anonymous function computes `a² + b²`. The number of arguments in the call must match exactly the number of declared parameters: `LAMBDA` does not support optional parameters or "rest arguments" (there's no equivalent to Common LISP's `&rest`, or to the variadic arguments of Chapter 4's arithmetic primitives — those are variadic because they're interpreter primitives, not user-defined functions).

7.2.3 LAMBDA Is a Value, Not an Action

This is the point that's hardest to internalize coming from imperative languages: **evaluating a `LAMBDA` expression does not run the body**. It only *packages* it. The body `(* X X)` isn't touched at all until the resulting function is applied to a concrete argument.

Put another way: `LAMBDA` is a special form (§3.4) that, when evaluated, produces a **function value** — internally, MYLISP represents it as a closure of type `TCLOSURE` (Appendix B.6): a pair made up of the `LAMBDA` expression itself and a reference to the environment in which it appeared. That value can be treated exactly like a number or a list: stored in a variable, passed as an argument, returned as a result. This property — **functions are first-class values** — is what was announced in §7.1, and `LAMBDA` is the concrete mechanism that makes it possible.

A `LAMBDA` function can be stored in a variable with `DEFINE`:

```
MYLISP> (DEFINE SQUARE (LAMBDA (X) (* X X)))
SQUARE
MYLISP> (SQUARE 7)
49
```

Now `SQUARE` is a symbol whose value is the function. Calling `(SQUARE 7)` evaluates `SQUARE` (obtaining the function), and then applies the function to the argument `7`. Note that `DEFINE` doesn't add any power to the function: `SQUARE` and `(LAMBDA (X) (* X X))` are, at every moment, exactly the same value — one simply has a name in the global environment and the other doesn't.

7.2.4 Its Full Power: Passing and Returning Functions

The fact that `LAMBDA` produces an ordinary value, not an action, is what enables two uses that are the heart of functional programming in LISP:

Passing an anonymous function as an argument. There's no need to name a function that will only be used once. If, in Chapter 5, we have a function `MY-MAP` that applies a function to every element of a list, we don't need to define `SQUARE` beforehand if we're only going to use it there:

```
MYLISP> (MY-MAP (LAMBDA (X) (* X X)) '(1 2 3 4 5))
(1 4 9 16 25)
```

The squaring function is created, used once, and discarded — without cluttering the global environment with a name (`SQUARE`) that was only needed once. This pattern — an anonymous `LAMBDA` as the argument of a higher-order function — is the normal way of working with `MY-MAP`, `FILTER`, and `MY-REDUCE` in Chapter 5, and with their final versions in Chapter 9.

Returning a function from another function. Since `LAMBDA` is a value, a function can build and return another function at run time:

```
(DEFINE MAKE-ADDER
  (LAMBDA (N)
    (LAMBDA (X) (+ X N))))
```

`(MAKE-ADDER 5)` doesn't compute a number: it evaluates `MAKE-ADDER`'s body, which is itself a `LAMBDA` expression, and that expression is evaluated until *another* function is obtained — one that already has the value `N = 5` "built in." The result isn't just any function: it's a **closure**, because it remembers the environment in which it was created even after `MAKE-ADDER` has finished running. That mechanism — exactly what gets captured, and what you can and can't do with it in MYLISP — is the whole subject of §7.9; for now it's enough to see that it's **the very same `LAMBDA`**, with no extra syntax, that makes it possible.

These two capabilities — function as argument, function as result — are exactly the working definition of "higher-order function." They are not an extra MYLISP feature: they are a direct, free consequence of `LAMBDA` evaluating to a first-class value, as explained in §7.2.3.

7.3 DEFINE: Naming Values and Functions

`DEFINE` associates a symbol with a value in the **global environment**:

```
(DEFINE name value)
```

For functions:

```
(DEFINE SQUARE (LAMBDA (X) (* X X)))
(DEFINE CUBE   (LAMBDA (X) (* X X X)))
(DEFINE PI     3.14159)
```

```
MYLISP> (SQUARE 4)
16
MYLISP> (CUBE 3)
27
MYLISP> (* 2 PI 5)
31.4159
```

`DEFINE` can redefine an existing symbol. The new definition replaces the old one:

```
MYLISP> (DEFINE X 10)
X
MYLISP> X
10
MYLISP> (DEFINE X 99)
X
MYLISP> X
99
```

7.4 DEFUN: Shorthand Syntax for Functions

`DEFUN` is a special form that abbreviates the pattern `(DEFINE name (LAMBDA ...))`:

```
(DEFUN name (parameters...) body)
```

The following two are equivalent:

```
; With DEFINE and LAMBDA:
(DEFINE SQUARE (LAMBDA (X) (* X X)))

; With DEFUN:
(DEFUN SQUARE (X) (* X X))
```

`DEFUN` is more concise and resembles Common LISP syntax. In code files it's the preferred form because it takes up less of the REPL's or a file's 80-character limit.

```
(DEFUN ADD (A B) (+ A B))
(DEFUN GREATER (A B) (IF (> A B) A B))
(DEFUN EVENP (N) (= 0 (- N (* (/ N 2) 2))))
```

> **Note:** internally, `DEFUN` simply expands to `DEFINE` with a `LAMBDA`. There is no functional difference between the two forms.

7.5 Functions with Multiple Expressions in the Body

The body of a `LAMBDA` or `DEFUN` is a single expression. If you need to run several things, use `PROGN`:

```
(DEFUN DEBUG (X)
  (PROGN
    (PRINT X)
    (* X X)))
```

```
MYLISP> (DEBUG 5)
5
25
```

`PRINT` shows the intermediate value; `(* X X)` is the value returned.

7.6 Recursion

7.6.1 What Is Recursion?

In most imperative languages — BASIC, Pascal, C — repetition is expressed with loops: `FOR`, `WHILE`, `REPEAT`. LISP has no loops. Repetition is expressed through **recursion**: a function that calls itself.

The idea seems circular at first glance. How can a function be defined in terms of itself without falling into an infinite loop? The answer lies in the structure of the problem: every recursive problem has at least two cases.

The **base case** is the simplest possible situation, one that has an immediate, direct answer with no need to recurse. The **recursive case** breaks the problem down into a smaller one of the same kind, and calls the function itself on that subproblem. The key is that every recursive call moves closer to the base case. At some point it's reached, the recursion stops, and the partial results are combined back up to the original call.

To compute the factorial of N, we can reason like this: if N is 0, the result is 1 (base case, by definition). If N is greater than 0, the factorial of N is N times the factorial of N−1 (recursive case, which moves closer to 0 at each step). In mathematical notation: `N! = N × (N−1)!`. In MYLISP, this definition is written almost literally:

```
FACT(5) = 5 × FACT(4)
            = 5 × 4 × FACT(3)
                      = 5 × 4 × 3 × FACT(2)
                                  = 5 × 4 × 3 × 2 × FACT(1)
                                              = 5 × 4 × 3 × 2 × 1 × FACT(0)
                                                                  = 1
```

The recursion "descends" down to `FACT(0)` and then "ascends," combining the results: `1`, `1×1=1`, `2×1=2`, `3×2=6`, `4×6=24`, `5×24=120`.

7.6.2 The General Pattern

Every recursive function in MYLISP follows this skeleton:

```
(DEFUN MY-FUN (ARGUMENT)
  (COND
    (base-case        direct-result)
    (T                (combine (MY-FUN smaller-argument)
                               other-data))))
```

"Smaller argument" may mean N−1 for an integer, or `(CDR list)` for a list (which has one fewer element). "Combine" may be a sum, a multiplication, a `CONS`, an `APPEND`... whatever the problem requires.

7.6.3 Factorial

Without recursion, the only way to compute `5!` in MYLISP would be to write out every operation by hand:

```
; Without recursion: you'd have to expand manually for each N
MYLISP> (* 5 (* 4 (* 3 (* 2 (* 1 1)))))
120
```

This works for `5!`, but it's useless as a general function: we'd need a different expression for every value of N. There's no `FOR` or `WHILE` to iterate with. Recursion solves exactly this problem: it lets you write the general rule once and have the interpreter apply it as many times as needed.

```
; With recursion: a single definition valid for any N
(DEFUN FACT (N)
  (IF (= N 0)
    1
    (* N (FACT (- N 1)))))
```

```
MYLISP> (FACT 5)
120
MYLISP> (FACT 0)
1
MYLISP> (FACT 10)
3628800
```

> **Warning!** `FACT` has 4 characters, well within the limit. If you had written `FACTORIAL` (9 characters), it would be truncated to `FACTORIA`, and a call to `FACTORIAL` would give "undefined symbol."

7.6.4 Fibonacci

The Fibonacci sequence is a series of numbers in which each term is the sum of the two before it: `0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...`. The first two terms are 0 and 1 by definition (the two base cases), and from there `FIB(N) = FIB(N-1) + FIB(N-2)`. It's the canonical example of recursion with two base cases and two recursive calls in the general case.

```
(DEFUN FIB (N)
  (COND
    ((= N 0) 0)
    ((= N 1) 1)
    (T (+ (FIB (- N 1))
          (FIB (- N 2))))))
```

```
MYLISP> (FIB 0)
0
MYLISP> (FIB 7)
13
MYLISP> (FIB 10)
55
```

7.6.5 Integer Power

Without `EXPT`, we define it:

```
(DEFUN POWER (BASE EXP)
  (COND
    ((= EXP 0) 1)
    ((> EXP 0) (* BASE (POWER BASE (- EXP 1))))
    (T (/ 1 (POWER BASE (- EXP))))))
```

```
MYLISP> (POWER 2 10)
1024
MYLISP> (POWER 3 4)
81
MYLISP> (POWER 2 -3)
1/8
```

Power with a negative exponent returns an exact rational.

7.6.6 Tail Recursion and the Stack Problem

Behavior under deep recursion is the most noticeable difference between the two platforms:

> **QL:** every recursive call adds a frame to Pascal's call stack. If the recursion is deep enough, the stack runs out and the interpreter destroys all user definitions to recover. In practice, the 24,000-cell heap tends to run out before the stack for most programs, but the threat is real for very long lists or recursions with no base case. For computations with many levels, the accumulator-based version is the recommended solution. **Next:** the evaluator is iterative — user recursion consumes no CPU stack at all. You can write functions with hundreds of levels of recursion with no problem. The only real limit is the 32,000-cell heap: every active call consumes some cells for the environment and the arguments, but this limit is far more generous than the QL's stack.

7.6.7 Recursion with an Accumulator (Tail Position)

The accumulator-based version of factorial:

```
(DEFUN FACT-ACC (N ACC)
  (IF (= N 0)
    ACC
    (FACT-ACC (- N 1) (* N ACC))))

(DEFUN FACT (N)
  (FACT-ACC N 1))
```

```
MYLISP> (FACT 5)
120
```

Tail recursion is not automatically optimized by MYLISP (it doesn't implement TCO — *tail call optimization*), but the accumulator-based structure can be clearer for the programmer.

7.7 How to Debug Recursive Functions

Recursion is powerful but can be baffling when it doesn't work as expected. The most effective debugging technique in MYLISP is **manual tracing with PRINT**: inserting calls to `PRINT` at the start of the function body to see how the arguments change at each level of recursion.

7.7.1 Basic Tracing

```
(DEFUN FACT (N)
  (PROGN
    (PRINT N)               ; shows N on every call
    (IF (= N 0)
      1
      (* N (FACT (- N 1))))))
```

```
MYLISP> (FACT 4)
4
3
2
1
0
24
```

The trace shows the "descent" down to the base case (`N = 0`) and then the final result (`24`). If the base case never shows up, the function is stuck in infinite recursion.

7.7.2 Detecting Infinite Recursion

The most common symptom of a recursion bug is that the interpreter stops responding. This happens because the function keeps calling itself without ever getting closer to the base case, consuming resources until the interpreter can't continue. When that happens, MYLISP **destroys all user definitions** and shows the prompt again. You will have lost the session's work if you hadn't saved it to a file.

> **QL:** what runs out is Pascal's call stack. The recursion depth before failure is relatively low. **Next:** what runs out is the heap (32,000 cells). The depth can be hundreds of levels before failure, but the end result is the same.

The two most common mistakes are: forgetting the base case, and not moving closer to it in the recursive case.

```
; BUG: the base case is never reached for negative N
(DEFUN FACT (N)
  (IF (= N 0)
    1
    (* N (FACT (- N 1)))))

MYLISP> (FACT -1)   ; will call FACT(-1), FACT(-2), FACT(-3)... forever
```

The fix: add input validation.

```
(DEFUN FACT (N)
  (COND
    ((< N 0) (PRINT "error: negative"))
    ((= N 0) 1)
    (T (* N (FACT (- N 1))))))
```

7.7.3 Tracing Two Arguments

For functions with an accumulator, print both:

```
(DEFUN FACT-ACC (N ACC)
  (PROGN
    (DISPLAY "N=") (PRINT N)
    (DISPLAY "ACC=") (PRINT ACC)
    (IF (= N 0)
      ACC
      (FACT-ACC (- N 1) (* N ACC)))))
```

```
MYLISP> (FACT-ACC 3 1)
N=3
ACC=1
N=2
ACC=3
N=1
ACC=6
N=0
ACC=6
6
```

You can see how `ACC` accumulates the result as `N` descends. Once the function works correctly, remove the `PRINT` calls from the final code.

7.8 LET: Local Variables

`LET` introduces **local variables** that only exist inside its body. Its structure is:

```
(LET ((var-1 val-1)
      (var-2 val-2)
      ...)
  single-expression)
```

> **The body of LET accepts a single expression.** Unlike Scheme and Common LISP, where the body of `LET` has an implicit `PROGN`, in MYLISP the body is **a single expression**. If you need to run several things inside a `LET`, wrap them in an explicit `PROGN`:
>
> ```
> (LET ((X 5))
> (PRINT X)        ; ERROR: this is a second argument, not a body
> (* X X))
>
> (LET ((X 5))
> (PROGN           ; CORRECT: PROGN groups the two expressions into one
>  (PRINT X)
>  (* X X)))
> ```

```
MYLISP> (LET ((X 5)
              (Y 3))
          (+ X Y))
8
```

`X` and `Y` only exist inside the `LET`. Outside it, they aren't accessible (unless they already existed in the global environment).

7.8.1 LET Has Parallel Scope

An important property: in `LET`, **all the values on the right are evaluated in the current environment, before any of the new variables become visible**. This is called parallel binding.

```
(DEFINE X 10)
(LET ((X 1)
      (Y X))     ; this X is the global one, not the new one
  (LIST X Y))
```

```
MYLISP> (LET ((X 1) (Y X))
          (LIST X Y))
(1 10)
```

`Y` takes the value of `X` **before** the `LET` creates the new `X`. That's why `Y = 10` (the global `X`), not `1`.

If you need `Y` to use the new `X`, you must nest the `LET`s:

```
MYLISP> (LET ((X 1))
            (LET ((Y X))
              (LIST X Y)))
(1 1)
```

7.8.2 Using LET to Avoid Repeated Computations

```
(DEFUN HYPOT-SQ (A B)
  (LET ((A2 (* A A))
        (B2 (* B B)))
    (+ A2 B2)))     ; returns the square of the hypotenuse
```

```
MYLISP> (HYPOT-SQ 3 4)
25
```

`A2` and `B2` are computed only once even though they're used multiple times in the body.

7.8.3 LET with Local Helper Functions

```
(DEFUN NORMALIZE (L)
  (LET ((TOTAL (SUM-LIST L)))
    (MY-MAP (LAMBDA (X) (/ X TOTAL)) L)))
```

Here `TOTAL` is the sum of the list, computed once. Then each element is divided by that total. The result is the normalized list (all elements sum to 1):

```
MYLISP> (NORMALIZE '(1 2 3 4))
(1/10 1/5 3/10 2/5)
```

7.9 Lexical Closures

A **closure** is a function that "remembers" the environment in which it was created, even after that environment is no longer active. It's one of LISP's most powerful and elegant mechanisms.

When MYLISP evaluates a `LAMBDA` expression, it doesn't just create the function — it creates a closure: the function together with a reference to the lexical environment in which it was defined.

7.9.1 A Simple Example

```
(DEFINE MAKE-ADDER
  (LAMBDA (N)
    (LAMBDA (X) (+ X N))))
```

`MAKE-ADDER` returns a function. Calling it with `5` returns a function that adds 5 to any number:

```
MYLISP> (DEFINE ADD5 (MAKE-ADDER 5))
ADD5
MYLISP> (ADD5 10)
15
MYLISP> (ADD5 3)
8
MYLISP> (ADD5 100)
105
```

What happens here? When we evaluate `(MAKE-ADDER 5)`:

1. An environment is created where `N = 5`.
2. The body `(LAMBDA (X) (+ X N))` is evaluated.
3. A closure is created that captures that environment: the function remembers that `N = 5`.
4. That closure is returned.

When we later call `(ADD5 10)`, the function looks up `N` in the captured environment and finds `5`. It computes `10 + 5 = 15`.

7.9.2 Creating Multipliers

```
(DEFINE MAKE-MULTIPLIER
  (LAMBDA (FACTOR)
    (LAMBDA (X) (* X FACTOR))))

(DEFINE DOUBLE (MAKE-MULTIPLIER 2))
(DEFINE TRIPLE (MAKE-MULTIPLIER 3))
```

```
MYLISP> (DOUBLE 7)
14
MYLISP> (TRIPLE 4)
12
MYLISP> (DOUBLE (TRIPLE 5))
30
```

7.9.3 Closures as "Objects": What MYLISP Can and Can't Do

In languages with lexical mutation (such as Scheme with `set!`), closures can encapsulate mutable state and act like simple objects. In MYLISP this **is not possible**: `DEFINE` always writes to the global environment, never to the environment captured by the closure. A "counter" built out of closures would silently break: two different counters would share and clobber the same global variable.

What does work elegantly in MYLISP are **behavior-parameterizing closures** — closures that capture constant values, not mutable state:

7.9.4 Closures for Parameterizing Behavior

A very practical use case: passing functions as arguments.

```
(DEFUN APPLY-TO (F LIST)
  (MY-MAP F LIST))
```

```
MYLISP> (APPLY-TO (LAMBDA (X) (* X X)) '(1 2 3 4 5))
(1 4 9 16 25)
MYLISP> (APPLY-TO (LAMBDA (X) (+ X 100)) '(1 2 3))
(101 102 103)
```

Here the function `F` is an anonymous closure created at the moment of the call.

7.10 The Single Namespace (Lisp-1)

MYLISP is a **Lisp-1**: functions and variables share the same namespace. This contrasts with Common LISP (Lisp-2), which has separate namespaces.

The practical consequence is that you can use variables whose name matches that of a function:

```
(DEFINE SUM (LAMBDA (A B) (+ A B)))
(DEFINE SUM 99)   ; now SUM is a number, not a function
```

```
MYLISP> (SUM 3 4)
ERROR: ... ; SUM is no longer a function
```

This is rarely a problem in practice, but you should be aware that redefining a symbol affects both its use as a variable and its use as a function.

7.10.1 Colliding with Primitives: A Real Danger

The most dangerous case is accidentally redefining an interpreter primitive. MYLISP has no protection against this:

```
MYLISP> (DEFINE CAR 5)
5
MYLISP> (CAR '(A B C))
ERROR: ...   ; CAR is no longer a function, it's the number 5
```

The `CAR` primitive has been destroyed for this session. **There is no way to restore it without restarting the interpreter.** The only way out is:

1. Type `BYE` to exit MYLISP.
2. Relaunch the interpreter from the QL.
3. Reload all your working files with `LOAD`.

To avoid this problem, choose function and variable names that don't collide with the primitives. A useful convention is to use your own prefixes (`MY-CAR`, `MY-`, etc.) for all user code. The complete list of primitives is in Appendix A.

7.10.2 The Advantage of Lisp-1

The advantage of Lisp-1 is that higher-order functions are simpler to write: there's no need for `FUNCALL` or `#'`:

```
; In Common LISP (Lisp-2):
(mapcar #'(lambda (x) (* x x)) '(1 2 3))

; In MYLISP (Lisp-1):
(MY-MAP (LAMBDA (X) (* X X)) '(1 2 3))
```

7.11 Mutually Recursive Functions

Two functions that call each other. In MYLISP, since `DEFINE` is global, we can define them in any order as long as both are defined before either is called:

```
(DEFUN EVEN? (N)
  (IF (= N 0)
    T
    (ODD? (- N 1))))

(DEFUN ODD? (N)
  (IF (= N 0)
    NIL
    (EVEN? (- N 1))))
```

```
MYLISP> (EVEN? 4)
T
MYLISP> (EVEN? 7)
NIL
MYLISP> (ODD? 5)
T
```

Since both are in the global environment by the time they're called, there's no problem with forward references.

Continues in Chapter 8: Strings, PRINT, and LOAD

Chapter 8 — Strings, PRINT, and LOAD

8.1 Text Strings in MYLISP

MYLISP supports text strings as a native data type. A string is a sequence of characters delimited by double quotes:

```
"hello world"
"result: "
"MYLISP 1.0"
```

Strings are **literals**: they evaluate to themselves.

```
MYLISP> "hello"
"hello"
MYLISP> (DEFINE TITLE "MYLISP for QL")
TITLE
MYLISP> TITLE
"MYLISP for QL"
```

8.1.1 String Limitations

MYLISP has strict limits on strings:

- **Maximum length:** 36 characters per string.
- **Maximum count:** 50 distinct strings for the whole session (including those defined in files loaded with `LOAD`).

```
MYLISP> "This string is exactly 36 chars lon"
"This string is exactly 36 chars lon"
```

If you try to create a string longer than 36 characters, it will be truncated or produce an error depending on the implementation.

> **Warning!** The string table has room for 50 entries. If your program creates many distinct strings (for example, strings generated dynamically in a loop), you can exhaust it. Reuse strings whenever possible.

8.1.2 Using Strings as Messages

Strings are useful for adding descriptive text to results:

```
(DEFUN SHOW-INFO (LABEL VALUE)
  (PROGN
    (PRINT LABEL)
    (PRINT VALUE)))
```

```
MYLISP> (SHOW-INFO "Result:" (* 6 7))
"Result:"
42
42
```

Notice that `PRINT` prints the value and returns the printed value. The last `42` is the value returned by the `SHOW-INFO` function (which returns whatever `PRINT VALUE` returns).

8.2 Output Functions: PRINT, DISPLAY, NEWLINE, and SYMNAME

MYLISP has four primitives for writing to the screen. Each behaves differently, and it's worth knowing them well in order to build readable output.

8.2.1 PRINT

`PRINT` prints its argument to the screen **using reader format** (strings appear in quotes, symbols in uppercase) followed by a newline, and returns that same argument.

```
MYLISP> (PRINT 42)
42
42
MYLISP> (PRINT "hello")
"hello"
"hello"
MYLISP> (PRINT '(1 2 3))
(1 2 3)
(1 2 3)
```

The first line of each block is what `PRINT` writes; the second is the return value shown by the REPL (which is the same value). `PRINT` is the fundamental debugging tool: since it returns its argument, it can be inserted at any point in an expression without changing the result.

```
(DEFUN FACT (N)
  (PROGN
    (PRINT N)
    (IF (= N 0)
      1
      (* N (FACT (- N 1))))))
```

```
MYLISP> (FACT 4)
4
3
2
1
0
24
```

It shows the trace of recursive calls before returning the final result.

8.2.2 DISPLAY

`DISPLAY` prints its argument **without reader format**: strings are shown without quotes and escape characters are interpreted. It adds no trailing newline. It returns the printed value.

```
MYLISP> (DISPLAY "hello")
hello
MYLISP> (DISPLAY 42)
42
MYLISP> (DISPLAY '(1 2 3))
(1 2 3)
```

The difference from `PRINT` shows up especially with strings: `(PRINT "hello")` writes `"hello"` with quotes and the REPL shows the returned value; `(DISPLAY "hello")` writes `hello` with no quotes and **shows nothing else** (it returns VOID, a special value the REPL doesn't print). `DISPLAY` is the right primitive for building text output meant for the end user, not the debugger.

To separate several `DISPLAY` calls onto different lines, use `NEWLINE`.

8.2.3 NEWLINE

`NEWLINE` emits a line break to the output and returns `NIL`. It takes no arguments.

```
MYLISP> (NEWLINE)

NIL
```

Combined with `DISPLAY`, it lets you build precisely formatted output:

```
(DEFUN REPORT (LABEL VAL)
  (PROGN
    (DISPLAY LABEL)
    (DISPLAY " ")
    (DISPLAY VAL)
    (NEWLINE)))
```

```
MYLISP> (REPORT "Result:" 42)
Result: 42
NIL
```

Compare this with `PRINT`: it would have shown `"Result:"`, `" "`, and `42` each on its own line and in quotes if they were strings. `DISPLAY` + `NEWLINE` gives you total control over formatting.

8.2.4 SYMNAME

`SYMNAME` returns a symbol's name as a **text string**. It's the gateway between the world of symbols and the world of strings.

```
MYLISP> (SYMNAME 'HELLO)
"HELLO"
MYLISP> (SYMNAME 'X)
"X"
MYLISP> (SYMNAME 'FACT)
"FACT"
```

`SYMNAME` is especially useful in the CAS when we want to print algebraic expressions in a readable way. Without `SYMNAME`, a symbol like `X` prints with `PRINT` as `X` (fine for the REPL), but if you want to build a string that contains that name — for a descriptive error message, or to generate text — you need to convert it to a string first.

```
(DEFUN UNDEF-VAR-ERR (VAR)
  (PROGN
    (DISPLAY "Undefined variable: ")
    (DISPLAY (SYMNAME VAR))
    (NEWLINE)))
```

```
MYLISP> (UNDEF-VAR-ERR 'ALPHA)
Undefined variable: ALPHA
NIL
```

> **Note:** `SYMNAME` only accepts symbols. If you pass it a number, a list, or another type, the behavior is undefined. Check with `SYMBOLP` before calling it if the argument's type isn't guaranteed.

8.2.5 STR<: Alphabetical Comparison

`STR<` compares two values lexicographically and returns `T` if the first comes before the second alphabetically. It accepts strings (`TSTRING`) or symbols (`TSYM`) interchangeably.

```
MYLISP> (STR< "ALFA" "BETA")
T
MYLISP> (STR< "ZETA" "ALFA")
NIL
MYLISP> (STR< 'ANN 'LOU)
T
MYLISP> (STR< 'ZETA 'ALFA)
NIL
```

When passed symbols directly, `STR<` compares them without needing to convert them to a string with `SYMNAME` first, which is more efficient. This lets you define a symbol comparator in a single line:

```
(DEFUN SYM< (A B) (STR< A B))
```

And use it to sort lists of symbols:

```
MYLISP> (QSORT '(LOU ANN EVE MARIO) SYM<)
```

> **Note:** `STR<` compares according to the QL's character codes. Since MYLISP converts every symbol to uppercase, the comparison is always between uppercase letters and produces a natural alphabetical order.

8.2.6 STRCAT: String Concatenation

`STRCAT` concatenates two values by converting them to strings and returns the resulting string. It accepts strings (`TSTRING`), symbols (`TSYM`), integers (`TINT`), and rationals (`TRAT`). It does not accept reals (`TFLOAT`). The result is truncated to 36 characters (the `STRLEN` limit).

```
MYLISP> (STRCAT "hello" " world")
"hello world"
MYLISP> (STRCAT "N=" 42)
"N=42"
MYLISP> (STRCAT "frac=" 1/3)
"frac=1/3"
MYLISP> (STRCAT "var=" 'X)
"var=X"
```

`STRCAT` is especially useful for building descriptive error messages or dynamic filenames:

```
(DEFUN ERR-MSG (KIND DETAIL)
  (PROGN
    (DISPLAY (STRCAT "Error in " KIND))
    (DISPLAY ": ")
    (DISPLAY DETAIL)
    (NEWLINE)))
```

```
MYLISP> (ERR-MSG "DERIV" "unknown variable")
Error in DERIV: unknown variable
NIL
```

8.2.7 Output Function Comparison

Function	Newline	Quotes on strings	Returns
`PRINT`	Yes	Yes	The argument
`DISPLAY`	No	No	VOID (not visible in the REPL)
`NEWLINE`	Yes (only that)	—	`NIL`
`SYMNAME`	Doesn't print	—	String with the symbol's name
`STR<`	Doesn't print	—	`T` or `NIL`
`STRCAT`	Doesn't print	—	Concatenated string

8.3 Loading Programs with LOAD

`LOAD` reads and evaluates a LISP code file. It's how you work with programs too long for the REPL.

`LOAD` accepts the filename as a **string** or as a **symbol**:

```
(LOAD "myprog")   ; main form, with a string
(LOAD 'MYPROG)    ; alternative with a symbol (truncated to 8 characters)
```

The naming convention depends on the platform:

> **QL:** files carry a device prefix: `"mdv1_name"` (microdrive 1) or `"mdv2_name"` (microdrive 2). The string form is recommended because the underscore in the name isn't valid in a LISP symbol. **Next:** files live on the SD card with Unix-style names: `"utils.lsp"`, `"cas.lsp"`, etc. The extension is optional but helps identify files.

8.3.1 The Format of LISP Files

A LISP file is just text. It contains LISP expressions separated by spaces or line breaks. MYLISP reads and evaluates each expression in order:

```
; Example file (QL: mdv1_arith / Next: arith.lsp)

(DEFUN SQUARE (X) (* X X))
(DEFUN CUBE (X) (* X X X))
(DEFUN SUM-SQ (A B) (+ (SQUARE A) (SQUARE B)))

(PRINT "Arithmetic functions loaded")
```

Loading this file:

```
MYLISP> (LOAD "mdv1_arith")
"Arithmetic functions loaded"
T
```

The final `T` indicates that `LOAD` completed with no errors.

8.3.2 GC During LOAD

The GC's behavior during `LOAD` varies between platforms:

> **QL:** the GC triggers when the heap reaches **50%** of capacity (instead of the usual 80%). This is a safety measure: code files can define many functions and structures, and a more frequent GC guarantees there's enough room. **Next:** the GC does **not** trigger automatically during `LOAD`. It's assumed the 32,000-cell heap is enough for any reasonable file. If you want to force a collection between loads, use `(CLEAN)` from the REPL between one load and the next.

8.3.3 Organizing Code into Files

For larger projects (such as the CAS in Chapter 10), it's good practice to split the code into modules and load them in order:

```
; Main file (QL: mdv1_cas / Next: cas.lsp)

(LOAD "mdv1_utils")      ; QL  — list utilities
; (LOAD "utils.lsp")    ; Next — same utilities
(LOAD "mdv1_arithb")     ; basic arithmetic
(LOAD "mdv1_deriv")      ; symbolic differentiator
(LOAD "mdv1_simplif")    ; simplifier

(PRINT "CAS loaded and ready")
```

8.3.4 Filenames and the SYMLEN Limit

Remember that filenames passed to `LOAD` are **strings**, not symbols. Strings aren't subject to the 8-character symbol limit. That said, the QDOS filesystem has its own name-length restrictions.

```
MYLISP> (LOAD "mdv1_list-utilities")
```

This is valid as long as the name fits within the platform's filesystem.

8.3.5 What Happens If There's an Error During LOAD?

If an error occurs in some expression in the file, MYLISP shows the error message and **keeps evaluating the rest of the file**. Functions defined before the error remain available; those after the error may be incomplete.

That's why it's important to test functions individually at the REPL before organizing them into a file.

8.4 Typical File-Based Workflow

The recommended workflow for developing with MYLISP is:

1. **Design** the function on paper or in your head.
2. **Test** it at the REPL with simple cases:

   ```
   MYLISP> (DEFUN HELLO (X) (+ X 1))
   MYLISP> (HELLO 5)
   6
   ```

3. **Write** the code in a file using the text editor available on your platform (Quill or the SuperBASIC editor on the QL; any text editor on the Next, or on the PC if you're using CSpect).
4. **Load** the file with `LOAD`:

   ```
   MYLISP> (LOAD "mdv1_myfile")   ; QL
   MYLISP> (LOAD "myfile.lsp")    ; Next
   ```

5. **Test** the loaded functions at the REPL.

6. **Fix** the file and repeat from step 4 if there are errors.

8.5 A Complete Session: A Times-Table Program

As a complete example of using `PRINT` and `LOAD`, here's a small program that generates a multiplication table:

```
; File: mdv1_tables (QL) / tables.lsp (Next)

(DEFUN TABLE-ROW (N M)
  (COND
    ((> M 10) NIL)
    (T (PROGN
         (PRINT (* N M))
         (TABLE-ROW N (+ M 1))))))

(DEFUN TABLE (N)
  (PROGN
    (PRINT "Table of:")
    (PRINT N)
    (TABLE-ROW N 1)))
```

Loading and using it:

```
MYLISP> (LOAD "mdv1_tables")   ; QL
MYLISP> (LOAD "tables.lsp")    ; Next
"Tables loaded"
MYLISP> (TABLE 7)
"Table of:"
7
7
14
21
28
35
42
49
56
63
70
NIL
```

The final `NIL` is the return value when `M > 10` in the base case.

8.6 Summary of Practical Limitations

When working with strings and files, keep in mind:

- Strings: 36 characters maximum, 50 in memory at once, maximum.
- `LOAD`: the filename must include the device prefix (`mdv1_`, `mdv2_`) on the QL.
- The GC triggers at 50% during `LOAD` (vs. 80% at the REPL) on the QL.
- There's no way to read from the keyboard from inside a program; all interactive input goes through the REPL.

Continues in Chapter 9: Program Gallery

Chapter 9 — Program Gallery

9.1 Programming in MYLISP: The Functional Style

This chapter gathers complete programs written in MYLISP. Each illustrates techniques you can reuse in your own projects. The style is functional: functions don't modify their arguments, they return new values, and the composition of simple functions builds solutions to complex problems.

All the programs are meant to be loaded from a file with `LOAD`. Names are 8 characters or fewer.

9.2 Factorial: Three Versions

The **factorial** of a natural number `n`, written `n!`, is the product of every positive integer from `1` to `n`: `n! = 1 · 2 · 3 · ... · n`, with the special case `0! = 1` (the empty product). So `5! = 1 · 2 · 3 · 4 · 5 = 120`. It's the function that counts, for instance, in how many distinct ways `n` objects can be arranged in a row, and it appears constantly in combinatorics and in Taylor series.

The mathematical definition is already recursive — `n! = n · (n-1)!`, with base case `0! = 1` — so it translates to LISP almost word for word. Let's look at three ways of writing that same idea, each with a different nuance.

9.2.1 Direct Version

```
(DEFUN FACT (N)
  (IF (= N 0) 1 (* N (FACT (- N 1)))))
```

Simple and clear. It builds a chain of multiplications on the call stack.

9.2.2 Version with an Accumulator

```
(DEFUN FACT-ACC (N ACC)
  (IF (= N 0) ACC (FACT-ACC (- N 1) (* N ACC))))

(DEFUN FACT (N) (FACT-ACC N 1))
```

The result accumulates in `ACC`. At each call, the partial result is already computed. Conceptually equivalent, but with tail structure.

9.2.3 Version with COND to Validate Input

```
(DEFUN FACT (N)
  (COND
    ((< N 0) (PRINT "Error: negative"))
    ((= N 0) 1)
    (T (* N (FACT (- N 1))))))
```

```
MYLISP> (FACT 6)
720
MYLISP> (FACT -1)
"Error: negative"
"Error: negative"
```

9.3 Fibonacci Numbers

The **Fibonacci sequence** is one of the best-known numeric sequences: each term is the sum of the two preceding it, starting from `0` and `1`:

```
F(0)=0, F(1)=1, F(2)=1, F(3)=2, F(4)=3, F(5)=5, F(6)=8, F(7)=13, F(8)=21, F(9)=34, F(10)=55 ...
```

That is: `0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...` — each number is obtained by adding the two before it (`3 = 1 + 2`, `5 = 2 + 3`, `8 = 3 + 5`, and so on). The mathematical definition is the recurrence:

```
F(n) = F(n-1) + F(n-2),  with F(0) = 0 and F(1) = 1
```

Leonardo of Pisa (nicknamed *Fibonacci*) popularized it in Europe in 1202 in his work *Liber Abaci*, using it to model the growth of a rabbit population. Today it reappears in very different contexts: in botany (the arrangement of sunflower seeds), in the golden ratio (the quotient `F(n+1)/F(n)` converges to `φ ≈ 1.618`), and — what matters most to us here — as the canonical example for illustrating the difference between "naive" recursion and efficient recursion.

9.3.1 Direct Version (Exponential)

```
(DEFUN FIB (N)
  (COND
    ((= N 0) 0)
    ((= N 1) 1)
    (T (+ (FIB (- N 1)) (FIB (- N 2))))))
```

This version translates the mathematical recurrence almost letter for letter, which is why it's the easiest to read. But it's deceptively expensive: to compute `(FIB 5)` it needs `(FIB 4)` and `(FIB 3)`; to compute `(FIB 4)` it again needs `(FIB 3)` (again!) and `(FIB 2)`; and so on. The same subproblem gets recomputed over and over, a number of times that grows exponentially with `N`. Elegant but slow for large N: it computes the same values repeatedly.

```
MYLISP> (FIB 10)
55
MYLISP> (FIB 15)
610
```

9.3.2 Version with an Accumulator (Linear)

```
(DEFUN FIB-ACC (N A B)
  (COND
    ((= N 0) A)
    ((= N 1) B)
    (T (FIB-ACC (- N 1) B (+ A B)))))

(DEFUN FIB (N) (FIB-ACC N 0 1))
```

Here `A` and `B` are always two consecutive Fibonacci numbers: at the start, `A = F(0) = 0` and `B = F(1) = 1`. At each recursive call, the pair advances one position: the new `A` is the old `B`, and the new `B` is `A + B` — the next term in the series. No call is ever repeated: `(FIB-ACC 5 0 1)` visits the pair `(0,1)`, then `(1,1)`, then `(1,2)`, then `(2,3)`, then `(3,5)`, and returns `5`. This version is O(n): it computes each Fibonacci number exactly once, instead of recomputing it an exponential number of times.

```
MYLISP> (FIB 20)
6765
MYLISP> (FIB 25)
75025
```

9.3.3 Generating a List of Fibonacci Numbers

```
(DEFUN FIB-LIST (N)
  (COND
    ((= N 0) (LIST 0))
    (T (APPEND (FIB-LIST (- N 1))
               (LIST (FIB N))))))
```

```
MYLISP> (FIB-LIST 8)
(0 1 1 2 3 5 8 13 21)
```

9.4 Greatest Common Divisor

The **greatest common divisor** of two integers `A` and `B`, written `gcd(A, B)`, is the largest integer that divides both exactly. For example, `gcd(48, 18) = 6`, because `6` divides both and there's no larger common divisor.

Euclid's algorithm — described in Euclid's *Elements* more than 2000 years ago, and considered one of the oldest known algorithms — computes the `gcd` without needing to factor either number, resting on a single observation: **the `gcd` of two numbers doesn't change if we replace the larger one with the remainder of dividing it by the smaller one**. Formally:

```
gcd(A, B) = gcd(B, A mod B)      (if B ≠ 0)
gcd(A, 0) = A                    (base case)
```

Repeating that substitution, the numbers get smaller and smaller until one of them reaches `0` — at which point the other one *is* the `gcd` we're looking for. Traced by hand for `gcd(48, 18)`:

```
gcd(48, 18) = gcd(18, 48 mod 18) = gcd(18, 12)
            = gcd(12, 18 mod 12) = gcd(12, 6)
            = gcd(6,  12 mod 6)  = gcd(6, 0)
            = 6
```

The translation to MYLISP is direct thanks to the `MOD` primitive (section 4.9), which returns exactly the integer-division remainder the recurrence needs:

```
(DEFUN GCD (A B)
  (COND
    ((= B 0) A)
    (T (GCD B (MOD A B)))))
```

Each recursive call is exactly one step of the trace above: `(GCD B (MOD A B))` replaces the pair `(A, B)` with `(B, A mod B)`, exactly as the formula requires. When `B` reaches `0`, the base clause returns `A` — which by that point is already the greatest common divisor.

```
MYLISP> (GCD 48 18)
6
MYLISP> (GCD 100 75)
25
MYLISP> (GCD 17 13)
1
```

That last example, `gcd(17, 13) = 1`, shows two **coprime** numbers: they share no factor other than `1`.

9.4.1 Least Common Multiple

The **least common multiple** of `A` and `B`, `lcm(A, B)`, is the smallest positive integer that is a multiple of both at once. Rather than searching for it by brute force, it rests on an identity relating `gcd` and `lcm`: the product of the two numbers always equals the product of their `gcd` and their `lcm` (`A · B = gcd(A,B) · lcm(A,B)`), so we just need to solve for it:

```
(DEFUN LCM (A B)
  (/ (* A B) (GCD A B)))
```

```
MYLISP> (LCM 4 6)
12
MYLISP> (LCM 15 10)
30
```

Let's verify the identity with the first case: `gcd(4,6) = 2` and `lcm(4,6) = 12`, and indeed `2 · 12 = 24 = 4 · 6`.

9.5 Sorting Lists

Sorting a list means rearranging its elements from smallest to largest. We'll look at two classic strategies that are very different from each other: **insertion sort**, which builds the result by inserting one element at a time into its proper place, and **quicksort**, which splits the problem into smaller pieces and solves them separately.

9.5.1 Sorted Insertion

```
(DEFUN INSERT-SORTED (X L)
  (COND
    ((NULL L) (LIST X))
    ((<= X (CAR L)) (CONS X L))
    (T (CONS (CAR L) (INSERT-SORTED X (CDR L))))))
```

Inserts `X` into the correct position of the already-sorted list `L`:

```
MYLISP> (INSERT-SORTED 4 '(1 3 5 7))
(1 3 4 5 7)
```

9.5.2 Insertion Sort

The idea behind **insertion sort** is the same one anyone uses to sort cards in their hand: you take a card from the deck and insert it in its place among the ones already sorted; you repeat with the next card; and so on until none are left to place. `INSERTION-SORT` translates this into recursion: to sort the whole list, first recursively sort the rest (`CDR L`), then insert the head (`CAR L`) into the correct place in that already-sorted rest, using `INSERT-SORTED`.

```
(DEFUN INSERTION-SORT (L)
  (COND
    ((NULL L) NIL)
    (T (INSERT-SORTED (CAR L) (INSERTION-SORT (CDR L))))))
```

```
MYLISP> (INSERTION-SORT '(5 3 8 1 9 2 6))
(1 2 3 5 6 8 9)
MYLISP> (INSERTION-SORT '(3 1 4 1 5 9 2 6))
(1 1 2 3 4 5 6 9)
```

It's easy to understand and to code, but its cost grows with the square of the list's size ($O(n^2)$): each insertion may, in the worst case, walk through the entire already-sorted part.

9.5.3 Quicksort

Quicksort follows a different, "divide and conquer" strategy: instead of inserting one element at a time, it picks any element of the list as the **pivot** (here, always the first one), splits the rest into two groups — those smaller than the pivot and those greater than or equal to it — sorts each group separately (recursively, with the same algorithm), and finally concatenates: sorted smaller elements, pivot, sorted greater elements. `LESS-THAN` and `GEQ-THAN` are the two filters that build those groups; `QSORT` combines them.

```
(DEFUN LESS-THAN (X L)
  (COND
    ((NULL L) NIL)
    ((< (CAR L) X) (CONS (CAR L) (LESS-THAN X (CDR L))))
    (T (LESS-THAN X (CDR L)))))

(DEFUN GEQ-THAN (X L)
  (COND
    ((NULL L) NIL)
    ((>= (CAR L) X) (CONS (CAR L) (GEQ-THAN X (CDR L))))
    (T (GEQ-THAN X (CDR L)))))

(DEFUN QSORT (L)
  (COND
    ((NULL L) NIL)
    ((NULL (CDR L)) L)
    (T (APPEND
         (QSORT (LESS-THAN (CAR L) (CDR L)))
         (LIST (CAR L))
         (QSORT (GEQ-THAN (CAR L) (CDR L)))))))
```

```
MYLISP> (QSORT '(3 6 8 10 1 2 1))
(1 1 2 3 6 8 10)
MYLISP> (QSORT '(5 4 3 2 1))
(1 2 3 4 5)
```

Traced by hand, `(QSORT '(3 6 8 1))` takes `3` as the pivot, splits `(1)` (smaller) from `(6 8)` (greater or equal), sorts each group separately — trivial here, since they're single-element lists or already sorted — and rebuilds `(1) ++ (3) ++ (6 8) = (1 3 6 8)`. On average, quicksort is much faster than insertion sort ($O(n \log n)$ versus $O(n^2)$), but since the pivot here is always the first element, a list that was already sorted (or sorted in reverse) would produce the worst possible case, also $O(n^2)$.

9.6 Searching Lists

9.6.1 Linear Search

Linear search is the most basic strategy possible: walk through the list element by element, comparing each one to what we're looking for, until we find it or reach the end. It doesn't require the list to be sorted, but in the worst case — the element isn't there, or it's at the end — it has to look at every element: O(n).

```
(DEFUN SEARCH (X L)
  (COND
    ((NULL L) NIL)
    ((EQUAL X (CAR L)) T)
    (T (SEARCH X (CDR L)))))
```

```
MYLISP> (SEARCH 5 '(1 3 5 7 9))
T
MYLISP> (SEARCH 4 '(1 3 5 7 9))
NIL
```

9.6.2 Search That Returns a Position

```
(DEFUN POSITION-OF (X L N)
  (COND
    ((NULL L) -1)
    ((EQUAL X (CAR L)) N)
    (T (POSITION-OF X (CDR L) (+ N 1)))))

(DEFUN WHERE (X L) (POSITION-OF X L 0))
```

```
MYLISP> (WHERE 'C '(A B C D E))
2
MYLISP> (WHERE 'Z '(A B C D E))
-1
```

9.6.3 Binary Search in a Sorted List

When data is **sorted**, there's no need to look at it one element at a time: **binary search** compares the target element to the one right in the middle of the collection. If it matches, we're done. If it's smaller, the element can only be in the left half; if it's larger, only in the right half. Each comparison discards half of what remained, so the number of steps grows with the logarithm of the size (O(log n)) instead of with the size itself — over an array, searching among a million sorted elements only takes about 20 comparisons.

The problem is that this efficiency depends on being able to jump directly to the middle element, and a LISP linked list doesn't allow that: to reach the Nth element you have to walk through the N-1 before it with `CDR`, one at a time.

```
(DEFUN NTH (L N)
  (IF (= N 0)
    (CAR L)
    (NTH (CDR L) (- N 1))))

(DEFUN MIDDLE (L)
  (NTH L (/ (LENGTH L) 2)))
```

`NTH` gets the Nth element by walking the list, and `MIDDLE` uses it to reach the central element — but that access alone already costs O(n), the same as walking the whole list. The "savings" from binary search get lost exactly at the step that should be instant. That's why we say binary search over lists is costly: in practice, it doesn't improve on linear search. To truly take advantage of the "discard half at each step" idea you need a structure with direct access, like search trees (section 5.8), where descending left or right really is a constant-cost operation.

9.7 MAP, FILTER, and REDUCE Defined in Pure LISP

These three higher-order functions are fundamental in functional programming. MYLISP doesn't include them as primitives, but we can define them in a few lines.

9.7.1 MY-MAP: Transforming Each Element

```
(DEFUN MY-MAP (F L)
  (COND
    ((NULL L) NIL)
    (T (CONS (F (CAR L))
             (MY-MAP F (CDR L))))))
```

```
MYLISP> (MY-MAP (LAMBDA (X) (* X X)) '(1 2 3 4 5))
(1 4 9 16 25)
MYLISP> (MY-MAP (LAMBDA (X) (CONS X (LIST (* X X))))
                '(1 2 3 4))
((1 1) (2 4) (3 9) (4 16))
```

9.7.2 MY-FILTER: Selecting Elements

```
(DEFUN MY-FILTER (PRED L)
  (COND
    ((NULL L) NIL)
    ((PRED (CAR L))
     (CONS (CAR L) (MY-FILTER PRED (CDR L))))
    (T (MY-FILTER PRED (CDR L)))))
```

```
MYLISP> (MY-FILTER (LAMBDA (X) (> X 3)) '(1 2 3 4 5 6))
(4 5 6)
MYLISP> (MY-FILTER (LAMBDA (X) (= 0 (MOD X 2)))
                   '(1 2 3 4 5 6 7 8 9 10))
(2 4 6 8 10)
```

The second example filters even numbers using the `MOD` primitive (explained in section 4.9).

9.7.3 MY-REDUCE: Reducing to an Accumulated Value

```
(DEFUN MY-REDUCE (F INIT L)
  (COND
    ((NULL L) INIT)
    (T (MY-REDUCE F
                  (F INIT (CAR L))
                  (CDR L)))))
```

```
MYLISP> (MY-REDUCE + 0 '(1 2 3 4 5))
15
MYLISP> (MY-REDUCE * 1 '(1 2 3 4 5))
120
MYLISP> (MY-REDUCE (LAMBDA (A X) (CONS X A)) NIL '(1 2 3))
(3 2 1)
```

The last example reverses a list using `REDUCE`.

9.7.4 Composing MAP, FILTER, and REDUCE

```
; Sum of squares of the odd numbers from 1 to 10
MYLISP> (MY-REDUCE
          +
          0
          (MY-MAP
            (LAMBDA (X) (* X X))
            (MY-FILTER
              (LAMBDA (X) (NOT (= 0 (- X (* (/ X 2) 2)))))
              '(1 2 3 4 5 6 7 8 9 10))))
165
```

The odd numbers from 1 to 10 are `(1 3 5 7 9)`. Their squares are `(1 9 25 49 81)`. Their sum is `165`.

9.8 Square Root by Newton's Method

Newton's method (or Newton-Raphson) is a general technique for approximating the roots of a function through successive improvements of an initial estimate. To compute `√x`, we frame the problem as "find the root of `f(G) = G² − x`" and apply Newton's general formula, which produces this update rule:

```
G_new = (G + X/G) / 2
```

That is: if `G` is an estimate of `√x`, the average of `G` and `X/G` is a *better* estimate. Intuitively, if `G` undershoots, `X/G` overshoots (and vice versa), so averaging pulls them together from both sides. Repeating the process, `G` converges very quickly to the true value — the number of correct digits roughly doubles at each step.

Traced by hand for `√2`, starting from `G = 1.0`:

```
G0 = 1.0
G1 = (1.0 + 2.0/1.0) / 2 = 1.5
G2 = (1.5 + 2.0/1.5) / 2 = 1.41666...
G3 = (1.41666... + 2.0/1.41666...) / 2 = 1.41421...
```

After three steps it already matches `√2 ≈ 1.41421356` to five decimal places. Since `SQRT` isn't implemented as a primitive in MYLISP (Appendix B.7), we can approximate it with this method:

```
(DEFUN GOOD-ENOUGH (G X)
  (< (ABS (- (* G G) X)) 0.001))

(DEFUN IMPROVE-GUESS (G X)
  (/ (+ G (/ X G)) 2.0))

(DEFUN SQRT-ITER (G X)
  (IF (GOOD-ENOUGH G X)
    G
```

```
    (SQRT-ITER (IMPROVE-GUESS G X) X)))

(DEFUN MY-SQRT (X)
  (SQRT-ITER 1.0 X))

(DEFUN ABS (X)
  (IF (< X 0) (- X) X))
```

```
MYLISP> (MY-SQRT 2.0)
1.41422...
MYLISP> (MY-SQRT 9.0)
3.00009...
MYLISP> (MY-SQRT 25.0)
5.00000...
```

The approximation converges in a few steps. The `GOOD-ENOUGH` function checks whether the square of the estimate is close enough to the target value.

9.9 Reversing a List (Efficient Version)

Reversing a list means producing a new list with the same elements in the opposite order: `(1 2 3 4 5)` reversed is `(5 4 3 2 1)`. The version from Chapter 5 was O(n²) because it used `APPEND` at every step (`APPEND` has to walk its entire first argument, section 5.3). This accumulator version avoids `APPEND` entirely: at each step, it moves the head of `L` to the head of `ACC` with `CONS`, which is a constant-cost operation. The result is O(n):

```
(DEFUN REVERSE-ACC (L ACC)
  (IF (NULL L)
    ACC
    (REVERSE-ACC (CDR L) (CONS (CAR L) ACC))))

(DEFUN REVERSE (L)
  (REVERSE-ACC L NIL))
```

```
MYLISP> (REVERSE '(1 2 3 4 5))
(5 4 3 2 1)
MYLISP> (REVERSE '(A B C D))
(D C B A)
MYLISP> (REVERSE NIL)
NIL
```

9.10 Flattening a Nested List

"Flattening" a list means walking a structure with sublists nested to any depth and producing a single flat list with all the atoms it contained, in the same order. `FLATTEN` distinguishes two cases while walking the list: if the head is an atom (not a list), it's kept as is and we continue with the rest; if the head is itself a list, it also needs to be flattened, and the result concatenated with the flattened rest. Converting a list with sublists into a flat list:

```
(DEFUN FLATTEN (L)
  (COND
    ((NULL L) NIL)
    ((ATOM (CAR L))
     (CONS (CAR L) (FLATTEN (CDR L))))
    (T (APPEND (FLATTEN (CAR L))
               (FLATTEN (CDR L))))))
```

```
MYLISP> (FLATTEN '(1 (2 3) (4 (5 6)) 7))
(1 2 3 4 5 6 7)
MYLISP> (FLATTEN '((A B) (C (D E))))
(A B C D E)
```

9.11 Primes: The Sieve of Eratosthenes

A **prime** number is an integer greater than 1 that's only divisible by 1 and itself. The **Sieve of Eratosthenes**, attributed to the Greek mathematician Eratosthenes of Cyrene (3rd century BC), is one of the oldest and most elegant methods for finding every prime up to a given limit: start from the list of every integer from 2 to N, and repeatedly take the first number remaining (which is always prime, because nothing smaller than it has eliminated it), remove every one of its multiples from the list, and repeat with the next surviving number.

Traced by hand for the primes up to 20: we start from `(2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20)`. We take `2` (prime) and remove its multiples: what's left is `(3 5 7 9 11 13 15 17 19)`. We take `3` (prime) and remove its multiples (`9`, `15`): what's left is `(5 7 11 13 17 19)`. Everything that survives from here on is already prime, because any composite number less than or equal to 20 must have a factor less than or equal to `√20 ≈ 4.5`, and we've already sieved by 2 and by 3.

The implementation translates that process directly: `WITHOUT-MULTIPLES` filters out the multiples of a given number (using `MOD`, section 4.9), `RANGE` generates the initial list of candidates, and `SIEVE` applies the whole process, always taking the head of the list as the next confirmed prime:

```
; Filters out the multiples of N from a list, using MOD (section 4.9)
(DEFUN WITHOUT-MULTIPLES (N L)
  (MY-FILTER
    (LAMBDA (X) (NOT (= 0 (MOD X N))))
    L))

; Generates a list of integers from A to B
(DEFUN RANGE (A B)
  (IF (> A B)
    NIL
    (CONS A (RANGE (+ A 1) B))))

; Sieve of Eratosthenes
(DEFUN SIEVE (L)
  (COND
    ((NULL L) NIL)
    (T (CONS (CAR L)
             (SIEVE (WITHOUT-MULTIPLES (CAR L) (CDR L)))))))

(DEFUN PRIMES (N)
  (SIEVE (RANGE 2 N)))
```

```
MYLISP> (PRIMES 30)
(2 3 5 7 11 13 17 19 23 29)
MYLISP> (PRIMES 50)
(2 3 5 7 11 13 17 19 23 29 31 37 41 43 47)
```

Continues in Part II: Design and Structure of a CAS

\newpage

Part II — Symbolic Computation: Building a CAS

Chapter 10 — Representation and Data Abstraction

> **In one sentence:** we're going to represent mathematical expressions as unevaluated Lisp lists, and build an access interface to those lists (predicates, selectors, constructors) so that no future function in the CAS ever has to manipulate the list structure by hand.

10.1 Motivation: What a CAS Is, and Why LISP

You've surely seen some calculator, a math homework app, or a search engine where you type `derivative of x^2` and it instantly answers `2x`. Or you ask it to simplify `(x + 1)^2 - (x^2 + 2x)` and it answers `1`. It's not magic (though it seemed like it to me as a kid), nor does the program "know" math the way a person does: it's **symbolic algebra** — manipulating mathematical expressions as structures, applying mechanical rules, without needing to know what concrete number `x` corresponds to.

That's the difference from what you already know how to do in MYLISP since Chapter 4: `(+ 2 3)` is **numeric computation** — the interpreter substitutes, operates, and returns `5`. But `x + x → 2x` involves no number at all: `x` has no concrete value, and yet we know for certain that the expression can be rewritten in an equivalent, simpler form. The same happens with differentiation: the power rule says the derivative of x^n is $n \cdot x^{n-1}$ — a purely syntactic transformation on the *shape* of the expression, not on its numeric value. A **CAS** (*Computer Algebra System*) is precisely a program that knows how to apply those transformations: differentiate, simplify, and — though this manual doesn't cover it — integrate, factor, or solve equations, all while keeping expressions in exact symbolic form instead of approximating them with numbers.

This isn't an isolated academic exercise: CAS software has existed for decades and is a real working tool in science and engineering — Macsyma (Matlab, Mathematica, Maple, Maxima, and SymPy are its relatives or descendants, each with its own history). And here a circle closes with Part I of this manual: **Macsyma**, developed starting in 1968 at MIT, one of the first and most influential symbolic algebra systems in history, was written in **MacLisp** — the very dialect from which MYLISP borrows much of its syntax (§1.4.2). That's no coincidence. From its origin in 1958, LISP was a notation designed to represent and manipulate symbolic expressions (§1.1.1) — and that's exactly what a CAS needs.

10.1.1 Why LISP Fits This Problem So Well

Above all, a CAS needs a way to represent a mathematical expression as a data structure the program can inspect and transform — what computer science calls an **abstract syntax tree** (AST). In most languages, building that requires designing a new data type, writing a parser, and keeping separate functions for "code" and for "data." In LISP none of that is needed, thanks to a property you already saw in Chapter 3: **homoiconicity** (§3.6) — code and data share the same syntax, the S-expression. The list `(+ X 3)` already *is* a tree: `+` is the operation, and `X` and `3` are its two branches. There's nothing to invent; we only need to decide, as we will in §10.2, what convention to follow for organizing those lists.

Everything else we need was already built in Part I, and we'll use it constantly from here on:

- **`QUOTE`** (§3.5) so we can write `'(+ X 3)` as data, without the interpreter trying to evaluate it as a call to `+`.
- **Recursion** (§7.6) to walk nested expressions: differentiating a sum requires differentiating each of its addends, which may themselves be sums or products — the same "solve the base case, combine with the solution of the rest" idea you already used to sum lists or reverse them.
- **Lists and their predicates, selectors, and constructors** (Chapter 5) to classify, extract, and rebuild the pieces of an expression, without touching raw `CAR` / `CDR` in every new function (a rule that becomes formal law in §10.3).
- **`COND`** (§6.3) for the case cascades that constantly appear when walking an expression: "if it's a number, do this; if it's a sum, do that; if it's a product, ...".

- **`LAMBDA` and closures** (§7.2, §7.9) for the higher-order functions that will show up when walking and transforming lists of terms.

In other words: Part II doesn't introduce a new language or special tricks. It builds a small but real CAS — capable of differentiating and simplifying algebraic expressions — by combining, chapter by chapter, the same MYLISP pieces you already know. The final goal, which we'll reach piece by piece across Chapters 10 through 19, looks like this:

```
MYLISP> (DERIVA '(POT X 2) 'X)
(* 2 X)
```

That's exactly the `x² → 2x` this section opened with — except now you'll know, line by line, how it's built.

10.2 The Decision That Shapes Everything: How We Represent an Expression

We're going to represent a mathematical expression as an **unevaluated** MyLISP list. That is, `X + 3` will be represented literally as the list `(+ X 3)`, but we will never hand it to the evaluator as-is — it will always be quoted with `QUOTE` (or `'`), because otherwise the interpreter itself would evaluate it as a call to the `+` function and we'd lose the symbolic expression.

```
(DEFINE EXPR1 '(+ X 3))   ; EXPR1 is DATA, it isn't evaluated
EXPR1                     ; -> (+ X 3)
```

This is the foundation of everything: in MyLISP, code and data share the same syntax (S-expressions), so an algebraic AST comes "for free" — we don't need to invent a new format, just *decide on a convention* for how those lists are organized.

10.2.1 Convention for Chapter 10 (Binary, Not Yet Simplified)

Mathematical expression	MyLISP representation
constant	number: `3`, `1/2`, `2.5`
variable	symbol: `X`, `Y`
sum `a + b`	`(+ a b)`
product `a * b`	`(* a b)`
power `a^b`	`(POT a b)`

Note on power: we can't use `^` as an operator because MyLISP's lexer only recognizes alphanumeric characters plus `-`, `?`, `!` (and the already-predefined arithmetic operators like `+`, `*`, `/`) as valid in a symbol. `^` would produce a lexical error. That's why we use the word `POT`.

For now we work **binary only** (`(+ a b)`, not `(+ a b c)`), even though MyLISP's real `+` is variadic — this is a deliberate pedagogical decision, just as in SICP. Extending to variadic form will be a natural exercise later, once the simplifier exists.

10.3 The Golden Rule: Never Touch `CAR`/`CDR` Directly Outside This Chapter

From here on, **no other function in the CAS will access the list structure directly**. Everything will go through constructors, selectors, and predicates. This is the "abstraction barrier": if tomorrow we decide to change the representation (for example, `(SUMA a b)` instead of `(+ a b)`, or switch to tagged lists), only *this* chapter needs to be rewritten — nothing else finds out about the change.

10.4 Platform Note: The 8-Character Trap

MyLISP truncates symbol names to **8 significant characters, silently, with no warning**. If we used SICP's classic naming as-is (`MULTIPLIER` / `MULTIPLICAND`), both would truncate to `MULTIPLI` and **collide** — they'd become the same symbol, with incorrect results and no error message at all. That's why this manual has used, from the start, a table of short names designed not to collide:

SICP	MyLISP (≤8 chars, no collision)
`addend` / `augend`	`ADDEND` / `AUGEND`
`multiplier` / `multiplicand`	`FACTOR1` / `FACTOR2`
`make-sum` / `make-product`	`MAKESUM` / `MAKEPROD`
`is-sum?` / `is-product?`	`ISSUM?` / `ISPROD?`
`base` / `exponent`	`BASE` / `EXPON`
`make-exponentiation`	`MAKEPOW`

10.5 Classification Predicates

```
(DEFUN ISCONST? (E) (NUMBERP E))

(DEFUN ISVAR? (E) (SYMBOLP E))

(DEFUN ISSUM? (E)
  (AND (LISTP E) (EQ (CAR E) '+)))

(DEFUN ISPROD? (E)
  (AND (LISTP E) (EQ (CAR E) '*)))

(DEFUN ISPOW? (E)
  (AND (LISTP E) (EQ (CAR E) 'POT)))
```

`EQ` compares interned symbols by identity — it's the correct, cheapest comparison for checking the operator at the head of the list (we don't need `EQUAL`, which would walk structures).

10.6 Selectors

```
(DEFUN ADDEND (E) (CAR (CDR E)))          ; first addend
(DEFUN AUGEND (E) (CAR (CDR (CDR E))))    ; second addend

(DEFUN FACTOR1 (E) (CAR (CDR E)))         ; first factor
(DEFUN FACTOR2 (E) (CAR (CDR (CDR E))))   ; second factor

(DEFUN BASE  (E) (CAR (CDR E)))           ; base of the power
(DEFUN EXPON (E) (CAR (CDR (CDR E))))     ; exponent of the power
```

10.7 Constructors (Still "Dumb" — Not Yet Simplifying)

```
(DEFUN MAKESUM (A1 A2) (LIST '+ A1 A2))
(DEFUN MAKEPROD (A1 A2) (LIST '* A1 A2))
(DEFUN MAKEPOW  (B E)   (LIST 'POT B E))
```

We deliberately do **not** check here whether `A1` is `0` or `A2` is `1`. That comes in the simplifier chapter, once its absence actually hurts (once we see `DERIV` explode). For now, a constructor only constructs.

On the name: it might seem shorter and more direct to call them `SUM`, `PROD`, `POW` instead of `MAKESUM`, `MAKEPROD`, `MAKEPOW`. Right now, with such simple constructors, that difference looks merely cosmetic. Chapter 13 explains why the `MAKE` prefix stops being cosmetic as soon as these functions start doing more than packaging arguments.

10.8 Name Check (No 8-Character Collisions)

Name	Length	Collides?
`ISCONST?`	8	exact, no margin
`ISVAR?`	6	no
`ISSUM?`	6	no
`ISPROD?`	7	no
`ISPOW?`	6	no
`ADDEND`	6	no
`AUGEND`	6	no
`FACTOR1` / `FACTOR2`	7	no
`BASE` / `EXPON`	4 / 5	no
`MAKESUM`	8	exact, no margin — careful if a suffix is ever added
`MAKEPROD`	8	same
`MAKEPOW`	7	ok

10.9 Trying It in the REPL

```
(DEFINE E1 (MAKESUM 'X 3))
E1                          ; -> (+ X 3)

(ISSUM? E1)                 ; -> T
(ISPROD? E1)                ; -> NIL
(ADDEND E1)                 ; -> X
(AUGEND E1)                 ; -> 3

(DEFINE E2 (MAKEPROD (MAKESUM 'X 1) 'Y))
E2                          ; -> (* (+ X 1) Y)
(ISPROD? E2)                ; -> T
(FACTOR1 E2)                ; -> (+ X 1)
(ISSUM? (FACTOR1 E2))       ; -> T

(DEFINE E3 (MAKEPOW 'X 2))
E3                          ; -> (POT X 2)
(ISPOW? E3)                 ; -> T
(BASE E3)                   ; -> X
(EXPON E3)                  ; -> 2

(ISCONST? 3)                ; -> T
```

```
(ISVAR? 'X)                  ; -> T
(ISCONST? 'X)                ; -> NIL
```

10.10 Conscious Technical Debt

`ISVAR?` as currently defined would also say `T` for `T` and `NIL`, because internally they're symbols. We won't fix this yet — it's exactly the kind of edge case worth noting and resolving once a real case that needs it shows up (probably in `DERIV`, when differentiating with respect to a specific variable).

10.11 Summary of What We Built in This Chapter

- **Representation:** mathematical expressions as unevaluated MyLISP lists, with `+`, `*`, and `POT` as binary operators.
- **Predicates:** `ISCONST?`, `ISVAR?`, `ISSUM?`, `ISPROD?`, `ISPOW?`.
- **Selectors:** `ADDEND` / `AUGEND`, `FACTOR1` / `FACTOR2`, `BASE` / `EXPON`.
- **Constructors:** `MAKESUM`, `MAKEPROD`, `MAKEPOW` (not yet simplifying — that arrives in Chapter 13).
- **Golden rule established:** no function outside this chapter touches `CAR` / `CDR` directly on a CAS expression.

10.12 Next Chapter

Chapter 11 — Substitution (`SUBST`) and Numeric Evaluation: the first function that walks the AST built in this chapter, using only the constructors/selectors/predicates defined here — never direct `CAR` / `CDR`.

Continues in Chapter 11: Substitution (SUBST) and Numeric Evaluation

---

Chapter 11 — Substitution (SUBST) and Numeric Evaluation

> **In one sentence:** we're going to write the first two functions that truly walk the tree from Chapter 10 — `SUBST` to substitute variables, `EVALEXPR` to reduce to a number — and with them we fix the traversal pattern that will repeat throughout the rest of the CAS, while also working out how to signal a domain failure in a Lisp with no exceptions.

11.1 The Tree We Already Had Without Knowing It

In languages like C or Java, representing an abstract syntax tree (AST) forces you to define explicit classes, nodes, and pointers: a `SumNode` class with two fields, `left` and `right`; another `VariableNode` with a name; some inheritance or type-union mechanism so every node fits the same structure... In Lisp none of that is needed.

An expression like

```
'(+ (* 2 X) 3)
```

already is the AST. It isn't a representation *of* a tree — it is a tree: a list whose first element is the operator and whose remaining elements are the operands, which may in turn be lists (subtrees) or atoms (leaves). This property — that code and data share the same syntactic form, the S-expression — is called **homoiconicity**, and it's the reason Lisp has been the preferred language for building symbolic systems since the 1960s.

Chapter 10 didn't build a new data structure. It built the **access interface** to a tree that MyLISP's lists already gave us for free: `ISSUM?`, `ISPROD?`, `ISPOW?` are the "what kind of node is this?" questions; `ADDEND`, `AUGEND`, `FACTOR1`, `FACTOR2`, `BASE`, `EXPON` are the "what are its children?" questions; and `MAKESUM`, `MAKEPROD`, `MAKEPOW` are the way to build a new node without touching the list by hand.

The goal of this chapter is to learn to **navigate** that tree: visit each node, decide what to do based on its type, recurse down into the children, and reassemble the result on the way back up. It's the pattern we'll repeat in practically every function of the CAS from here on.

11.2 1. Symbolic Substitution

11.2.1 The Idea

Substituting means walking the tree and, every time we reach a leaf that's the variable we're looking for, replacing it with something else — a number, another variable, or an entire subexpression. The result is still a valid tree in our representation, so it can be substituted again, differentiated, or whatever else, on top of it.

11.2.2 The Recursive Structure (the Pattern That Will Always Repeat)

Every function that walks our expressions has the same shape:

1. **Base case — constant.** A constant contains no variable. It's returned as is.
2. **Base case — variable.** If it's the variable we're looking for, the substitution value is returned. If it's a different variable, it's returned unchanged.
3. **Recursive case — sum / product / power.** There's no leaf to examine yet: we need to **go down** into the children, substitute in each one separately, and **come back up**, rebuilding the node with the corresponding constructor.

Point (3) is the key that distinguishes "manipulating a list" from "manipulating a tree abstractly": at no point do we use `CAR`/`CDR` directly on `E` inside `SUBST` — we use the selectors from Chapter 10, and we rebuild with the constructors from Chapter 10. If tomorrow we change the internal representation, `SUBST` doesn't find out.

11.2.3 The Code

```
(DEFUN SUBST (E VBL VAL)
  (COND ((ISCONST? E) E)
        ((ISVAR? E)
         (COND ((EQ E VBL) VAL)
               (T E)))
        ((ISSUM? E)
         (MAKESUM (SUBST (ADDEND E) VBL VAL)
                  (SUBST (AUGEND E) VBL VAL)))
        ((ISPROD? E)
         (MAKEPROD (SUBST (FACTOR1 E) VBL VAL)
                   (SUBST (FACTOR2 E) VBL VAL)))
        ((ISPOW? E)
         (MAKEPOW (SUBST (BASE E) VBL VAL)
                  (SUBST (EXPON E) VBL VAL)))))
```

A note on the `COND`: there's no final `(T ...)` clause for "unknown type, error." This is a deliberate decision, not an oversight — if in the future we add a new node type (for example, `SIN`/`COS`) and forget to give it a case here, the interpreter will silently return `NIL` instead of breaking with a loud error. It's a debatable choice between "fail fast and loud" or "fail silent" — we leave it noted as an open design decision, revisable if in practice it causes more confusion than protection.

11.2.4 Tracing It by Hand (Recommended Before Coding It)

For `(SUBST '(+ X 3) 'X 5)`:

1. `E` is `(+ X 3)` → `ISSUM?` is true.
2. It's decomposed: `ADDEND` → `X`, `AUGEND` → `3`.

3. Each is substituted separately: `(SUBST 'X 'X 5)` → `5` (variable case, matches); `(SUBST '3 'X 5)` → `3` (constant case, unchanged).
4. It's rebuilt: `(MAKESUM 5 3)` → `(+ 5 3)`.

The result **doesn't reduce to `8`** — `SUBST` doesn't simplify, it only substitutes. That's intentional: separating "substitute" from "simplify" is exactly what will let us clearly see, in the next chapter, the problem the simplifier solves.

11.3 2. Numeric Evaluation

11.3.1 The Idea

Numerically evaluating an expression at a point is, fundamentally, applying `SUBST` until no free variable remains, and then reducing each operator in the tree to its numeric result. The first temptation is to delegate the whole tree to the host interpreter's `EVAL`:

```
(DEFUN EVALEXPR (E) (EVAL E))   ; naive version -- see below for why not
```

This works for a reason worth exposing: `+` and `*` are symbols that **happen** to coincide with real MyLISP functions with exactly the same semantics we've given them in our AST. It isn't that `EVALEXPR` is "evaluating the CAS's sum operator" — it's blindly handing the whole tree to the interpreter and trusting that the names match. That coincidence breaks the moment `POT` shows up, which we made up ourselves and MyLISP doesn't know about.

If the CAS has to be a portable library over any Lisp — without requiring the host interpreter to know its operators — then that requirement must apply **equally** to `+`, `*`, and `POT`. The solution is for `EVALEXPR` to use the same recursive skeleton as `SUBST`: classify with the predicates from Chapter 10, go down into the children, reduce each operator explicitly.

But there's a second, deeper question hiding behind the first: **what does `EVALEXPR` do when it reaches a leaf that's still a variable?** The naive version answered "ask MyLISP's global environment with `(EVAL E)`" — and that mixes two binding systems our design wants to keep separate: the *mathematical* binding created by `SUBST` (an explicit substitution inside the CAS's tree) and the *program* binding created by `DEFINE`/`LET` in the host interpreter's environment. If a CAS user tries to evaluate `X + 2` without having done `SUBST` first, the right answer isn't "ask MyLISP whether it happens to have a global variable called `X`" — it's for the CAS's own domain to clearly say: *this expression has a free variable, it can't be reduced to a number*.

11.3.2 Why We Can't "Throw an Error" — and What We Do Instead

In Common Lisp or Scheme we'd write something like `(error "free variable")` and the condition system would take care of interrupting evaluation in a controlled way. **MyLISP has none of that.** There's no `ERROR` function callable from Lisp, and there's no exception mechanism or `CONDITION-CASE`. The only errors that exist are the ones the interpreter itself throws internally on an incorrect type (for example, adding a symbol), and those are unrecoverable from Lisp code — they abort evaluation without our functions being able to intercept them.

This limitation, far from being an obstacle, forces us to use the only error-handling technique that **is** expressible in a Lisp this minimal: instead of *throwing* an error, **return a value that means error**, and make whoever calls the function check for it explicitly before continuing. It's a pattern with its own name and a long tradition:

- In **C**, it's the special return code (`-1`, `NULL`, `errno`) that every function must check by hand after each call, because the language has no exceptions.
- In **modern functional languages** (Haskell, Rust, OCaml...), it's exactly the idea behind `Option`/`Maybe` and `Result`/`Either` types: a function that can fail doesn't return "the result or an exception," it returns "a box that contains either the valid result, or an explicit marker that there isn't one" — and the language's type system forces you to open the box and look before using the contents.

Our `'VARLIBRE` is that same idea, implemented by hand because MyLISP has no type system to give it to us for free: a **sentinel value** representing "there's no number here, there's a domain failure," which every function that receives it must check explicitly before continuing to operate on it. The difference from `Option`/`Result` is only a matter of machinery — there, the compiler forces you to check it; here, the discipline of checking it is our own responsibility, clause by clause of the `COND`. It's a good example of how a real platform limitation (no exceptions) forces us, out of necessity, to rediscover by hand a pattern that comes built in in other languages.

11.3.3 The Code

```
(DEFUN LIBRE? (X) (EQ X 'VARLIBRE))

(DEFUN POTENCIA (B N)
  (COND ((= N 0) 1)
        (T (* B (POTENCIA B (- N 1))))))

(DEFUN EVALSUMA (E)
  (LET ((A (EVALEXPR (ADDEND E))))
    (COND ((LIBRE? A) A)
          (T (LET ((B (EVALEXPR (AUGEND E))))
               (COND ((LIBRE? B) B)
                     (T (+ A B)))))))))

(DEFUN EVALPROD (E)
  (LET ((A (EVALEXPR (FACTOR1 E))))
    (COND ((LIBRE? A) A)
          (T (LET ((B (EVALEXPR (FACTOR2 E))))
               (COND ((LIBRE? B) B)
                     (T (* A B)))))))))

(DEFUN EVALPOW (E)
  (LET ((B (EVALEXPR (BASE E))))
    (COND ((LIBRE? B) B)
          (T (LET ((N (EVALEXPR (EXPON E))))
               (COND ((LIBRE? N) N)
                     (T (POTENCIA B N)))))))))

(DEFUN EVALEXPR (E)
  (COND ((ISCONST? E) E)
        ((ISVAR? E)
         (PROGN (PRINT "CAS: variable libre, no evaluable:")
                (PRINT E)
                'VARLIBRE))
        ((ISSUM? E) (EVALSUMA E))
        ((ISPROD? E) (EVALPROD E))
        ((ISPOW? E) (EVALPOW E))))
```

Notice that `EVALEXPR` is split into four functions (`EVALSUMA`, `EVALPROD`, `EVALPOW`, and `EVALEXPR` itself) instead of nesting the three `COND`+`LET` directly inside a single `DEFUN`. This isn't just style — while developing this chapter, the version with everything nested in one function caused an **address error** (a CPU-level 68000 fault, not a controlled interpreter error) when loaded on the QL/emulator. It was fixed by repairing MyLISP's parser in the interpreter project, but the practical lesson stands: **expressions with many levels of nesting inside a single `DEFUN` are a real point of risk on this kind of platform**, not just a matter of style or readability. Splitting into smaller helper functions reduces that risk in addition to improving readability.

Three details worth understanding well, because they're the meatiest part of the whole chapter:

1. **`'VARLIBRE` fits exactly into 8 characters**, with no truncation — and its name already explains the failure without

needing an extra message. `LIBRE?` is the predicate that recognizes it, deliberately without the `IS` prefix used by `ISSUM?` / `ISPROD?` /etc., because it isn't asking about an AST node type but about a control value — a subtle distinction worth its own note if any reader notices it.

2. **No real arithmetic operator (`+`, `*`, `POTENCIA`) is ever executed on the sentinel.** Every recursive branch first checks whether the result of evaluating the child is `'VARLIBRE`, and if it is, returns it immediately **without** ever calling `+` / `*` / `POTENCIA`. If we didn't do this check, `(+ 'VARLIBRE 3)` would pass a symbol to MyLISP's real sum, and we'd fall right back into the problem we wanted to avoid: a low-level interpreter error, outside the CAS's control.
3. **Propagation is manual, level by level of the recursion — nothing happens automatically.** This is the most honest language-design lesson in the chapter: in a Lisp with no exceptions, "an error at the leaves is noticed at the root" doesn't happen by itself; **every** intermediate function has to cooperate explicitly by checking for and forwarding the sentinel upward. It's the price of the interpreter's simplicity, and it's exactly the same work that in Rust the `?` operator does automatically over a `Result` — here we write it by hand with `LET` + `COND` in every clause.

Platform note: `POTENCIA` uses simple recursion with a non-negative integer exponent. In MyLISP on the QL (Pro Pascal), the `{$A-}`/no-recursion restriction that constrains the CPC project in Turbo Pascal 3.0 CP/M-80 doesn't apply, so this recursion doesn't have that trap — but it's noted as a known limitation that it doesn't yet cover negative or fractional exponents.

11.3.4 The `VARLIBRE?` Incident and the Name-Audit Rule

When the control predicate was first written, it was called `VARLIBRE?` — **9 characters**. Truncated to 8 it becomes `VARLIBRE`, which is exactly the same symbol we use as the sentinel value (`'VARLIBRE`). The predicate collided with its own subject: defining `VARLIBRE?` silently overwrote the data symbol `VARLIBRE` with a function, breaking the `(EQ X 'VARLIBRE)` comparison in a way that's very hard to diagnose at a glance.

The lesson for the rest of the manual: **before settling on the final name of any new function, you have to count characters including any suffix (`?`, `1`, `2`…) and check the result against every name already defined**, not just "it looks short." It's easy to get comfortable with words that are already long (`VARLIBRE`) and then add a suffix without recounting. From here on, every chapter closes with a name-audit table before being considered verified.

11.3.5 Chaining SUBST and EVALEXPR

```
(DEFINE E1 (MAKESUM 'X 3))

(SUBST E1 'X 5)              ; -> (+ 5 3)          (pure substitution)
(EVALEXPR (SUBST E1 'X 5))   ; -> 8                (numeric evaluation)
```

With more than one variable, they're substituted one after another:

```
(DEFINE E2 (MAKEPROD (MAKESUM 'X 1) 'Y))

(SUBST E2 'X 2)                          ; -> (* (+ 2 1) Y)   ; Y is still free
(SUBST (SUBST E2 'X 2) 'Y 10)             ; -> (* (+ 2 1) 10)
(EVALEXPR (SUBST (SUBST E2 'X 2) 'Y 10))  ; -> 30
```

And if we try to evaluate numerically without having substituted every variable, the CAS no longer asks MyLISP's global environment anything — it detects the domain failure itself:

```
(EVALEXPR (SUBST E2 'X 2))
; CAS: variable libre, no evaluable:
; Y
; -> VARLIBRE
```

The result is the symbol `VARLIBRE`, not a number — and any function that uses `EVALEXPR` as a piece of something larger can check it with `LIBRE?` before trusting the result, exactly the way you check a `Result` / `Option` before "unwrapping" it.

11.3.6 Trying It with Powers

```
(DEFINE E3 (MAKEPOW 'X 2))
(SUBST E3 'X 4)                ; -> (POT 4 2)
(EVALEXPR (SUBST E3 'X 4))     ; -> 16
```

With the recursive version of `EVALEXPR`, `POT` is no longer a special case or a limitation: it's just another `COND` clause, treated exactly like `+` and `*`. The expression `(POT 4 2)` is never handed to the host's `EVAL` — `EVALEXPR` recognizes it itself and calls `POTENCIA`.

11.4 To Verify

- ☐ Test `(SUBST '(+ X 3) 'X 5)` and confirm it gives `(+ 5 3)` **without reducing** — `SUBST` substitutes, it doesn't evaluate.
- ☐ Chain `EVALEXPR` on that same result: `(EVALEXPR (SUBST '(+ X 3) 'X 5))` should give `8`.
- ☐ Test the nested free-variable case, `(EVALEXPR (SUBST E2 'X 2))` with `E2 = (MAKEPROD (MAKESUM 'X 1) 'Y)` (defined above), and confirm the CAS detects that `Y` is still free and returns `VARLIBRE` with its message — without any low-level interpreter type error propagating.

11.5 Chapter 11 Name-Audit Table

Name	Length	Collides?
`SUBST`	5	no
`LIBRE?`	6	no
`POTENCIA`	8	exact, no margin
`EVALSUMA`	8	exact, no margin
`EVALPROD`	8	exact, no margin
`EVALPOW`	7	no
`EVALEXPR`	8	exact, no margin

Four names sit exactly at the 8-character limit with no margin (`POTENCIA`, `EVALSUMA`, `EVALPROD`, `EVALEXPR`). They don't collide with each other or with anything from Chapter 10, but any future suffix on these names (for example, a hypothetical `EVALEXPR2`) must be reviewed with the same audit before being considered safe.

11.5.1 A Note That Will Be Fulfilled Later

`SUBST` has no arithmetic logic of its own — when it rebuilds the tree after substituting, it delegates entirely to `MAKESUM` / `MAKEPROD` / `MAKEPOW`. Right now those constructors are the "dumb" ones from Chapter 10, so `(SUBST E1 'X 5)` gives `(+ 5 3)` without reducing, exactly as shown above. But in Chapter 13 we're going to **redefine those same constructors in the same place** so that they automatically combine constants — and since `SUBST` doesn't know or care which version of `MAKESUM` is in effect, it will inherit that change without anyone touching a line of it. From then on, this same example will give `8` directly, not `(+ 5 3)`. It won't be a bug or a regression: it will be this chapter's abstraction barrier working exactly as designed. If you reach this point after having done Chapter 13, and `SUBST` gives you already-combined results, that's exactly what's happening.

11.6 Summary of What We Built in This Chapter

- **`SUBST`**: recursive substitution of a variable for a value, following the classify/descend/rebuild pattern that will repeat throughout the CAS.
- **`EVALEXPR`** (+ helpers `EVALSUMA` / `EVALPROD` / `EVALPOW` and `POTENCIA`): self-contained numeric evaluation, without delegating the tree's structure to the host interpreter's `EVAL`.
- **`LIBRE?`** and the sentinel symbol `'VARLIBRE`: the "value that means error" technique for a Lisp with no exceptions, with explicit manual propagation at every level of the recursion.
- **Two platform lessons** that stand as precedent for the rest of the manual: watch out for non-ASCII characters when copy-pasting code, and watch out for nesting too many levels inside a single `DEFUN` (the address error).

11.7 Next Chapter

Chapter 12 — "Raw" Symbolic Differentiation (`DERIV`): the first function that will generate expressions with obvious redundancies, motivating why Chapter 13 introduces a simplifier.

Continues in Chapter 12: "Raw" Symbolic Differentiation (DERIV)

Chapter 12 — "Raw" Symbolic Differentiation (DERIV)

> **In one sentence:** we're going to mechanically encode the high-school rules of differentiation, without trying to make the result pretty — precisely so we can *see*, with our own eyes on the REPL screen, the problem Chapter 13 is going to solve.

12.1 Why Now, and Not Before

Up to now, `SUBST` and `EVALEXPR` have walked the tree without **aggressively creating** new structure — at most they rebuilt a node just as "big" as the original. `DERIV` is the first function that **generates** trees: each differentiation rule doesn't substitute, it combines — it takes two already-differentiated subexpressions and joins them with `+`, `*`, nested derivatives... The size of the resulting tree can grow without bound, and seeing it happen on the REPL screen is the best possible motivation for the chapter that follows. This isn't a whim of the book's organization: the need to simplify is only understood once you've suffered the explosion.

12.2 The Theory: Differentiating Is Applying Rules Mechanically

The derivative of an algebraic expression with respect to a variable is computed with a small, fixed set of rules — the same ones anyone learns in high school, except here we write them as code instead of applying them by hand:

- **Constant:** the derivative of a number is always `0`. It doesn't matter what the number is, it doesn't depend on the variable.
- **Variable:** if it's exactly the variable we're differentiating with respect to, the derivative is `1`. If it's any other variable (a symbolic constant, in practice), the derivative is `0`.
- **Sum — sum rule:** `d/dx(u + v) = du/dx + dv/dx`. Each addend is differentiated separately and the results are added.
- **Product — product rule:** `d/dx(u · v) = (du/dx · v) + (u · dv/dx)`. This is the first rule that **isn't** "differentiate each child separately and combine the same as before" — it mixes the derivatives with the original, undifferentiated expressions, which is what triggers the tree size explosion.
- **Power — power rule (simple case, constant exponent):** `d/dx(uⁿ) = n · u^(n-1) · du/dx`. Here we assume, for

this first "raw" version, that the exponent is a constant and doesn't depend on `x` — differentiating variable exponents (`x^x`) is a much more advanced case that we deliberately leave out.

Notice a pattern that repeats in the product rule and the power rule: the derivative of a compound node needs **both** the original subexpression **and** its derivative. This is different from `SUBST`, where only the recursive result was needed. It's the first time the Chapter 11 "traversal pattern" isn't enough as is — it has to be adapted.

12.3 Tracing It by Hand Before Coding It

Let's differentiate `x · x` with respect to `x`, applying the product rule literally, without simplifying anything:

```
d/dx(x . x) = (d/dx(x) . x) + (x . d/dx(x))
            = (1 . x) + (x . 1)
```

The correct mathematical result is `2x`, but the mechanical rule gives us `(1 . x) + (x . 1)` — a valid expression, but completely redundant. Nobody would write this by hand. We're going to let `DERIV` generate it exactly as is, without shame, because **that's exactly the pedagogical point**: seeing the ugliness with your own eyes is what will justify Chapter 13.

12.4 The Code

```
(DEFUN DSUMA (E VBL)
  (MAKESUM (DERIV (ADDEND E) VBL) (DERIV (AUGEND E) VBL)))

(DEFUN DPROD (E VBL)
  (MAKESUM (MAKEPROD (DERIV (FACTOR1 E) VBL) (FACTOR2 E))
           (MAKEPROD (FACTOR1 E) (DERIV (FACTOR2 E) VBL))))

(DEFUN DPOW (E VBL)
  (MAKEPROD (MAKEPROD (EXPON E) (MAKEPOW (BASE E) (- (EXPON E) 1)))
            (DERIV (BASE E) VBL)))

(DEFUN DERIV (E VBL)
  (COND ((ISCONST? E) 0)
        ((ISVAR? E) (COND ((EQ E VBL) 1) (T 0)))
        ((ISSUM? E) (DSUMA E VBL))
        ((ISPROD? E) (DPROD E VBL))
        ((ISPOW? E) (DPOW E VBL))))
```

`DERIV` is split into three helper functions (`DSUMA`, `DPROD`, `DPOW`) instead of having the three rules nested inside a single `DEFUN`, just as we did with `EVALEXPR` in Chapter 11 — but here the reason is different and deserves its own explanation.

12.4.1 A Second Platform Limit: the `LOAD` Buffer

The "everything in one `DEFUN`" version of `DERIV` didn't produce a lexical error or an address error — it produced an error when loaded with `LOAD` from a file. The reason: `DoLoad` accumulates **every continuation line of a single definition** into one buffer before parsing it, and that buffer has a **500-character** limit. The monolithic definition of `DERIV`, counting all the indentation needed to make it readable, took up **547 characters** — over the limit.

This is a different limit from the "80 characters per line" one we already knew about (that's the limit of QDOS's *interactive keyboard buffer*, not of `LOAD` reading a file). Here the limit isn't per line, but per **complete accumulated definition**. The lesson for the manual: on MyLISP/QL you have to watch **two different size limits, for two different reasons**:

1. When typing interactively at the REPL: a maximum of ~80 characters accumulated by QDOS's keyboard buffer.
2. When loading from a file with `LOAD`: a maximum of 500 characters per complete definition (all its continuation lines added together), accumulated in `DoLoad`'s buffer.

Splitting a large function into smaller pieces isn't just good style here — it's the technique that solves **both** limits at once, in addition to the deep-nesting risk we already saw with Chapter 11's address error. Three different reasons, one same solution: small, helper functions instead of a single giant function.

An important note about the power rule, inside `DPOW`: `(- (EXPON E) 1)` is computed with MyLISP's **real subtraction**, not symbolically — because in this first version we assume the exponent is already a numeric constant (`2`, `3`...), never a subexpression. If in the future we wanted symbolic exponents, this line would have to change completely; for now it's a conscious limitation, not an oversight, and it's worth saying so explicitly in the manual.

12.5 Trying It — and Seeing the Explosion with Your Own Eyes

```
(DERIV '(+ X 3) 'X)              ; -> (+ 1 0)
(DERIV (MAKEPROD 'X 'X) 'X)      ; -> (+ (* 1 X) (* X 1))
(DERIV (MAKEPOW 'X 2) 'X)        ; -> (* (* 2 (POT X 1)) 1)
```

None of these three results is mathematically wrong — but none is what a human would write either. `(+ 1 0)` should be `1`. `(+ (* 1 X) (* X 1))` should be `2X`. `(* (* 2 (POT X 1)) 1)` should be `2X`. The bigger the starting expression, the worse it gets: try differentiating twice in a row (`(DERIV (DERIV E 'X) 'X)`) and you'll see how the "garbage" piles up on top of previous garbage, with no apparent limit.

This is the real, not made-up, problem that Chapter 13 will solve: a simplifier that, applied to these very results, reduces them to the form we'd expect to see written on paper.

12.6 To Verify

- ☐ Test `(DERIV '(+ X 3) 'X)` and confirm `(+ 1 0)`.
- ☐ Test `(DERIV (MAKEPROD 'X 'X) 'X)` and confirm `(+ (* 1 X) (* X 1))`.
- ☐ Test `(DERIV (MAKEPOW 'X 2) 'X)` and confirm `(* (* 2 (POT X 1)) 1)`.
- ☐ Test a second derivative (`DERIV` on the result of `DERIV`) and observe the tree's growth.

12.7 Chapter 12 Name-Audit Table

Name	Length	Collides?
`DSUMA`	5	no
`DPROD`	5	no
`DPOW`	4	no
`DERIV`	5	no

Deliberately short names: `DERIVSUM` / `DERIVPROD` / `DERIVPOW` would have been the more "obvious" choice by parallel with `EVALSUMA` / `EVALPROD` / `EVALPOW` from Chapter 11, but `DERIVPROD` is 9 characters — over the limit. The short prefix `D` is used instead of the full `DERIV` for the helpers, leaving plenty of margin on all four names.

12.8 Summary of What We Built in This Chapter

- **`DERIV`** (+ helpers `DSUMA` / `DPROD` / `DPOW`): mechanical symbolic differentiation, deliberately without simplifying.
- **A second documented platform limit**: the `LOAD` buffer accumulates 500 characters per complete definition, distinct from the 80-character-per-line limit of the interactive keyboard.
- **The real problem motivating the rest of the book**: correct but unreadable expressions, which get worse with each successive derivative.

12.9 Next Chapter

Chapter 13 — Simplification via "Smart Constructors": solves exactly the problem we just generated here, but not with a new function that walks the tree afterward — by redefining `MAKESUM`, `MAKEPROD`, and `MAKEPOW` from Chapter 10 so they apply identities like `x+0 = x`, `x·1 = x`, `x·0 = 0` at the very moment each node is built. The reader will see how `DERIV`'s "ugly" output becomes readable **without changing a single line of `DERIV`**, thanks to the abstraction barrier we set up in Chapter 10.

Continues in Chapter 13: Simplification via "Smart Constructors"

Chapter 13 — Simplification via "Smart Constructors"

> **In one sentence:** instead of cleaning up the tree after building it, we intercept construction itself — redefining `MAKESUM`, `MAKEPROD`, and `MAKEPOW` from Chapter 10 so they apply algebraic identities (identity element, absorbing element, constant evaluation) before creating any node. The result improves `DERIV`'s output without touching a single line of `DERIV`.

13.1 From Chapter 12's Wall to the Solution

The `DERIV` code from the previous chapter is correct and doesn't need touching: isolating each rule into `DSUMA`, `DPROD`, and `DPOW` was already the right call for keeping it readable. And answering the question anyone asks themselves after finishing Chapter 12 — "should I make `DERIV` smarter?" — the answer is no, not at all. Any attempt to make `DERIV` "know" how to simplify while differentiating would ruin that chapter's pedagogical purpose: the reader had to crash into the wall of combinatorial explosion to understand, with their own eyes, the sentence that sums up the whole problem:

> Symbolic computation without simplification is computationally useless, because it exhausts memory and the user's patience.

A small example is enough to see it: when differentiating `x²` (represented internally as `x · x`), the product rule generated `(1 · X) + (X · 1)`. If you apply another derivative to that result, the tree fills up with zeros and ones scattered everywhere, none of which contributes any real mathematical information. That's the real, not made-up, problem we're solving now.

13.2 The Idea: Simplify at Construction Time, Not Afterward

There are two ways to attack this problem. The first — the more intuitive one at first glance — would be to write a `SIMP` function that walks an already-built tree and cleans it up afterward, with its own predicates, selectors, and rewrite rules. The second, more elegant and with more tradition in Lisp (it's exactly the technique SICP uses for `make-sum`/`make-product`), is to never let the "dirty" tree come into existence in the first place: intercept the construction in the constructor itself.

13.3 The Math Before the Lisp

Before writing a single line of code, we need to lay out clearly **which** algebraic identity justifies each rule and **in what order** they must be checked — because the order isn't a stylistic detail, it's part of the specification: if checked in the wrong order, some rule would never fire. None of these rules depend on Lisp or MyLISP; they're properties of addition and multiplication you already know from high-school math, here we're simply giving them a technical name and ordering them precisely:

For `A1 + A2`:

Priority	Condition	Result	Name of the property
0	`A1` is `0`	`A2`	additive identity (`0 + a = a`)
1	`A2` is `0`	`A1`	additive identity (`a + 0 = a`)
2	`A1` and `A2` are both numbers	their actual sum	constant folding (if we already know both values, there's no reason to leave the sum undone)
3	any other case	`(+ A1 A2)`	no applicable identity, has to be represented symbolically

For `A1 · A2`:

Priority	Condition	Result	Name of the property
0	`A1` is `0` or `A2` is `0`	`0`	absorbing element of the product (`0 · a = 0`, regardless of `a`)
1	`A1` is `1`	`A2`	multiplicative identity (`1 · a = a`)
2	`A2` is `1`	`A1`	multiplicative identity (`a · 1 = a`)
3	`A1` and `A2` are both numbers	their actual product	constant folding
4	any other case	`(* A1 A2)`	no applicable identity

For B^E:

Priority	Condition	Result	Name of the property
0	`E` is `0`	`1`	any base raised to `0` is `1` (by convention, even if the base is `0` — we don't get into that edge case here)
1	`E` is `1`	`B`	raising to the first power changes nothing
2	`B` is `1`	`1`	`1` raised to any exponent is still `1`
3	`B` and `E` are both numbers	the actual computed power	constant folding
4	any other case	`(POT B E)`	no applicable identity

Notice one detail that distinguishes sum/product from power: in sum and product, **the identity element is checked before constant folding**, whereas in power the order between "exponent 0/1" and "both numbers" has less practical impact because the cases don't overlap in the same way. Even so, we always follow the general rule: **first the trivial cases that depend on a single argument, then constant folding, and finally the generic case** — it's the same "most specific first" principle we already use when ordering the clauses of any `COND` throughout the whole CAS.

Once these three tables are clear **on paper**, translating them to Lisp is mechanical: each row becomes a clause of a `COND`, in the same priority order. The code adds no new idea — it only expresses it in executable form.

13.4 Replacing Chapter 10's Constructors

With the tables now defined, we're going to **replace** — not add alongside, replace — the three functions `MAKESUM`, `MAKEPROD`, and `MAKEPOW` we wrote in Chapter 10. There, they were deliberately "dumb": they just packaged their arguments into a list with `LIST`, without looking at their contents. From here on, those same three functions — same name, same signature, same place in the program — start checking the tables above before building anything. Any other function that already used them (`SUBST`, `DERIV`, `DSUMA`, `DPROD`, `DPOW`) keeps calling them exactly the same way and doesn't need to change a single line — it's Chapter 10's abstraction barrier proving its worth: the client code never knew how a node was built, so it isn't affected by the node now being built better.

13.5 Why They're Called `MAKESUM`, Not `SUM`

Before the code, a naming decision that deserves explaining here and not earlier: it might seem cleaner to call these functions `SUM`, `PROD`, `POW` — shorter, more direct. We don't, and the reason carries more weight now than it did in Chapter 10.

The `MAKE` prefix is a deliberate signal: it says "this **builds** a tree node," not "this **computes** a result." In Chapter 10 that distinction was barely noticeable, because `MAKESUM` always built a list without exception — there was nothing to confuse. But from here on, `MAKESUM` starts behaving in two different ways depending on its arguments: `(MAKESUM 2 3)` returns the number `5`, while `(MAKESUM 'X 3)` returns the list `(+ X 3)`. If the function were simply called `SUM`, a new reader — or you yourself, rereading the code a few months from now — might reasonably expect it to **always** return a number, as the name "sum" suggests. The `MAKE` prefix is precisely what prevents that mistaken expectation: it reminds you, at every call site, that the function belongs to the family of the CAS representation's **constructors** (alongside `ADDEND`/`AUGEND`, `ISSUM?`...), not to the family of MyLISP's real arithmetic operators. That it sometimes "saves work" by returning the numeric result directly is an internal optimization of the constructor, not a change in what the function *is*.

It's a seemingly small distinction, but it's the same one that separates "a function that knows when it can take a shortcut" from "a function whose behavior silently depends on the type of its arguments without the name warning you." The cost of typing three more letters to call `MAKESUM` instead of `SUM` is, compared to that, insignificant.

13.6 The Code

```
(DEFUN CERO? (X) (AND (ISCONST? X) (= X 0)))
(DEFUN UNO?  (X) (AND (ISCONST? X) (= X 1)))

(DEFUN MAKESUM (A1 A2)
  (COND ((CERO? A1) A2)
        ((CERO? A2) A1)
        ((AND (ISCONST? A1) (ISCONST? A2)) (+ A1 A2))
        (T (LIST '+ A1 A2))))

(DEFUN MAKEPROD (A1 A2)
  (COND ((OR (CERO? A1) (CERO? A2)) 0)
        ((UNO? A1) A2)
        ((UNO? A2) A1)
        ((AND (ISCONST? A1) (ISCONST? A2)) (* A1 A2))
        (T (LIST '* A1 A2))))

(DEFUN MAKEPOW (B E)
  (COND ((CERO? E) 1)
        ((UNO? E) B)
        ((UNO? B) 1)
        ((AND (ISCONST? B) (ISCONST? E)) (POTENCIA B E))
        (T (LIST 'POT B E))))
```

A few notes on specific decisions:

- **`CERO?`/`UNO?` check `ISCONST?` before anything else.** This is essential: if `A1` is the list `(+ X Y)` or the symbol `X`,

calling `(= A1 0)` directly would fail with an interpreter type error — exactly the same "don't call a real operator on something that isn't a number" problem we already saw with the `VARLIBRE` sentinel in Chapter 11. The type check always comes first.

- **`=` is used, not `EQ` / `EQUAL`, to compare against zero/one.** Recall MyLISP's design: `=` compares *numeric value*, while `EQUAL` distinguishes exact types (`(EQUAL 0 0.0)` would be `NIL`). For the simplifier we care about the value, not the internal representation — `0`, `0/1`, or `0.0` should all be treated as "zero" for algebraic purposes.
- **`MAKEPOW` reuses `POTENCIA`**, the same function we already defined in Chapter 11 for `EVALPOW`. There's no need to duplicate the exponentiation logic — it's a numeric utility function, not specific to evaluation, so it makes sense to share it.
- **The clause order follows the tables above exactly: the trivial single-argument cases (`CERO?` / `UNO?`) come before constant folding.** This matters in one specific edge case: `MAKESUM(0, 0)`. With this order, `CERO? A1` is satisfied first and returns `A2` (which is also `0`) without ever executing the real sum — the final result is the same number, but the path taken is different, and in a system with more complex rules that order can indeed change the result. That's why the mathematical tables in the previous section aren't just documentation — they're the exact specification the code must follow to the letter.

13.7 The Retroactive Effect on Chapter 12

Without changing a single line of `DERIV`, `DSUMA`, `DPROD`, or `DPOW`, the same three examples from the previous chapter improve like this:

Expression	Chapter 12 ("dumb" constructors)	Chapter 13 (smart constructors)
`d/dx(x + 3)`	`(+ 1 0)`	`1`
`d/dx(x · x)`	`(+ (* 1 X) (* X 1))`	`(+ X X)`
`d/dx(x²)`	`(* (* 2 (POT X 1)) 1)`	`(* 2 X)`

The first and third cases end up perfectly clean — `1` and `2X` are exactly what a human would write by hand. The second case, `(+ X X)`, improves a lot (there are no more redundant ones or products) but **doesn't** collapse to `2X`. This isn't a shortcoming of the chapter — it's the exact boundary of what a smart constructor can solve on its own: `x + 0` and `x · 1` are identities that depend only on **one** of the two arguments, while "combining like terms" (`x + x = 2x`) requires **comparing the two operands against each other** and recognizing that they're the same subexpression. That's a problem of a different nature — it requires structural equality between subexpressions and a notion of canonical order — and it's exactly what we'll tackle in the next chapter.

13.8 A Rule That Looks Obvious but Is Mathematically False: `0^x`

It's tempting to also add "if the base is `0`, the result is `0`," symmetric to the `UNO?` rule we just added. **We don't, and the reason matters for the manual**: unlike `1^x = 1`, which is true for *every* real `x` with no exception, `0^x = 0` is **only** true when `x` is strictly positive:

- `0⁰` equals, by convention, `1` in most algebraic contexts — not `0`.
- `0^(-2) = 1/0²` — division by zero, **undefined**, not `0`.
- `0³ = 0` — here yes, but only because the exponent is positive.

When `E` is a symbolic variable in `(MAKEPOW 0 X)`, we don't know whether `X` is positive, zero, or negative — so we can't guarantee the identity holds. Unconditionally simplifying to `0` would produce a **mathematically incorrect** result for two of the three possible cases of `X`, and it would do so silently: the CAS wouldn't be "a little" wrong, it would return a specific, incorrect number with the same confidence as if it were correct. That's much worse than not simplifying at all.

That's why, as `MAKEPOW` stands now, `(MAKEPOW 0 'X)` leaves the result as `(POT 0 X)`, unevaluated — and that's the **correct** behavior, not an outstanding limitation to fix. The general lesson for the whole CAS, not just this case: **an algebraic identity can only be coded as an unconditional rule if it's true for *all* possible values of the variable** — it's not enough for it to be true in the typical or expected case. When certainty depends on the sign, the magnitude, or some other assumption about the variable that the CAS can't verify, the right thing to do is not simplify. Adding that ability to reason under assumptions (`assume(x > 0)`, for instance) is a real feature of advanced CAS software, and it's outside the scope of this manual.

13.9 Trying It

```
(MAKESUM 2 3)                ; -> 5   (constant folding)
(MAKESUM 'X 0)                ; -> X
(MAKESUM 0 'X)                ; -> X
(MAKEPROD 'X 0)                ; -> 0
(MAKEPROD 'X 1)                ; -> X
(MAKEPOW 'X 0)                ; -> 1
(MAKEPOW 'X 1)                ; -> X
(MAKEPOW 1 'X)                ; -> 1   (new rule: base 1)
(MAKEPOW 0 'X)                ; -> (POT 0 X)   (NOT simplified, correct)
(MAKEPOW 2 3)                ; -> 8   (constant folding)

(DERIV '(+ X 3) 'X)             ; -> 1
(DERIV (MAKEPROD 'X 'X) 'X)     ; -> (+ X X)
(DERIV (MAKEPOW 'X 2) 'X)       ; -> (* 2 X)
```

13.10 To Verify

- ☐ Test the eight standalone cases of `MAKESUM` / `MAKEPROD` / `MAKEPOW`, plus the two new ones: `(MAKEPOW 1 'X)` → `1` and `(MAKEPOW 0 'X)` → `(POT 0 X)` **not simplified** (correct behavior).
- ☐ Re-test the three `DERIV` examples from Chapter 12 and confirm the improvement in the table above.
- ☐ Confirm that `(DERIV (MAKEPROD 'X 'X) 'X)` gives `(+ X X)` and **doesn't** collapse to `(* 2 X)` — this is the expected behavior at this point in the book, not a bug.

13.11 Chapter 13 Name-Audit Table

Name	Length	Collides?
`CERO?`	5	no
`UNO?`	4	no

`MAKESUM`, `MAKEPROD`, and `MAKEPOW` were already audited in Chapter 10 — here they're redefined, not renamed, so there's no need to check them again.

13.12 Summary of What We Built in This Chapter

- **`CERO?` / `UNO?`**: helper predicates for detecting the identity elements of sum and product, checking type before comparing values.
- **`MAKESUM`, `MAKEPROD`, `MAKEPOW` (smart version)**: same names, same signatures as in Chapter 10, now applying the algebraic-identity tables before building any node.
- **A case rejected on purpose** (`0^X → 0` for a symbolic exponent): the lesson that an identity can only be coded unconditionally if it's true for *all* possible values of the variable.
- **Practical confirmation of Chapter 10's abstraction barrier**: `SUBST`, `DERIV`, and all their helpers improve their

output without a single line of theirs having changed.

13.13 Next Chapter

Chapter 14 — Canonical Order and Like Terms: solves the case left outside this chapter (`x + x → 2x`), through structural comparison between subexpressions and a notion of order that lets us detect when two terms are "the same" except for a numeric coefficient.

Continues in Chapter 14: Canonical Order and Like Terms

Chapter 14 — Canonical Order and Like Terms

> **In one sentence:** Chapter 13's smart constructors can't solve `x + x → 2x` because they only ever see two arguments at a time; here we build `SIMPSUM`, a function that temporarily flattens the chain of sums into an ordinary list, groups by like term, and rebuilds the tree — without ever changing the AST representation.

14.1 What a "Canonical Form" Is

Before writing code, we need to understand the concept that gives the chapter its name. Two expressions can be **mathematically equivalent** without being **structurally equal** as Lisp lists: `(+ X X)` and `(* 2 X)` represent the same value for any `X`, but they're different trees, and `EQUAL` would say they aren't equal. A **canonical form** is a fixed rule that says: "of all the possible ways of writing an equivalent expression, this is the only one we're going to consider correct" — so that, once everything passes through that rule, comparing two expressions for equivalence reduces to comparing their canonical forms with `EQUAL`. Without a canonical form, `X + X`, `2 · X`, and `X · 2` coexist as three different representations of the same mathematical fact, and the CAS has no way of knowing it. This chapter builds the rule that collapses all those variants into one.

14.2 The Exact Problem We Left Open

In Chapter 13 we saw that the smart constructors solve `x + 0`, `x · 1`, `x · 0`... but they don't solve `x + x → 2x`. The underlying reason, worth stating precisely before moving on: **`MAKESUM` only ever sees two arguments at a time**. When passed `X` and `X`, it has no way of knowing they're "the same term" except by comparing them — and even if it did, the real problem shows up with longer chains: `(+ (+ X Y) X)` has three real addends (`X`, `Y`, `X`) spread across two nested calls to `MAKESUM`, each seeing only one pair. No local simplification can see the full list of addends at once. This is a structural limit, not an oversight: **an operation that only looks at two arguments can't solve a problem that depends on comparing all of them against each other**.

14.3 Two Possible Paths — and Why We Choose the Third

There are two "obvious" ways to solve this:

- **Strict binary trees with rotation**: keep the usual binary representation and program a mechanism that reorganizes the tree (rotating it into a normalized form) so addends in different branches can be compared. It's viable, but it's the harder path: it entangles "find like terms" logic with "keep the tree shape" logic, when in reality these are two independent problems.
- **N-ary operators (flat lists) throughout the representation**: redesign `+` and `*` to accept any number of arguments starting from Chapter 10 (`(+ X Y X)` instead of nested binary trees). It's the "professional" solution — this is how Maxima or SymPy internally represent sums and products — but it has a real cost for this manual: it would force us to rewrite `ADDEND`/`AUGEND`, `SUBST`, `DERIV` (`DSUMA`/`DPROD`), and `EVALEXPR` (`EVALSUMA`/`EVALPROD`) — that is, **the four previous chapters, already verified on real hardware**. From the standpoint of building a production CAS, this would be the right path in the medium term; from the standpoint of a manual that teaches step by step, it's a leap that breaks all the continuity already built and adds a large representational complexity to solve a

local problem.

The path we follow here — **flatten, group, and rebuild** — never changes the AST's official representation. `ISSUM?`, `MAKESUM`, `ADDEND`, `AUGEND`, and everything already in `SUBST`, `DERIV`, and `EVALEXPR` stay exactly the same. What we do is write a *new*, self-contained function that:

1. Temporarily converts the tree of sums into an **ordinary Lisp list** (not a CAS structure, a normal list of the host language) where "all the addends at once" can indeed be seen.
2. Groups that list by like term.
3. Rebuilds a binary tree of sums again, using the same `MAKESUM`/`MAKEPROD` as always.

It's the same "convert to a more convenient working form, operate, rebuild" idea we already used without naming it in earlier chapters — here the convenient working form is a flat list instead of another CAS tree.

14.4 Tracing the Whole Pipeline by Hand, Before the Code

Before looking at the four functions, it's worth seeing **what the data looks like** at each boundary between one function and the next — because the real leap in this chapter isn't in the difficulty of each function on its own (they're all a `COND` with recursion, as in earlier chapters), but in the fact that each one speaks a different data "language":

```
(+ (+ X Y) X)                    <- CAS tree (nested S-expression)
      |
      |  FLATSUM
      ▼
(X Y X)                          <- ordinary, flat Lisp list
      |
      |  AGRUPA  (uses COEF/LITERAL term by term)
      ▼
((X . 2) (Y . 1))                <- association list (literal . coefficient)
      |
      |  REBUILD
      ▼
(+ (* 2 X) Y)                    <- CAS tree again
```

Four data shapes, three transitions. It's worth naming them explicitly so you don't get lost reading the code:

1. **Input and output are CAS trees** — the same `(+ ...)` lists we've been handling since Chapter 10.
2. **`FLATSUM` produces an ordinary Lisp list**, with no `+` involved at all: three loose symbols, `(X Y X)`. At this point we're no longer "inside" the CAS's representation — it's a normal list of the host language, which is why from here on `APPEND`, `CONS`, `CAR`/`CDR` can be used freely without violating any abstraction barrier: the barrier protects the CAS tree, not the internal working lists we ourselves create and discard.
3. **`AGRUPA` produces an association list**: pairs `(literal . coefficient)`, one per distinct term. `X` appeared twice in the flat list, so here it's already condensed into a single pair `(X . 2)`. This is the structure where the simplification really "happens" — the previous two steps only set the stage.
4. **`REBUILD` speaks the CAS's language again**: for each `(literal . coefficient)` pair it builds `(* coefficient literal)` with `MAKEPROD`, and chains them all together with `MAKESUM` — returning, once again, a legitimate S-expression from the CAS tree.

With this trace in mind, each function that follows is simply "how the corresponding arrow of the diagram above is produced in Lisp" — there's no need to hold all four in your head at once.

14.5 Step 1: Extracting the Coefficient and Literal Part of a Term

To be able to group `X`, `2X`, and `X·2` as "the same term with different coefficients," we need to break down each addend into a `(coefficient, literal part)` pair:

- `3` (pure constant) → coefficient `3`, no literal part.
- `X` (variable alone) → coefficient `1`, literal `X`.
- `(* 2 X)` **or** `(* X 2)` → coefficient `2`, literal `X` — the order of the factors shouldn't matter.
- `(* X Y)` (product with no constant at all) → coefficient `1`, literal the entire product, not decomposed any further.

```
(DEFUN COEF (E)
  (COND ((ISCONST? E) E)
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E)) (FACTOR1 E))
               ((ISCONST? (FACTOR2 E)) (FACTOR2 E))
               (T 1)))
        (T 1)))

(DEFUN LITERAL (E)
  (COND ((ISCONST? E) 1)
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E)) (FACTOR2 E))
               ((ISCONST? (FACTOR2 E)) (FACTOR1 E))
               (T E)))
        (T E)))
```

The trick for making `(* 2 X)` and `(* X 2)` give the same result is checking **`FACTOR1` and `FACTOR2` separately**, without assuming the constant always occupies the same position. And one deliberate detail: `LITERAL` of a pure constant returns `1`, not `NIL`. This lets a pure constant come back out intact when rebuilt with `(MAKEPROD COEF LITERAL)`, with no special case at all — `(MAKEPROD 3 1)` already knows how to simplify to `3` thanks to the `UNO?` rule from Chapter 13. Designing the constructors well back then is saving us work now.

Explicit limit of this extraction: it's single-level. It recognizes a constant multiplying *anything else*, but it doesn't reorder products of several variables (`(* X Y)` and `(* Y X)` will still be different literals as far as this function is concerned) nor decompose products with more than one nested constant factor. For this chapter's goal — collapsing sums like `X + X` or `2X + 3X` — it's enough; going further would require a canonical form *inside* products too, which is out of scope here.

14.6 Step 2: Flattening the Chain of Sums

```
(DEFUN FLATSUM (E)
  (COND ((ISSUM? E)
         (APPEND (FLATSUM (ADDEND E)) (FLATSUM (AUGEND E))))
        (T (LIST E))))
```

Every addend that isn't itself a sum gets wrapped in a one-element list (`(LIST E)`); every sum gets decomposed recursively and concatenated with `APPEND`. The result of `(FLATSUM '(+ (+ X Y) X))` is the list `(X Y X)` — three loose elements, with no `+` involved. Notice that `APPEND` and `LIST` are operations of the *host language* on ordinary lists, not `MAKESUM` or anything from the CAS: at this step we deliberately step outside the symbolic tree's domain.

This is exactly the first arrow of the trace above: the tree `(+ (+ X Y) X)` goes in, the flat list `(X Y X)` comes out.

14.7 Step 3: Grouping by Literal Part

We need to walk the flattened list and accumulate, for each distinct literal, the sum of its coefficients. Without `SETQ`, this is done by building a new association list at each step, in a purely functional way:

```
(DEFUN ASSOCADD (LIT CF ALIST)
  (COND ((NULL ALIST) (LIST (CONS LIT CF)))
        ((EQUAL (CAR (CAR ALIST)) LIT)
         (CONS (CONS LIT (+ (CDR (CAR ALIST)) CF)) (CDR ALIST)))
        (T (CONS (CAR ALIST) (ASSOCADD LIT CF (CDR ALIST))))))

(DEFUN AGRUPA (TERMS)
  (COND ((NULL TERMS) NIL)
        (T (ASSOCADD (LITERAL (CAR TERMS))
                     (COEF (CAR TERMS))
                     (AGRUPA (CDR TERMS))))))
```

`ASSOCADD` takes a literal, a coefficient, and an association list `((literal . accumulated-coefficient) ...)`, and returns a new list: if the literal already exists, it adds the coefficient to the existing entry; if not, it adds a new entry. The comparison is done with **`EQUAL`, not `EQ`** — because a literal can be a symbol (`X`) or an entire list (`(* X Y)`), and `EQ` only guarantees reliable identity for symbols.

`AGRUPA` walks the flattened list term by term, extracting `LITERAL`/`COEF` from each one and accumulating them with `ASSOCADD`. The result of `(AGRUPA '(X Y X))` is `((X . 2) (Y . 1))` — the second arrow of the trace: from the flat list to the association list.

14.8 Step 4: Rebuilding the Tree

```
(DEFUN REBUILD (ALIST)
  (COND ((NULL ALIST) 0)
        (T (MAKESUM (MAKEPROD (CDR (CAR ALIST)) (CAR (CAR ALIST)))
                    (REBUILD (CDR ALIST))))))

(DEFUN SIMPSUM (E) (REBUILD (AGRUPA (FLATSUM E))))
```

For each `(literal . coefficient)` entry, `(MAKEPROD coefficient literal)` is built — which already simplifies on its own, thanks to Chapter 13 — and they're all added together with cascading `MAKESUM`. The base case, `(NULL ALIST) → 0`, isn't a whim: it's the "identity element" of an empty sum, and here something worth explicitly pointing out happens, because it's a real instance of the layered architecture working: when the list has a single term, `REBUILD` ends up generating `(MAKESUM term 0)`, and it's Chapter 13's `CERO?` rule — written without this chapter in mind at all — that automatically removes that leftover `0`. We haven't had to code any special case for "the last term doesn't carry an extra `+ 0`": it was already solved by work done two chapters ago.

`SIMPSUM` chains the three steps together. It's the function the reader will use directly, and its only two input/output boundaries are CAS trees — the third arrow of the trace closes the loop, returning to the same "language" `FLATSUM` started with.

14.9 Trying It — Closing the Thread Left Open Since Chapter 12

```
(SIMPSUM '(+ (+ X Y) X))          ; -> (+ (* 2 X) Y)
(SIMPSUM '(+ X X))                ; -> (* 2 X)
```

And now the example we've been carrying since we first started differentiating: the derivative of `x · x` with respect to `x`, which in Chapter 13 got stuck at `(+ X X)` with no way to go further:

```
(SIMPSUM (DERIV (MAKEPROD 'X 'X) 'X))     ; -> (* 2 X)
```

Finally. Three chapters after posing the problem, `d/dx(x²) = 2x` comes out exactly as you'd write it by hand.

One more case, to confirm that the order of the factors doesn't matter and that already-existing coefficients combine correctly:

```
(SIMPSUM '(+ (+ (* 2 X) (* 3 X)) Y))     ; -> (+ Y (* 5 X))
(SIMPSUM '(+ X (* X 2)))                  ; -> (* 3 X)
```

Notice in the first example that the result comes out as `(+ Y (* 5 X))` and not `(+ (* 5 X) Y)` — the order of the addends in the rebuilt sum depends on the order in which `REBUILD` processes the association list, not on the order they originally appeared in. This is mathematically correct (addition is commutative), but it's a real limitation of this first version: **there's no canonical order yet between different addends**, only between occurrences of the same literal. Fixing a stable order (alphabetical, for instance) so that two equivalent expressions *always* produce the same tree — not just the same value — is a natural refinement for later, and we leave it noted as such.

14.10 Three Ideas Worth Highlighting Before Closing the Chapter

14.10.1 1. Total Synergy with Chapter 13

The most important achievement of this chapter isn't in `FLATSUM` or `AGRUPA` — it's hidden in a single line of `REBUILD`:

```
(MAKESUM (MAKEPROD (CDR (CAR ALIST)) (CAR (CAR ALIST))) ...)
```

`REBUILD` **blindly trusts** that `MAKESUM` and `MAKEPROD` will clean up any trivial case that shows up. We already saw this remove the leftover `+ 0` from the list's last term. But the effect goes further: if, when grouping two like terms, their coefficients cancel out to **exactly zero** — for example, `X` and `-1·X` — the entire term disappears from the result, with `REBUILD` knowing nothing about cancellations or having any special case for it:

```
(SIMPSUM '(+ (+ X (* -1 X)) Y))     ; -> Y            (the X term vanishes)
(SIMPSUM '(+ X (* -1 X)))           ; -> 0            (total cancellation)
```

This happens because `MAKEPROD(0, X)` already knows to return `0` (the absorbing-element rule), and `MAKESUM` already knows to remove that `0` from the sum (the identity-element rule) — exactly the same two rules from Chapter 13, applying here without anyone having explicitly invoked them again. We haven't written a single `IF` to "clean zeros out of the final list." The layers of abstraction are doing the heavy lifting for us — which is the sign that the layered architecture, with abstraction barriers set up properly since Chapter 10, is paying dividends.

14.10.2 2. Pure Functional Recursion, No Mutable State

`ASSOCADD` and `AGRUPA` manage what in another language would be "accumulated state" (the growing association list) without using any mutable variable — because MyLISP has no `SETQ`. Updating an existing entry is done by rebuilding the entire list with the modified entry:

```
(CONS (CONS LIT (+ (CDR (CAR ALIST)) CF)) (CDR ALIST))
```

This isn't a limitation we're working around — it's the natural way to program in a Lisp with no destructive assignment, and it's exactly the same discipline we already applied to the `VARLIBRE` sentinel in Chapter 11: instead of mutating something in place, a new version is built and returned. It's Lisp at its most purely functional, and it shows up here without needing any extra design decision — it's simply the only option available.

14.10.3 3. A Note on Algorithmic Performance

`FLATSUM` uses `APPEND` inside the recursion. In most Lisp implementations — MyLISP included, because `APPEND` has to walk and rebuild its entire first argument to attach the second one — this gives this phase a worst-case complexity of **O(n²)**: a chain of sums heavily unbalanced to the left (`(+ (+ (+ (+ X Y) Z) W) ...)`) forces the same already-built sublists to be walked over and over. For the expressions of a didactic CAS like ours, with small trees, this cost is completely irrelevant in practice. But it's a real, known limitation of the "flatten with `APPEND`" technique, and it's worth saying so plainly for any reader with more programming experience who wonders why it hasn't been optimized: the answer is that, at this scale, optimizing it would be premature complexity — but a production CAS working on large expressions would need a linear-cost flattening strategy (accumulating in reverse order and reversing at the end, for example), not the one we use here.

14.11 To Verify

- ☐ Test `(SIMPSUM '(+ (+ X Y) X))` and confirm `(+ (* 2 X) Y)` (the exact order of the addends may vary, see the note above).
- ☐ Test `(SIMPSUM '(+ X X))` and confirm `(* 2 X)`.
- ☐ Test `(SIMPSUM (DERIV (MAKEPROD 'X 'X) 'X))` and confirm `(* 2 X)` — closing the thread from Chapter 12.
- ☐ Test `COEF` / `LITERAL` standalone on `(MAKEPROD 2 'X)` and `(MAKEPROD 'X 2)` and confirm they give the same pair in both cases.
- ☐ Test `(SIMPSUM '(+ (+ X (* -1 X)) Y))` and confirm it gives `Y` (the `X` term cancels out and disappears entirely).
- ☐ Test `(SIMPSUM '(+ X (* -1 X)))` on its own and confirm `0`.

14.12 Chapter 14 Name-Audit Table

Name	Length	Collides?
`COEF`	4	no
`LITERAL`	7	no
`FLATSUM`	7	no
`ASSOCADD`	8	exact, no margin
`AGRUPA`	6	no
`REBUILD`	7	no
`SIMPSUM`	7	no

One name at the exact limit (`ASSOCADD`), with no collision against anything from earlier chapters.

14.13 Summary of What We Built in This Chapter

- **`COEF` / `LITERAL`**: decomposition of a term into coefficient and literal part, insensitive to the order of factors in a product.
- **`FLATSUM`**: flattens a chain of nested sums into an ordinary Lisp list.
- **`ASSOCADD` / `AGRUPA`**: pure functional accumulation (no `SETQ`) of coefficients by literal, using `EQUAL` to compare compound literals.
- **`REBUILD` / `SIMPSUM`**: tree reconstruction relying entirely on Chapter 13's `MAKESUM` / `MAKEPROD`, which solves both the leftover `+0` and total term cancellation for free.
- **Closing the thread left open since Chapter 12**: `d/dx(x²) = 2x` now comes out exactly as you'd write it by hand.
- **A limit documented with a concrete example**: `SIMPSUM` doesn't reach sums nested inside products or powers —

the starting point for Chapter 15.

14.14 Next Chapter

`SIMPSUM` solves the problem posed at the start of the chapter, but it has a limit worth noting with a concrete example before closing: it only flattens the chain of sums hanging **directly** off each other through `+`. If a sum is nested inside a product or a power, `SIMPSUM` never gets to see it, because `FLATSUM` only descends while the root node is `ISSUM?`:

```
(SIMPSUM '(* 2 (+ X X)))   ; -> (* 2 (+ X X))   -- does NOT simplify the inside
```

This matters in practice: expressions that come out of differentiating compound bases (for example, the derivative of `(X+Y)²`) can produce sums tucked inside products, and `SIMPSUM` would leave them untouched. Chapter 15 builds `SIMP`: a simplifier that walks **the entire** tree recursively — with the same classify/descend/rebuild pattern as always — applying `SIMPSUM` to every sum node it finds, no matter how deeply nested, and only then will it make sense to build a top-level function that chains `SUBST`, `DERIV`, and `SIMP` for the CAS's end user to use directly.

Also noted for later, with no urgency: a stable canonical order among *different* addends (not just among repetitions of the same literal, which we already solve here), and extending the same flatten/group pattern to nested products and powers (`X · X → X²`, `(X²) · X → X³`), where the "literal" would become the base and the "coefficient" the exponent.

Continues in Chapter 15: Full-Tree Simplification (SIMP)

Chapter 15 — Full-Tree Simplification (SIMP)

> **In one sentence:** `SIMPSUM` only flattens the chain of sums that hang directly off each other; here we build `SIMP`, a function that walks the **entire** tree recursively and applies `SIMPSUM` at every sum node it finds, no matter how deeply nested.

15.1 The Exact Limit We Left Open

Closing Chapter 14 we saw a very specific case `SIMPSUM` doesn't solve:

```
(SIMPSUM '(* 2 (+ X X)))   ; -> (* 2 (+ X X))   -- the inside doesn't change
```

The reason is structural, not an oversight: `FLATSUM` only descends while the node it's examining is `ISSUM?`. As soon as the root node is a product, `FLATSUM` treats it as a single opaque term and never looks inside. `SIMPSUM` was designed to solve "a chain of sums," not "any expression that contains sums in any position" — and that distinction is exactly this chapter's problem.

15.2 The Theory: Simplify Bottom-Up

The solution isn't a new function, different from everything before it — it's the same traversal pattern we've been using since `SUBST` (classify the node, descend recursively into the children, rebuild), applied with a very specific ordering rule: **simplify the parts first, combine the whole afterward**. In technical terms, this is called a **postorder** traversal: to simplify a node, its children are fully simplified first, and only once both children are already in their simplest form is the current node combined.

This guarantees that, by the time we get around to examining a sum node, any sum hidden further down the tree — inside a product, inside a power, at any depth — has already been simplified before the current level sees it. The tree gets cleaned from the leaves toward the root, never the other way around.

15.3 The Code

```
(DEFUN SIMP (E)
  (COND ((ISCONST? E) E)
        ((ISVAR? E) E)
        ((ISSUM? E)
         (SIMPSUM (LIST '+ (SIMP (ADDEND E)) (SIMP (AUGEND E)))))
        ((ISPROD? E)
         (MAKEPROD (SIMP (FACTOR1 E)) (SIMP (FACTOR2 E))))
        ((ISPOW? E)
         (MAKEPOW (SIMP (BASE E)) (SIMP (EXPON E))))))
```

Three notes on specific decisions:

- **Constants and variables are returned as is** — they're the tree's leaves, there's nothing to simplify in them.
- **The sum case is the only one that calls `SIMPSUM`**, and it does so *after* recursively simplifying both children, not before. This is what lets a sum nested several levels down come out already collapsed by the time its parent — a product, say — gets it back.
- **The product and power cases don't call `SIMPSUM`** — they call `MAKEPROD`/`MAKEPOW`, the same smart constructors from Chapter 13. This is deliberate: `SIMPSUM` knows how to group terms of a *sum*; combining repeated factors inside a *product* (`X · X → X²`) is an analogous but distinct problem, which we deliberately leave outside this chapter.

15.4 Tracing It by Hand: `SIMP` on `(* 2 (+ X X))`

1. `E` is `(* 2 (+ X X))` → `ISPROD?` is true.
2. `FACTOR1` is simplified: `(SIMP 2)` → `2` (constant, no change).
3. `FACTOR2` is simplified: `(SIMP '(+ X X))` → here `ISSUM?` is true, so `(SIMPSUM (LIST '+ (SIMP 'X) (SIMP 'X)))` = `(SIMPSUM '(+ X X))` = **`(* 2 X)`** is called. The sum that was hidden inside the product has already come out resolved.
4. It's rebuilt with `MAKEPROD`: `(MAKEPROD 2 '(* 2 X))`. No `MAKEPROD` rule fires — it isn't zero, it isn't one, and `(* 2 X)` isn't a constant — so the result is `(* 2 (* 2 X))`.

15.5 A Victory and a New Limit, Both Honest

```
(SIMP '(* 2 (+ X X)))   ; -> (* 2 (* 2 X))
```

The victory: the inner sum **was** simplified — there's no more untouched `(+ X X)` inside the product, which was exactly Chapter 14's problem. The new limit: the mathematically correct result would be `4X`, but `(* 2 (* 2 X))` stays as a product of a product, without collapsing. `MAKEPROD` knows how to combine two numbers when it sees them directly as its two arguments, but here the second argument isn't a number — it's the expression `(* 2 X)`, which *contains* a number inside it. Detecting and combining nested constants inside products is the same kind of problem we solved for sums in Chapter 14 (flatten, group, rebuild), applied now to the product — and it's outstanding work, not a bug in this chapter. It's noted as a candidate for later.

15.6 Trying It — Closing a Tougher Case Than Chapter 14's

```
(SIMP '(* 2 (+ X X)))                          ; -> (* 2 (* 2 X))

; A power with a compound base: differentiate (X+Y)^2 with respect to X
(DEFINE E4 (MAKEPOW (MAKESUM 'X 'Y) 2))
(DERIV E4 'X)              ; -> (* 2 (+ X Y))   (already clean, no duplicate sums)
(SIMP (DERIV E4 'X))       ; -> (* 2 (+ X Y))   (SIMP changes nothing, it was already fine)

; The case that really demanded SIMP over SIMPSUM: a sum of two identical
; derivatives, each generating its own internal sum
(DEFINE E5 (MAKESUM (MAKEPROD 'X 'X) (MAKEPROD 'X 'X)))
(DERIV E5 'X)              ; -> (+ (+ X X) (+ X X))
(SIMP (DERIV E5 'X))       ; -> (* 4 X)
```

The last example is the one that really demonstrates `SIMP`'s value over `SIMPSUM` alone: the raw derivative has sums nested two levels deep, and `SIMP` collapses them all down to `(* 4 X)` in a single call, thanks to simplifying bottom-up before trying to combine anything at the top level.

15.7 To Verify

- ☐ Test `(SIMP '(* 2 (+ X X)))` and confirm `(* 2 (* 2 X))` — the inner sum simplified, the outer product not collapsed (expected behavior, not a bug).
- ☐ Test `(SIMP (DERIV E4 'X))` with `E4 = (X+Y)²` and confirm `(* 2 (+ X Y))`.
- ☐ Test `(SIMP (DERIV E5 'X))` with `E5 = X·X + X·X` and confirm `(* 4 X)` — the case that distinguishes `SIMP` from `SIMPSUM` alone.

15.8 Chapter 15 Name-Audit Table

Name	Length	Collides?
`SIMP`	4	no

A single new name, with no conflict against `SIMPSUM` or anything from earlier chapters.

15.9 Next Chapter

With `SIMP` verified, Chapter 16 builds the top-level orchestrator that chains `SUBST`, `DERIV`, and `SIMP` for the CAS's end user to use directly — the step we deliberately didn't take in this chapter, so as not to mix "build the complete simplifier" with "decide how it's exposed to the user." Also noted, with no urgency, is the limit discovered here: combining nested constants inside products (`2 · (2 · X) → 4X`), with the same flatten/group/rebuild technique we already used for sums, applied this time to the product.

Continues in Chapter 16: Product Simplifier (SIMPPROD)

Chapter 16 — Product Simplifier (SIMPPROD)

> **In one sentence:** we apply exactly the same technique from Chapter 14 (flatten, group, rebuild) to the product instead of the sum — swapping "coefficient that gets added" for "exponent that gets added," and "literal that gets compared" for "base that gets compared" — and with that we close the two gaps we left noted: `2·(2·X) → 4X` and `X·X → X²`.

16.1 The Two Specific Gaps We Left Open

At the end of Chapter 15 we saw that `SIMP` fixes sums nested inside a product, but leaves two related cases unsolved:

```
(SIMP '(* 2 (+ X X)))    ; -> (* 2 (* 2 X))   -- doesn't collapse to (* 4 X)
(SIMPPROD '(* X X))      ; -- this function doesn't even exist yet
```

The first is "nested constants inside a product not being combined." The second is "the same variable multiplied by itself isn't recognized as a power." Both are the same class of problem we solved for sums in Chapter 14: `MAKEPROD` only ever sees two arguments at a time, so it can't see "all the factors in a chain of products" in order to group them.

16.2 The Idea: The Same Pattern, With the Roles Swapped

You'll remember the pattern from Chapter 14: for sums, each term is decomposed into `(coefficient, literal)`, equal literals are grouped by adding coefficients, and the result is rebuilt. For products, the pattern is **structurally identical**, but with the roles swapped:

Chapter 14 (sum)	Chapter 16 (product)
decompose into `(coefficient, literal)`	decompose into `(base, exponent)`
group equal literals by **adding** coefficients	group equal bases by **adding** exponents
rebuild with `MAKEPROD(coef, literal)` and sum with `MAKESUM`	rebuild with `MAKEPOW(base, expon)` and multiply with `MAKEPROD`

This isn't a design coincidence — it's the same relationship that exists between sum and product in mathematics itself: adding the same quantity several times is multiplying it (`x + x = 2x`), and multiplying the same quantity several times is raising it to a power (`x · x = x²`). The "coefficient" of a sum is the "exponent" of a product — both count how many times the same thing repeats.

16.3 The Helper Constructors: `BASEOF` and `EXPOF`

Just as `COEF`/`LITERAL` knew how to decompose a sum's term, we need something that decomposes a product's factor into its base and its exponent:

```
(DEFUN BASEOF (E) (COND ((ISPOW? E) (BASE E)) (T E)))
(DEFUN EXPOF  (E) (COND ((ISPOW? E) (EXPON E)) (T 1)))
```

If the factor is already a power (`X²`), its base is `X` and its exponent is `2` — nothing to compute. If the factor is anything else (`X`, a constant, an unsimplified product...), it's treated as "raised to the 1," exactly the same trick as `LITERAL` returning `1` for pure constants: it lets the general case and the trivial case be handled with the same code later on.

16.4 Flattening the Product

```
(DEFUN FLATPROD (E)
  (COND ((ISPROD? E) (APPEND (FLATPROD (FACTOR1 E)) (FLATPROD (FACTOR2 E))))
        (T (LIST E))))
```

Identical to `FLATSUM`, swapping `ISSUM?`/`ADDEND`/`AUGEND` for `ISPROD?`/`FACTOR1`/`FACTOR2`. `(FLATPROD '(* 2 (* 2 X)))` gives the list `(2 2 X)`.

16.5 Separating Constants From Symbols

Here a real difference from Chapter 14 shows up: in a sum, every term — constants included — is grouped with the same mechanism (a pure constant simply has literal `1`). In a product we can't do the same, because constants play a different role: they don't get grouped by "base" — it would have to be something like "base 2, exponent 2" to combine `2 · 2`, and that would confusingly mix powers of numbers with powers of symbols. It's simpler and clearer to multiply all the constants directly together, separately:

```
(DEFUN NUMPROD (TERMS)
  (COND ((NULL TERMS) 1)
        ((ISCONST? (CAR TERMS)) (* (CAR TERMS) (NUMPROD (CDR TERMS))))
        (T (NUMPROD (CDR TERMS)))))

(DEFUN SYMTERMS (TERMS)
  (COND ((NULL TERMS) NIL)
        ((ISCONST? (CAR TERMS)) (SYMTERMS (CDR TERMS)))
        (T (CONS (CAR TERMS) (SYMTERMS (CDR TERMS))))))
```

`NUMPROD` walks the list and multiplies only the elements that are constants, ignoring the rest — the base case `1` is the product's identity element, just as `0` was for `REBUILD` in the sum. `SYMTERMS` does the complementary filtering: it keeps only the non-constant factors, which are the ones that actually need to be grouped by base.

16.6 Grouping by Base, Adding Exponents

```
(DEFUN ADDBASE (B EXP ALIST)
  (COND ((NULL ALIST) (LIST (CONS B EXP)))
        ((EQUAL (CAR (CAR ALIST)) B)
         (CONS (CONS B (+ (CDR (CAR ALIST)) EXP)) (CDR ALIST)))
        (T (CONS (CAR ALIST) (ADDBASE B EXP (CDR ALIST))))))

(DEFUN AGRUPAP (TERMS)
  (COND ((NULL TERMS) NIL)
        (T (ADDBASE (BASEOF (CAR TERMS))
                    (EXPOF (CAR TERMS))
                    (AGRUPAP (CDR TERMS))))))
```

Line by line, this is Chapter 14's `ASSOCADD`/`AGRUPA` with the names changed and `BASEOF`/`EXPOF` instead of `LITERAL`/`COEF`. `(AGRUPAP '(X X))` gives `((X . 2))` — the base `X` appears twice, so its accumulated exponent is `2`.

16.7 Rebuilding

```
(DEFUN REBUILDP (ALIST)
  (COND ((NULL ALIST) 1)
        (T (MAKEPROD (MAKEPOW (CAR (CAR ALIST)) (CDR (CAR ALIST)))
                     (REBUILDP (CDR ALIST))))))

(DEFUN SIMPPROD (E)
  (LET ((TERMS (FLATPROD E)))
    (MAKEPROD (NUMPROD TERMS)
              (REBUILDP (AGRUPAP (SYMTERMS TERMS))))))
```

For each `(base . exponent)` pair, `(MAKEPOW base exponent)` is built — which already knows how to reduce exponent `1` to just the base, thanks to Chapter 13 — and they're all multiplied together with cascading `MAKEPROD`. The base case `(NULL ALIST) → 1` is the product's identity element, and for the same reason we saw in `REBUILD`, it keeps the last term from carrying a leftover `· 1`: `MAKEPROD` already knows how to remove it.

`SIMPPROD` ties the three steps together: it flattens the product exactly once (with `LET`, so as not to recompute it twice), multiplies the numeric part separately, and rebuilds the symbolic part grouped by base.

16.8 Integrating `SIMPPROD` Into `SIMP`

With `SIMPPROD` already written, the change to `SIMP` is minimal and symmetric to the one it already had for sums:

```
(DEFUN SIMP (E)
  (COND ((ISCONST? E) E)
        ((ISVAR? E) E)
        ((ISSUM? E)
         (SIMPSUM (MAKESUM (SIMP (ADDEND E)) (SIMP (AUGEND E)))))
        ((ISPROD? E)
         (SIMPPROD (MAKEPROD (SIMP (FACTOR1 E)) (SIMP (FACTOR2 E)))))
        ((ISPOW? E)
         (MAKEPOW (SIMP (BASE E)) (SIMP (EXPON E))))))
```

Before, the `ISPROD?` case called `MAKEPROD` directly. Now it calls `SIMPPROD`, which does everything `MAKEPROD` already did plus groups repeated factors and combines nested constants. The `COND`'s structure is identical to what it's always been — classify, descend recursively, rebuild with the appropriate function — only "the appropriate function" for the product case has changed.

An important detail about this code, corrected after a review: the sum and product cases rebuild the intermediate node with `MAKESUM`/`MAKEPROD` — the constructors from Chapter 10/13 — and not with `(LIST '+ ...)`/`(LIST '* ...)` by hand. Using `LIST` directly here would have violated Chapter 10's abstraction barrier: no function outside that chapter should build a node without going through its constructors. The change is safe because, in the general case (when the already-simplified children are compound subexpressions, neither constants nor zero), `MAKESUM`/`MAKEPROD` don't fire any of their special rules and fall through to their final clause, which is literally `(LIST '+ A1 A2)` — identical to what was written by hand. They only differ when some child is already `0`/`1` or both are constants, and in those cases `SIMPSUM`/`SIMPPROD` would have arrived at the same result anyway through their own flattening — verified with a battery of edge cases, including the total cancellation of a child before reaching this point.

16.9 Trying It — Closing Both Gaps

```
(SIMPPROD '(* 2 (* 2 X)))      ; -> (* 4 X)
(SIMPPROD '(* X X))            ; -> (POT X 2)
(SIMPPROD '(* (* X X) X))      ; -> (POT X 3)

; the case left open in Chapter 15, now genuinely solved
(SIMP '(* 2 (+ X X)))          ; -> (* 4 X)
```

The last example is the real payoff: `SIMP` now walks the whole expression, simplifies the inner sum to `(* 2 X)` (Chapter 15), and now also combines the outer `2` with the inner `2` through `SIMPPROD`, giving `(* 4 X)` directly — the result a human would write without a second thought.

16.10 A Limit Inherited From `DPOW`, Not New to This Chapter

`ADDBASE` adds exponents using MyLISP's real `+`, which assumes exponents are always known numbers. This is consistent with a limitation that already existed since `DPOW` in Chapter 12: our CAS doesn't support symbolic exponents (`x^y` with `y` a variable). If that part is ever extended, `AGRUPAP`/`ADDBASE` would need revisiting too — but this isn't a regression from this chapter, it's the same limitation as always, now visible in one more place.

16.11 To Verify

- [ ] Test `(SIMPPROD '(* 2 (* 2 X)))` and confirm `(* 4 X)`.
- [ ] Test `(SIMPPROD '(* X X))` and confirm `(POT X 2)`.
- [ ] Test `(SIMPPROD '(* (* X X) X))` and confirm `(POT X 3)`.
- [ ] Test `(SIMP '(* 2 (+ X X)))` and confirm `(* 4 X)` — the real closing of Chapter 15's gap.
- [ ] Confirm there's no regression: `(SIMP (DERIV (MAKEPROD 'X 'X) 'X))` still gives `(* 2 X)`, and Chapter 15's `E5` case still gives `(* 4 X)`.

16.12 Chapter 16 Name-Audit Table

Name	Length	Collides?
`BASEOF`	6	no
`EXPOF`	5	no
`FLATPROD`	8	exact, no margin
`NUMPROD`	7	no
`SYMTERMS`	8	exact, no margin
`ADDBASE`	7	no
`AGRUPAP`	7	no
`REBUILDP`	8	exact, no margin
`SIMPPROD`	8	exact, no margin

Four names at the exact 8-character limit, none colliding with the more than 30 names already defined in earlier chapters.

16.13 Next Chapter

With `SIMP` now resolving nested sums and products at any depth, Chapter 17 can finally build the top-level orchestrator that chains `SUBST`, `DERIV`, and `SIMP` for the CAS's end user to use directly — the step we'd been postponing since Chapter 15, now on a much more solid simplification foundation. Also noted, with no urgency: the stable canonical order between *different* addends and factors (not just repetitions), which we already saw in Chapter 14 isn't guaranteed.

Continues in Chapter 17: The Orchestrator — DERIVA and DERIVAEN

Chapter 17 — The Orchestrator: DERIVA and DERIVAEN

> **In one sentence:** instead of one mega-function with optional parameters or modes, we write two small functions that compose the pieces that already exist — `DERIVA` for "differentiate and simplify," `DERIVAEN` for "differentiate, simplify, and evaluate at a point" — following the same philosophy as SICP: function composition already is the interface, there's no need to build one on top.

17.1 Why Not a Mega-Function

Before arriving here, two alternative designs were considered and both were rejected, for different reasons:

- **A single function with optional parameters and `NIL` as a switch** to turn off phases. Rejected because `NIL` already has its own heavily loaded meaning in our CAS — the empty list, the base case of half a dozen recursions — and reusing it also as "this phase doesn't run" is the same overloading of meaning we deliberately avoided by inventing the `VARLIBRE` sentinel in Chapter 11 instead of reusing something generic.
- **A function with an explicit mode**, in the style of a small command interpreter. Rejected because it suggests "differentiate" and "substitute" are mutually exclusive alternatives, when in reality they're independent operations that are normally **chained** (differentiate, and *then* simplify, and *then, if needed,* evaluate at a point) — it's a sequence, not a choice.

The decisive clue comes from looking at how SICP itself handles this: it doesn't solve it with an orchestrator function. When the text wants "differentiate and simplify," it simply writes `(simplify (deriv exp var))` right where it's needed. Function composition with nested parentheses **already is** the interface — nothing needs to be built on top.

17.2 The Code

```
(DEFUN DERIVA (E VBL) (SIMP (DERIV E VBL)))

(DEFUN DERIVAEN (E VBL VAL) (EVALEXPR (SUBST (DERIVA E VBL) VBL VAL)))
```

Two functions, each with a single responsibility and a name that describes it unambiguously:

- **`DERIVA`** — "give me the symbolic derivative, already simplified." Covers the most common use case: nobody wants to see `(+ (* 1 X) (* X 1))`, everybody wants `2X`.
- **`DERIVAEN`** — "give me the numeric value of the derivative at a specific point." Composes `DERIVA` with `SUBST` / `EVALEXPR`, which already exist and have already been verified since Chapter 11 — no new substitution or evaluation logic has been written here, the pieces have just been wired together.

Neither one needs optional parameters, flags, or modes.

17.3 Composing by Hand When More Is Needed

If an expression has more than one variable, `DERIVAEN` only substitutes the variable it's differentiating with respect to — the rest stay free on purpose. To fix them all, you compose by hand with `SUBST`, exactly as you would with any other piece of the CAS:

```
; E4 was already defined back in Chapter 15 as (X + Y)^2
(DERIVA E4 'X)                          ; -> (* 2 (+ Y X))  ; Y free

; Fix X=3 and Y=4 by hand, chaining SUBST:
(EVALEXPR (SUBST (SUBST (DERIVA E4 'X) 'X 3) 'Y 4))
; -> 14   (derivative = 2(x+y), at x=3,y=4 -> 2*7 = 14)
```

No new function is needed for this case — it's exactly the same technique of chaining `SUBST` we already used in Chapter 11, now applied to `DERIVA`'s result instead of to a standalone expression. This is proof that the two-small-functions design is enough: cases it doesn't cover directly are solved by composing pieces that already existed, without having to anticipate every possible combination in advance with a dedicated parameter.

17.4 Second-Order Derivatives, With No New Function at All

A more demanding case confirms that the two-small-functions design also covers higher-order derivatives, without having planned it explicitly: if `DFDX` is `DERIVA`'s result, it's already a valid CAS expression — built with the same `MAKESUM` / `MAKEPROD` / `MAKEPOW` as always — so `DERIVA` can be applied to it again with no changes at all.

```
; f(X,Y) = X^2 * Y^3
(DEFINE F (MAKEPROD (MAKEPOW 'X 2) (MAKEPOW 'Y 3)))

(DEFINE DFDX (DERIVA F 'X))
DFDX                          ; -> (* 2 (* (POT Y 3) X))

(DEFINE D2F (DERIVA DFDX 'Y))    ; mixed partial derivative d2f/dXdY
D2F                           ; -> (* 6 (* (POT Y 2) X))

(EVALEXPR (SUBST (SUBST D2F 'X 3) 'Y 5))
; -> 450
```

Practical note: if, while typing this example interactively on the QL, the interpreter seems to "fail" right before `(DEFINE D2F ...)`, it isn't any bug in `DERIVA` or `SIMP` — it's the ~80-character QDOS keyboard buffer limit we already know from Chapter 12. That limit doesn't just affect the code itself, it also affects any comment typed in the same interactive session right before the line that fails. Loading from a file with `LOAD` instead of typing by hand makes this risk disappear — one more reason to always prefer that route whenever the code isn't trivial.

17.5 Trying It

```
(DEFINE E7 (MAKEPOW 'X 2))
(DERIVA E7 'X)                ; -> (* 2 X)
(DERIVAEN E7 'X 5)            ; -> 10

(DEFINE E8 (MAKEPROD 'X 'X))
(DERIVA E8 'X)                ; -> (* 2 X)
(DERIVAEN E8 'X 5)            ; -> 10
```

17.6 To Verify

- ☐ Test `(DERIVA E7 'X)` and confirm `(* 2 X)`.
- ☐ Test `(DERIVAEN E7 'X 5)` and confirm `10`.
- ☐ Test the manual composition with two variables (`E4`) and confirm `14`.
- ☐ Test the mixed partial derivative of `X²·Y³` and confirm `(EVALEXPR (SUBST (SUBST D2F 'X 3) 'Y 5))` gives `450`.

17.7 Chapter 17 Name-Audit Table

Name	Length	Collides?
`DERIVA`	6	no
`DERIVAEN`	8	exact, no margin

Both names are clearly distinguishable from `DERIV` (5 characters, already existing) and don't collide with anything from the previous seven chapters.

17.8 Next Chapter

Still noted, with no urgency, is the same item carried over since Chapter 14: a stable canonical order between *different* addends and factors (not just repetitions of the same literal or base), so that two mathematically equivalent expressions always produce the same tree, not just the same value. It's the natural candidate to pick back up whenever the simplifier is taken further.

Continues in Chapter 18: Stable Canonical Order (EXPLESS?)

Chapter 18 — Stable Canonical Order (EXPLESS?)

> **In one sentence:** we need a genuine alphabetical order between different variables, and we're lucky that MyLISP already gives us exactly what's needed — `SYMNAME` and `STR<` (§8.2.4-8.2.5) — so this chapter comes down to using them to build `EXPLESS?`, a general expression comparator, and using it to order the terms of a sum or a product.

18.1 The Problem We Were Carrying

Since Chapter 14 we knew the order of different addends in `SIMPSUM`'s result wasn't guaranteed — only the order between repetitions of the *same* literal was. We saw it again in Chapter 17: `(DERIVA '(X+Y)^2 'X)` would sometimes give `(* 2 (+ Y X))` instead of the more intuitive `(* 2 (+ X Y))`. Mathematically they're identical, but two equivalent expressions producing different trees breaks the promise of a true canonical form (recall Chapter 14's definition): if `X+Y` is sometimes represented as `(+ X Y)` and other times as `(+ Y X)`, comparing two expressions with `EQUAL` to know whether they're "the same" stops working.

The solution is to sort the terms alphabetically before rebuilding the tree, so it doesn't matter what order they arrived in: the final result always places the variables in the same spot. That requires a comparator that can decide, for any pair of variable names, which one comes first alphabetically.

18.2 `SYMNAME` and `STR<`: Comparing Symbols as Text

Here we're lucky enough not to have to solve anything new in the interpreter: MyLISP already includes, since Chapter 8, exactly the two primitives needed.

- **`(SYMNAME SYMBOL)`** (§8.2.4) converts a symbol to a text string (`TSTRING`) with its name. It's the gateway between the world of symbols and the world of strings — and, like everything in MyLISP, the result is truncated to 8 characters, the same limitation we already know from Chapter 10.
- **`(STR< A B)`** (§8.2.5) compares two values lexicographically and returns `T` if the first precedes the second. It accepts both strings and symbols directly — if given symbols, it compares them without needing to convert them with `SYMNAME` first.

With these two primitives, comparing two variables alphabetically is as simple as `(STR< 'MASA 'TIEMPO)`. In fact, we already saw in Chapter 8 that a single line is enough to define a reusable symbol comparator: `(DEFUN SYM< (A B) (STR< A B))`. `EXPLESS?`, the comparator we build in this chapter, does exactly that job for variables, but it also knows how to classify constants and compound expressions via `TIPORDEN`.

18.3 Why the Order of Appearance Isn't Enough

Before building the comparator, it's worth ruling out a simpler alternative that seems reasonable at first glance: sorting the terms by the order in which they appeared in the original expression, with no comparator at all. It's easy to check that this doesn't work: `(X+Y)+X` and `Y+(X+X)` are the same mathematical sum, but a traversal that respects order of appearance leaves `X` first in one case and `Y` first in the other — **it isn't canonical**. The problem demands, by definition, an ordering criterion independent of how the input happened to be nested, and that's exactly what a real alphabetical comparison like `STR<`'s gives us.

18.4 The General Comparator: `EXPLESS?`

With `SYMNAME` and `STR<` available, `EXPLESS?` can compare anything:

```
(DEFUN TIPORDEN (E)
  (COND ((ISCONST? E) 0)
        ((ISVAR? E) 1)
        (T 2)))

(DEFUN EXPLESS? (E1 E2)
  (COND ((< (TIPORDEN E1) (TIPORDEN E2)) T)
        ((> (TIPORDEN E1) (TIPORDEN E2)) NIL)
        ((ISCONST? E1) (< E1 E2))
        ((ISVAR? E1) (STR< E1 E2))
        (T NIL)))
```

`TIPORDEN` sets the coarsest priority: constants come before variables, which come before any compound expression (sums, products, powers) — completely general, based only on Chapter 10's classification predicates. Within the same type, `EXPLESS?` breaks the tie: two constants are compared with the real `<`; two variables are compared with `(STR< E1 E2)` — genuine alphabetical comparison, for **any** variable name, not just ones in some table. Two compound expressions still return `NIL` in both directions: that's the one case left unsolved, and it's a problem of a different nature (it would compare whole structures, not names) that we note at the end of the chapter.

18.5 Sorting the List of Terms: Recursive Insertion

MyLISP has no built-in `SORT` function, so we write our own — a classic **insertion sort** algorithm, easy to reason about recursively:

```
(DEFUN INSERTA (PAR LISTA)
  (COND ((NULL LISTA) (LIST PAR))
        ((EXPLESS? (CAR PAR) (CAR (CAR LISTA))) (CONS PAR LISTA))
        (T (CONS (CAR LISTA) (INSERTA PAR (CDR LISTA))))))

(DEFUN ORDENA (LISTA)
  (COND ((NULL LISTA) NIL)
        (T (INSERTA (CAR LISTA) (ORDENA (CDR LISTA))))))
```

`ORDENA` walks the list to the end, recursively sorts the rest, and then uses `INSERTA` to place the first element in its spot within that already-sorted tail — the "insertion sort" pattern exactly as taught in any algorithms course, here expressed in a purely recursive way with no mutable variable at all, like the rest of the CAS. Note that `ORDENA` works on lists of `(key . value)` pairs — the same association lists `AGRUPA`/`AGRUPAP` already produce — and compares only the keys with `EXPLESS?`; the value (`coefficient` or `exponent`, depending on the case) travels along untouched.

18.6 Wiring It Into `SIMPSUM` and `SIMPPROD`

The change to the two already-existing functions is minimal, and it's a sign that the layered architecture keeps paying dividends: it's enough to slot `ORDENA` in between `AGRUPA`/`AGRUPAP` and `REBUILD`/`REBUILDP`.

```
(DEFUN SIMPSUM (E) (REBUILD (ORDENA (AGRUPA (FLATSUM E)))))

(DEFUN SIMPPROD (E)
  (LET ((TERMS (FLATPROD E)))
    (MAKEPROD (NUMPROD TERMS)
              (REBUILDP (ORDENA (AGRUPAP (SYMTERMS TERMS)))))))
```

No other function in the CAS — `DERIV`, `SUBST`, `SIMP`, `DERIVA`, `DERIVAEN` — needs to change a single line. They all build their results by calling `SIMPSUM`/`SIMPPROD`, so they all benefit from the new stable order automatically, exactly as `DERIV` benefited without changes from the smart constructors in Chapter 13.

18.7 Trying It

```
(SIMPSUM '(+ (+ X Y) X))     ; -> (+ (* 2 X) Y)
(SIMPSUM '(+ Y (+ X X)))     ; -> (+ (* 2 X) Y)   -- same result
(SIMPSUM '(+ (+ Y X) X))     ; -> (+ (* 2 X) Y)   -- same result again

; the case we've been carrying since Chapter 17
(DEFINE E4 (MAKEPOW (MAKESUM 'X 'Y) 2))
(DERIVA E4 'X)               ; -> (* 2 (+ X Y))   -- before: (* 2 (+ Y X))

; symbols with more than one letter, not just single letters
(SIMPSUM '(+ TIEMPO MASA))    ; -> (+ MASA TIEMPO)   -- real alphabetical order
(SIMPSUM '(+ MASA TIEMPO))    ; -> (+ MASA TIEMPO)   -- same result
```

The first three lines start from the same mathematical sum written in three different orders, and all three give **exactly the same tree** — not just the same value. The last two show that `STR<` isn't limited to single-letter variables: `MASA` (mass) and `TIEMPO` (time) are sorted in genuine, stable alphabetical order, exactly like any pair of single letters.

18.8 The Limit That Remains Open

With `STR<`, comparing **variables** has no reasonable limit: any name, of any length (up to the truncation to 8 characters we already know from Chapter 10, a different and already-documented limitation), gets sorted in true alphabetical order. The one thing `EXPLESS?` still doesn't solve is comparing **two compound expressions against each other** — for example, deciding whether `(* X Y)` should come before or after `(POT Z 2)` in a sum that mixed both as full terms. That's a problem of a different nature: it isn't about comparing names, but about defining an order between whole structures (by their main operator? recursively by their parts?), and it's outside this chapter's scope — noted as a possible future extension, not as a platform limitation like the previous ones.

18.9 To Verify

- [ ] Test the three variants of `(SIMPSUM ...)` with `X`, `Y` in different input order and confirm all three give `(+ (* 2 X) Y)`.
- [ ] Test `(DERIVA E4 'X)` and confirm `(* 2 (+ X Y))`.
- [ ] Test `(SIMPSUM '(+ TIEMPO MASA))` and `(SIMPSUM '(+ MASA TIEMPO))` and confirm both give `(+ MASA TIEMPO)` — real alphabetical order between symbols of more than one letter.
- [ ] Test `(STR< 'MASA 'TIEMPO)` and `(SYMNAME 'X)` standalone, to review these two Chapter 8 primitives before seeing them inside `EXPLESS?`.

18.10 Chapter 18 Name-Audit Table

Name	Length	Collides?
`TIPORDEN`	8	exact, no margin
`EXPLESS?`	8	exact, no margin
`INSERTA`	7	no
`ORDENA`	6	no

Two names at the exact 8-character limit, neither colliding with the more than 35 names already defined in earlier chapters.

`SYMNAME` and `STR<` don't appear in this table because they're primitives of the MyLISP interpreter itself, introduced in Chapter 8 — not functions defined in this chapter.

18.11 Next Chapter

With canonical order solved for the case that matters, Chapter 19 can tackle infix printing (`PRINTMAT`): converting the CAS tree into readable mathematical notation (`2x + y` instead of `(+ (* 2 X) Y)`), with the technical challenge of deciding when parentheses are needed based on operator precedence — the natural close of the whole manual.

Continues in Chapter 19: Infix Printing (PRINTMAT)

Chapter 19 — Infix Printing (PRINTMAT)

> **In one sentence:** we convert the CAS tree into readable mathematical notation — `2x + y` instead of `(+ (* 2 X) Y)` — by passing down, at each recursive call, the minimum precedence a subtree needs in order not to carry parentheses. It's the manual's natural close: the user stops reading S-expressions and starts reading mathematics.

19.1 Three Chapter 8 Primitives, Now With a New Use

Building `PRINTMAT` needs two very specific capabilities: **concatenating text** and **showing it without quotes or an automatic line break**. The good news is that MyLISP already gave us these in Chapter 8 — `STRCAT`, `DISPLAY`, and `NEWLINE` — designed back then for building messages and formatted output; here we reuse them to build, piece by piece, the text representation of an algebraic expression. `PRINT`, as we've known it since Chapter 11, adds a newline after each call and wraps strings in quotes — perfect for debugging, useless for building an output line piece by piece. The division of responsibilities among the three is, in fact, the same model Scheme uses:

- **`PRINT`** — debugging: shows with quotes and a line break, returns the value (so it can be inserted inside an expression without breaking the flow, as we already did in Chapter 11 with `VARLIBRE`).
- **`DISPLAY`** — user-facing output: no quotes, no line break.
- **`NEWLINE`** — explicit control over when to end the line.

19.2 The Theory: Precedence as "the Minimum Needed to Avoid Parentheses"

The central idea is simple and powerful: every operator has a precedence, and a subtree needs parentheses when its own precedence is **lower** than what's demanded by the context it appears in. Formalized as a table:

Node	Precedence
Constant or variable	4 (never needs parentheses)
Power (`POT`)	3
Product (`*`)	2
Sum (`+`)	1

```
(DEFUN PRECOP (E)
  (COND ((ISPOW? E) 3)
        ((ISPROD? E) 2)
        ((ISSUM? E) 1)
        (T 4)))
```

The decision rule, used to wrap or not wrap a subtree already converted to text:

```
(DEFUN ENVOLVER (E S MINIMA)
  (COND ((< (PRECOP E) MINIMA) (STRCAT "(" (STRCAT S ")")))
        (T S)))
```

`ENVOLVER` takes the original node `E` (to look up its precedence with `PRECOP`), the text `S` already generated for that node, and the minimum precedence demanded by whoever called it. If its own precedence is strictly lower than what's demanded, it gets wrapped in parentheses; if not, it's returned as is.

19.3 The Nuance Your Own Example Didn't Cover: Power Isn't Associative

The "strictly lower" rule is exactly correct for `+` and `*` because they're **associative** operators: `(+ X (+ Y Z))` can be written with no internal parentheses as `x + y + z` with no ambiguity, so a `+` nested inside another `+` (same precedence, not *lower*) doesn't need wrapping — and the rule, as is, already allows this correctly.

Power is different: `(x²)³` and `x^(2³)` **are not the same expression** (`64` versus `256` if you plug in values). If we applied the rule without further nuance to `(POT (POT X 2) 3)`, the base is itself a power with the *same* precedence (3) as the parent node — and "3 is not less than 3," so it wouldn't get wrapped, giving `x^2^3` with no parentheses: ambiguous and mathematically different from what the tree represents.

The solution isn't a new rule — it's requiring, **only for the position of a power's base**, a stricter minimum precedence than usual: instead of demanding "at least 3" (which would let another power through unwrapped), we demand "at least 4" — that is, that it be an atom (constant or variable). Anything that isn't a pure atom as the base of a power gets parentheses, no exceptions: `(x+y)²`, `(xy)²`, `(x²)³` — all three cases, correct, with the same rule:

```
(DEFUN INFBASE (B) (INFIJO B 4))
```

The exponent, on the other hand, doesn't need this reinforcement: `x^y^2` (a power as the exponent of another) reads naturally as `x^(y²)` — exponentiation associating to the right, the standard mathematical convention — which is exactly what our tree `(POT X (POT Y 2))` represents. There, the normal rule (minimum 3, not 4) is correct as is.

19.4 Typographical Finishing Touches That Come Free From the Same Technique

Before writing `INFIJO` in its final form, two details that greatly improve readability and that are solved with the same tool (checking whether a factor is a constant), with no new concept added:

Coefficient juxtaposition. `2 * X` reads better as `2x`, with no explicit operator — it's the usual convention in algebra. But this is **only** safe when one of the factors is a numeric constant: for two symbols (`(* X Y)`), juxtaposing them would give `xy`, which would be ambiguous or outright incorrect for symbols of more than one letter, like the ones from Chapter 18 (`MASA` and `TIEMPO` juxtaposed would give `MASATIEMPO`, unreadable). That's why juxtaposition is reserved **exclusively** for "constant multiplying anything else"; between two non-constant factors, an explicit `*` is used.

Implicit subtraction. `X + (-3)` reads better as `X - 3`, not as `X + -3`. It's enough to detect whether the second addend of a sum is "negative" (a negative constant, or a product with a negative coefficient) and, if so, print `" - "` followed by the sign-flipped version of that same term — reusing `MAKEPROD`, which already knows how to simplify the `-1 →` nothing case (coefficient `1` removes itself, thanks to Chapter 13).

```
(DEFUN NEGTERM? (E)
  (COND ((ISCONST? E) (< E 0))
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E)) (< (FACTOR1 E) 0))
```

```
               ((ISCONST? (FACTOR2 E)) (< (FACTOR2 E) 0))
               (T NIL)))
        (T NIL)))

(DEFUN NEGAR (E)
  (COND ((ISCONST? E) (- E))
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E))
                (MAKEPROD (- (FACTOR1 E)) (FACTOR2 E)))
               ((ISCONST? (FACTOR2 E))
                (MAKEPROD (FACTOR1 E) (- (FACTOR2 E))))
               (T E)))
        (T E)))
```

An elegant detail: `(NEGAR '(* -1 X))` gives `(MAKEPROD 1 X)`, which `MAKEPROD` already reduces to just `X` (the `UNO?` rule from Chapter 13) — so `- (-1)·X` prints as `- x`, not as `- 1x`, with no additional special case at all. The layered architecture pays dividends once again.

19.5 The Complete Code

```
(DEFUN PRECOP (E)
  (COND ((ISPOW? E) 3)
        ((ISPROD? E) 2)
        ((ISSUM? E) 1)
        (T 4)))

(DEFUN ENVOLVER (E S MINIMA)
  (COND ((< (PRECOP E) MINIMA) (STRCAT "(" (STRCAT S ")")))
        (T S)))

(DEFUN NEGTERM? (E)
  (COND ((ISCONST? E) (< E 0))
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E)) (< (FACTOR1 E) 0))
               ((ISCONST? (FACTOR2 E)) (< (FACTOR2 E) 0))
               (T NIL)))
        (T NIL)))

(DEFUN NEGAR (E)
  (COND ((ISCONST? E) (- E))
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E))
                (MAKEPROD (- (FACTOR1 E)) (FACTOR2 E)))
               ((ISCONST? (FACTOR2 E))
                (MAKEPROD (FACTOR1 E) (- (FACTOR2 E))))
               (T E)))
        (T E)))

(DEFUN INFBASE (B) (INFIJO B 4))

(DEFUN INFPROD (E)
  (COND ((ISCONST? (FACTOR1 E))
         (COND ((= (FACTOR1 E) -1) (STRCAT "-" (INFIJO (FACTOR2 E) 2)))
               (T (STRCAT (FACTOR1 E) (INFIJO (FACTOR2 E) 2)))))
        ((ISCONST? (FACTOR2 E))
         (COND ((= (FACTOR2 E) -1) (STRCAT "-" (INFIJO (FACTOR1 E) 2)))
               (T (STRCAT (FACTOR2 E) (INFIJO (FACTOR1 E) 2)))))
```

```
        (T (STRCAT (INFIJO (FACTOR1 E) 2)
                   (STRCAT "*" (INFIJO (FACTOR2 E) 2))))))

(DEFUN INFSUMA (E)
  (COND ((NEGTERM? (AUGEND E))
         (STRCAT (INFIJO (ADDEND E) 1)
                 (STRCAT " - " (INFIJO (NEGAR (AUGEND E)) 1))))
        (T (STRCAT (INFIJO (ADDEND E) 1)
                   (STRCAT " + " (INFIJO (AUGEND E) 1))))))

(DEFUN INFIJO (E MINIMA)
  (COND ((ISCONST? E) (STRCAT E ""))
        ((ISVAR? E) (STRCAT E ""))
        ((ISPOW? E)
         (ENVOLVER E
                   (STRCAT (INFBASE (BASE E))
                           (STRCAT "^" (INFIJO (EXPON E) 3)))
                   MINIMA))
        ((ISPROD? E) (ENVOLVER E (INFPROD E) MINIMA))
        ((ISSUM? E) (ENVOLVER E (INFSUMA E) MINIMA))))

(DEFUN PRINTMAT (E) (PROGN (DISPLAY (INFIJO E 0)) (NEWLINE)))
```

`INFIJO` is the recursive engine: it classifies the node, builds the text for its parts (lowering the appropriate minimum precedence for each position — `2` for a product's factors, `1` for addends, `3` for a power's exponent, a reinforced `4` for the base), and wraps the result with `ENVOLVER` only if needed. `PRINTMAT` is the function the reader will use: it converts to text with `MINIMA=0` (the lowest possible level, so the complete expression is never wrapped in unnecessary parentheses at the root) and shows it with `DISPLAY` + `NEWLINE`.

19.6 A Platform Limit Worth Saying Out Loud: `STRCAT` Truncates to 36 Characters

Every call to `STRCAT` that exceeds 36 characters gets truncated — and since `INFIJO` builds its result through successive concatenations, an expression whose text representation exceeds that length would lose information **silently**, with no warning at all. For every expression in this manual — derivatives of one or two variables, small exponents — the result always stays well under the limit (Chapter 17's mixed derivative, `6x*y^2`, takes up 6 characters). But this is a real limitation of this implementation, not an oversight: `PRINTMAT`, as it stands, **isn't designed for very long expressions**. Extending it properly would require not accumulating the complete result into a single string before showing it, but instead calling `DISPLAY` piece by piece as each fragment is generated — a deeper redesign that's outside the scope of this closing chapter, and noted here with the same honesty as the rest of the manual's platform limits.

19.7 Trying It — Closing the Circle

```
(PRINTMAT (MAKESUM (MAKEPROD 2 'X) 'Y))          ; 2x + y
(PRINTMAT (MAKEPROD 2 (MAKESUM 'X 'Y)))          ; 2(x + y)
(PRINTMAT (MAKESUM 2 (MAKEPROD 'X 'Y)))          ; 2 + x*y
(PRINTMAT (MAKEPOW (MAKEPOW 'X 2) 3))            ; (x^2)^3
(PRINTMAT (MAKEPOW 'X (MAKEPOW 'Y 2)))           ; x^y^2
(PRINTMAT (MAKESUM 'X (MAKEPROD -1 'Y)))         ; x - y
(PRINTMAT (MAKEPROD -1 (MAKESUM 'X 'Y)))         ; -(x + y)

; the real payoff: Chapter 17's derivative, actually printed
(DEFINE E1 (MAKEPOW 'X 2))
(PRINTMAT (DERIVA E1 'X))                        ; 2x
```

```
; the mixed partial derivative from Chapter 17/18
(DEFINE F (MAKEPROD (MAKEPOW 'X 2) (MAKEPOW 'Y 3)))
(PRINTMAT (DERIVA (DERIVA F 'X) 'Y))             ; 6x*y^2
```

Three and a half chapters after posing the problem in Chapter 12 — differentiate, simplify, sort canonically, and now display in readable notation — the complete CAS, start to finish, does exactly what this manual's very first paragraph asked for: take in a mathematical expression and return another one, in a language a human reads effortlessly.

19.8 To Verify

- ☐ Test the seven standalone `PRINTMAT` cases and confirm each exact output.
- ☐ Test `(PRINTMAT (DERIVA E1 'X))` and confirm `2x`.
- ☐ Test `(PRINTMAT (DERIVA (DERIVA F 'X) 'Y))` and confirm `6x*y^2`.
- ☐ Confirm that `(PRINTMAT (MAKEPOW (MAKEPOW 'X 2) 3))` gives `(x^2)^3` **with** parentheses — the case that motivated the reinforced precedence for a power's base.

19.9 Chapter 19 Name-Audit Table

Name	Length	Collides?
`PRECOP`	6	no
`ENVOLVER`	8	exact, no margin
`NEGTERM?`	8	exact, no margin
`NEGAR`	5	no
`INFBASE`	7	no
`INFPROD`	7	no
`INFSUMA`	7	no
`INFIJO`	6	no
`PRINTMAT`	8	exact, no margin

Three names at the exact 8-character limit, none colliding with the more than 40 names already defined throughout the manual. Note that the chapter isn't called `PRINT-MATH` in the code — that name has 10 characters and a hyphen which, while valid in a symbol, wouldn't fit within the 8-character limit — but `PRINTMAT`, the same naming discipline we've been applying since Chapter 10.

19.10 Closing the Manual

With `PRINTMAT`, the CAS built across these ten chapters is complete from end to end: representation (Ch. 10), substitution and evaluation (Ch. 11), raw differentiation (Ch. 12), local simplification via smart constructors (Ch. 13), grouping like terms (Ch. 14) and full-tree simplification (Ch. 15-16), a top-level orchestrator (Ch. 17), stable canonical order (Ch. 18), and now readable output (Ch. 19) — all of it verified on real hardware at every step, with every platform limit documented the moment it was discovered, never hidden. Noted for later, with no urgency, are the natural candidates for anyone who wants to keep extending the CAS: canonical order between compound expressions, repeated factors inside sums of powers, and the domains explicitly left out from the start (symbolic integration, equation solving, transcendental functions). The next chapter gathers all of this into a reference guide and an honest account of what this CAS can do and what, deliberately, it leaves out.

Continues in Chapter 20: Summary of Our CAS — What We Built and What We Didn't

Chapter 20 — Summary of Our CAS: What We Built and What We Didn't

> **In one sentence:** we close Part II with a usage cheat sheet — for when you come back in a few weeks and just want to *use* the CAS without rereading the source code — and with an honest account of what this small symbolic algebra system can do and what, deliberately, we left out.

20.1 Basic Vocabulary (the Input Rules)

- Every input expression is built either with the Chapter 10/13 constructors (`MAKESUM`, `MAKEPROD`, `MAKEPOW`), or by writing a quoted list directly with the three operators the CAS recognizes: `'(+ ...)`, `'(* ...)`, `'(POT ...)`. Both forms produce exactly the same internal structure; the quoted list is faster to type, the constructors are more convenient when the expression is assembled inside Lisp code from variables.
- Calculation variables are always quoted: `'X`, not bare `X` (otherwise MyLISP would try to evaluate `X` as if it were a program variable, not a mathematical symbol).
- **There's no subtraction or division operator in the tree.** The CAS only recognizes `+`, `*`, and `POT`. A subtraction is expressed as a sum with a negative coefficient: `x - 3` is written `'(+ X -3)` or `(MAKESUM 'X -3)`; `x - y` is written `'(+ X (* -1 Y))`. `PRINTMAT` (see below) does know how to display this readably as `x - 3` / `x - y`, even though internally it's a sum.

20.2 Quick API Guide

The functions for direct use:

Function	What it does
`(DERIVA E VBL)`	Differentiates `E` with respect to `VBL` and returns the already-simplified result.
`(DERIVAEN E VBL VAL)`	Differentiates `E` with respect to `VBL` and evaluates the result by substituting `VBL` with `VAL`. Only fixes that one variable — if `E` has more than one, the rest stay free (see the two-variable example below).
`(EVALEXPR (SUBST E VBL VAL))`	Numerically evaluates an **already-existing** expression (without differentiating), fixing `VBL` at `VAL`.
`(PRINTMAT E)`	Displays any CAS expression in readable mathematical notation.

Pieces for you to compose yourself (Chapter 17's philosophy: no giant functions with flags, these small pieces get combined as needed):

Function	What it does
`(SUBST E VBL VAL)`	Substitutes `VBL` with `VAL` inside `E`, without evaluating.
`(DERIV E VBL)`	**Raw** symbolic derivative, unsimplified (`DERIVA` minus the `SIMP` step).
`(SIMP E)`	Simplifies any already-built expression, recursively across the whole tree.
`(EVALEXPR E)`	Numerically evaluates `E` if it has no free variables; if it does, returns the `VARLIBRE` sentinel (Chapter 11).

For more than one free variable, `SUBST` is chained by hand as many times as needed before `EVALEXPR` — there's no (nor is there a need for) a version of `DERIVAEN` with more parameters.

20.3 Translation Table: Mathematics → MyLISP

Mathematical expression	MyLISP construction
`x²`	`(MAKEPOW 'X 2)` — or `'(POT X 2)`
`2x`	`(MAKEPROD 2 'X)` — or `'(* 2 X)`
`x + 3`	`(MAKESUM 'X 3)` — or `'(+ X 3)`
`x – 3`	`(MAKESUM 'X -3)` — or `'(+ X -3)`
`x · y`	`(MAKEPROD 'X 'Y)` — or `'(* X Y)`
`x – y`	`'(+ X (* -1 Y))`
`x² + 3x`	`(MAKESUM (MAKEPOW 'X 2) (MAKEPROD 3 'X))` — or `'(+ (POT X 2) (* 3 X))`
`(x + y)²`	`(MAKEPOW (MAKESUM 'X 'Y) 2)` — or `'(POT (+ X Y) 2)`
`x²·y³`	`(MAKEPROD (MAKEPOW 'X 2) (MAKEPOW 'Y 3))`

20.4 Complete Workflow Example

```
; Define the function: f(x) = x^2 + 3x
(DEFINE F '(+ (POT X 2) (* 3 X)))

; See the readable function:
(PRINTMAT F)                    ; -> x^2 + 3x

; See its readable derivative:
(PRINTMAT (DERIVA F 'X))        ; -> 3 + 2x

; Compute the slope at X=5:
(DERIVAEN F 'X 5)                ; -> 13
```

Notice `3 + 2x`, not `2x + 3` — this isn't a bug, it's Chapter 18's canonical order at work (constants always come before variables). If this surprises you weeks after finishing the book, this is exactly why.

With two variables, `DERIVAEN` only fixes the one being differentiated; the other is substituted by hand with `SUBST`, chaining as many calls as there are remaining free variables:

```
(DEFINE G (MAKEPOW (MAKESUM 'X 'Y) 2))     ; g(x,y) = (x+y)^2

(PRINTMAT (DERIVA G 'X))                            ; -> 2(x + y)
(EVALEXPR (SUBST (SUBST (DERIVA G 'X) 'X 3) 'Y 4))   ; -> 14
```

20.5 What Our CAS Can Do

It's worth stating this positively before moving to the limitations, because the list is longer than it seems once you've built it chapter by chapter without stepping back to look at the whole:

- **Represents arbitrary algebraic expressions** with sums, products, and powers nested to any depth, using unevaluated MyLISP lists (Chapter 10).
- **Substitutes variables with values or other subexpressions**, and evaluates numerically once no free variable remains — detecting the opposite case with its own error mechanism, without depending on the host interpreter's environment (Chapter 11).
- **Differentiates symbolically** using the mechanical rules of calculus (sum, product, power with a constant exponent), including second-order derivatives and mixed partial derivatives of several variables (Chapters 12 and 17).
- **Really simplifies, not just superficially**: applies algebraic identities the moment each node is built (Chapter 13), combines like terms inside sums (Chapter 14) and repeated factors inside products (Chapter 16), and walks the entire tree so simplification doesn't stop halfway at the first level (Chapter 15).
- **Produces a canonical, stable result**: the same mathematical expression, written in different ways, always reduces to the same internal tree — including a genuine alphabetical order between variables of any length (Chapter 18).
- **Displays the result in readable mathematical notation**, with the minimum parenthesization each subexpression needs based on its precedence (Chapter 19) — instead of forcing you to read S-expressions.
- All of it built with the same MYLISP pieces presented in Part I: lists, recursion, `LAMBDA`, `COND`, homoiconicity — with no trick or magic primitive added along the way (§10.1).

20.6 The Known Boundaries: What Our CAS Doesn't Do

- **Doesn't support symbolic exponents** (`x^y` with `y` a variable) — `DPOW` / `ADDBASE` assume the exponent is always a known numeric constant (Chapters 12 and 16). A real CAS would need different differentiation rules for this case (the general logarithmic-differentiation rule), which are outside this manual's scope.
- **Doesn't guarantee canonical order between compound structures** — only between constants and between variables (genuine alphabetical comparison since Chapter 18). Two sums or products of compound expressions may appear in different order depending on how they were built.
- **`PRINTMAT` doesn't print very long expressions** — every intermediate piece it generates goes through `STRCAT`, which truncates at 36 characters (Chapter 19). This manual's expressions never reach that limit, but it's a real limitation, not a cosmetic one.
- **The interpreter's string table has only 50 entries**, and the garbage collector only triggers by looking at the main heap's occupancy, not `strtab`'s (a Chapter 19 finding). If you chain many calls to `PRINTMAT` / `STR<` / `SYMNAME` in a row during a long session, it can run out before the main heap gives a GC any reason to fire. If it happens, restart the interpreter or split your tests into smaller blocks loaded separately.
- **Doesn't include symbolic integration, equation solving, or transcendental functions** (`SIN`, `COS`, `LOG`...) — these were left out of scope from the manual's original outline, not as a last-minute limitation. Each one is, in itself, an entire chapter of new rules: integration requires recognizing patterns and sometimes has no elementary solution; solving equations requires manipulating the expression across the other side of an `=` that this CAS doesn't even represent; transcendental functions would require extending `DERIV`, `SIMP`, and `PRINTMAT` with new cases for each one.

20.7 From This CAS to a Real CAS

§10.1 opened Part II by recalling that systems like Macsyma, Maple, Mathematica, Maxima, or SymPy do, in essence, the same thing this small CAS does: represent expressions symbolically and transform them by applying rules. The difference isn't one of kind, but of scale — more differentiation and integration rules covering more functions, more simplification strategies competing with each other to choose the "simplest" form (a far from trivial problem: which is simpler, `(x+1)²` or `x²+2x+1`?), support for complex numbers and matrices, solving equations and systems, and years of work optimizing all of it to be fast on expressions with thousands of terms.

But the underlying architecture — represent, walk recursively, simplify by rule rewriting, keep a canonical form so comparison is possible — is exactly what's been built here, chapter by chapter, with LISP's simplest tools. Anyone who's followed the manual this far already has the conceptual foundation to read a real CAS's source code and recognize, underneath the added complexity, the same pieces: smart constructors, recursive simplifiers, a canonical order, and a function that turns trees into readable text.

\newpage

Appendices

Appendix A — Quick Reference

A.1 Special Forms

Form	Syntax	Description
`QUOTE`	`(QUOTE expr)` or `'expr`	Returns `expr` unevaluated
`IF`	`(IF cond then [else])`	Conditional branch
`COND`	`(COND (c1 e1) ... (T eN))`	Multi-way case
`AND`	`(AND e1 e2 ...)`	Conjunction with short-circuit
`OR`	`(OR e1 e2 ...)`	Disjunction with short-circuit
`PROGN`	`(PROGN e1 e2 ...)`	Sequence; returns the last one
`LET`	`(LET ((v1 e1) ...) body)`	Local variables (parallel binding)
`LAMBDA`	`(LAMBDA (params) body)`	Anonymous function / closure
`DEFINE`	`(DEFINE name value)`	Global definition
`DEFUN`	`(DEFUN name (params) body)`	Named function (shorthand for DEFINE+LAMBDA)
`LOAD`	`(LOAD "mdvX_name")`	Load and evaluate a file

A.2 List Primitives

Function	Syntax	Description
`CAR`	`(CAR list)`	First element
`CDR`	`(CDR list)`	Rest (everything except the first)
`CONS`	`(CONS elem list)`	Build a pair/list
`LIST`	`(LIST e1 e2 ...)`	Build a list of elements
`APPEND`	`(APPEND list1 list2)`	Concatenate two lists
`NULL`	`(NULL expr)`	Is it NIL?
`ATOM`	`(ATOM expr)`	Is it an atom?
`LISTP`	`(LISTP expr)`	Is it a list?

A.3 Numeric Primitives

Function	Syntax	Description
`+`	`(+ n1 n2 ...)`	Sum (variadic)
`-`	`(- n1 n2 ...)` or `(- n)`	Subtraction / negation
`*`	`(* n1 n2 ...)`	Multiplication (variadic)
`/`	`(/ n1 n2 ...)` or `(/ n)`	Exact division / reciprocal. Produces a rational if there's a remainder
`DIV`	`(DIV a b)`	Integer quotient (truncates toward zero). TINT only
`MOD`	`(MOD a b)`	Integer remainder (same sign as a). TINT only
`=`	`(= n1 n2)`	Numeric equality (via float)
`<`	`(< n1 n2)`	Less than
`>`	`(> n1 n2)`	Greater than
`<=`	`(<= n1 n2)`	Less than or equal
`>=`	`(>= n1 n2)`	Greater than or equal
`NUMBERP`	`(NUMBERP expr)`	Is it a number?

A.4 Equality Predicates

Function	Compares	Types	Use
`EQ`	Symbol identity	Symbols and NIL only	Comparing symbols
`=`	Numeric value (via float)	Numbers of any type	Numeric equality
`EQUAL`	Structure and type	Anything	Structural equality

A.5 Other Primitives

Function	Syntax	Description
`NOT`	`(NOT expr)`	Logical negation
`SYMBOLP`	`(SYMBOLP expr)`	Is it a symbol?
`EVAL`	`(EVAL expr)`	Evaluate an expression
`PRINT`	`(PRINT expr)`	Print with reader format + line break; returns the argument
`DISPLAY`	`(DISPLAY expr)`	Print without quotes or line break; returns VOID
`NEWLINE`	`(NEWLINE)`	Emit a line break; returns NIL
`SYMNAME`	`(SYMNAME symbol)`	Return the symbol's name as a string (max. 8 chars)
`STR<`	`(STR< a b)`	Alphabetical comparison; accepts TSTRING or TSYM
`STRCAT`	`(STRCAT a b)`	Concatenate two values as a string; accepts TSTRING, TSYM, TINT, TRAT; truncates to 36 chars

A.6 Functions Definable in Pure LISP

These functions are **not primitives** of MYLISP; they must be defined before use.

```
; List length
(DEFUN LENGTH (L)
  (IF (NULL L) 0 (+ 1 (LENGTH (CDR L)))))

; Nth element (0-based)
(DEFUN NTH (L N)
  (IF (= N 0) (CAR L) (NTH (CDR L) (- N 1))))

; CADR, CADDR shorthands
(DEFUN CADR   (L) (CAR (CDR L)))
(DEFUN CADDR  (L) (CAR (CDR (CDR L))))
(DEFUN CADDDR (L) (CAR (CDR (CDR (CDR L)))))

; Absolute value
(DEFUN ABS (X) (IF (< X 0) (- X) X))

; Max and min
(DEFUN MAXI (A B) (IF (> A B) A B))
(DEFUN MINI (A B) (IF (< A B) A B))

; Reverse a list
(DEFUN REVERSE (L)
  (DEFUN REVERSE-ACC (L ACC)
    (IF (NULL L) ACC (REVERSE-ACC (CDR L) (CONS (CAR L) ACC))))
  (REVERSE-ACC L NIL))

; MAP, FILTER, REDUCE
(DEFUN MY-MAP (F L)
  (IF (NULL L) NIL
    (CONS (F (CAR L)) (MY-MAP F (CDR L)))))
```

```
(DEFUN MY-FILTER (P L)
  (COND ((NULL L) NIL)
        ((P (CAR L)) (CONS (CAR L) (MY-FILTER P (CDR L))))
        (T (MY-FILTER P (CDR L)))))

(DEFUN MY-REDUCE (F I L)
  (IF (NULL L) I
    (MY-REDUCE F (F I (CAR L)) (CDR L))))

; GCD (Euclid's algorithm)
(DEFUN GCD (A B)
  (IF (= B 0) A (GCD B (MOD A B))))

; Square root (Newton-Raphson)
(DEFUN MY-SQRT (X) (SQRT-ITER 1.0 X))
(DEFUN SQRT-ITER (G X)
  (IF (< (ABS (- (* G G) X)) 0.001)
    G (SQRT-ITER (/ (+ G (/ X G)) 2.0) X)))
```

Continues in Appendix B: Architecture and Limitations

Appendix B — Architecture and Limitations

B.1 The Two MYLISP Implementations

MYLISP exists in two implementations that share the same language but differ in internal architecture:

MYLISP/QL is written in **Prospero Pro Pascal** and compiled for the Sinclair QL (Motorola 68008). It uses a **recursive descent evaluator**: the Pascal `Eval` function recursively calls itself to evaluate subexpressions; every LISP function call consumes a Pascal stack frame.

MYLISP/Next is written in **C with Z88DK** for the ZX Spectrum Next (Z80 at 28 MHz). It uses an **iterative evaluator** with an explicit task stack stored in paged memory. User recursion doesn't consume CPU stack; its real limit is the heap, not call depth.

B.2 Platform Comparison Table

Feature	MYLISP/QL	MYLISP/Next
Hardware	Sinclair QL, 68008 at 7.5 MHz	ZX Spectrum Next, Z80 at 28 MHz
Compiler / language	Prospero Pro Pascal	Z88DK / C
Storage	Microdrive (mdv1, *mdv2*)	SD card (esxdos)
Evaluator	Recursive (consumes Pascal stack)	Iterative (paged explicit stack)
Heap (`MAXCELL`)	**24,000 cells**	**32,000 cells**
Symbol table (`MAXSYM`)	200	200
Symbol length (`SYMLEN`)	**8 chars** (truncates)	**8 chars** (truncates)
String table (`MAXSTR`)	50	50
Max. string length (`STRLEN`)	36 chars	36 chars
Floats	Separate table (`MAXREAL`=100)	Packed into the cell
REPL buffer (total accumulated)	**80 characters**	**500 characters**
LOAD buffer	500 characters	500 characters
GC at the REPL	At 80% of the heap	At 80% of the heap
GC during LOAD	At 50% of the heap	**Doesn't trigger**
`(CLEAN)` from inside LOAD	Runs the GC	Ignored with a warning
Recursion depth	Limited by the Pascal stack	Limited only by the heap
Exit	`BYE`	`BYE`

B.3 The REPL and Its Limits

The REPL accepts multi-line expressions. The total accumulated length of an expression has a limit that varies by platform (see table B.2). For programs longer than a few lines, it's always better to save the code to a file and load it with `LOAD`.

B.4 The Namespace (Lisp-1)

MYLISP is a **Lisp-1**: there's a single environment holding both variables and functions. Defining `(DEFINE SUM 42)` after `(DEFUN SUM (A B) (+ A B))` destroys the function and replaces it with the number.

B.5 The Garbage Collector

MYLISP uses a **mark-and-sweep** garbage collector:

1. **Mark:** walks every cell reachable from the global environment and marks it as "alive."
2. **Sweep:** frees every unmarked cell, returning it to the free list.

The GC is completely silent: the user gets no warning when it runs automatically. The trigger thresholds and the behavior during `LOAD` vary between platforms (see table B.2).

Important guarantee: the GC never fires in the middle of an evaluation, only between complete expressions. This guarantees it can never corrupt partially built structures.

B.6 Internal Data Types

The types are identical on both platforms. The only difference is `TFLOAT`'s internal representation: on the QL it's stored as an index into a separate table; on the Next it's packed directly into the cell's bytes.

Type	LISP type	Description
`TPAIR`	Pair/list	Cons cell: two references (CAR, CDR)
`TSYM`	Symbol	Index into the symbol table
`TINT`	Integer	Signed 32-bit integer value (±2,147,483,647)
`TRAT`	Rational	(numerator, denominator) pair of integers
`TFLOAT`	Real	Floating-point number
`TNIL`	NIL	The empty list / false value
`TCLOSURE`	Closure	(LAMBDA, environment) pair
`TSTRING`	String	Text of up to 36 characters

B.7 Functions NOT Implemented

The following functions **don't exist** in MYLISP as primitives. Don't use their names expecting them to work:

`ABS`, `MAX`, `MIN`, `SQRT`, `EXPT`, `REVERSE`, `LENGTH`, `MAP`, `FILTER`, `REDUCE`, `APPLY`

Most can be defined in pure LISP (see Appendix A.6).

B.8 Differences From Common LISP and Scheme

Feature	MYLISP	Common LISP	Scheme
Namespace	Lisp-1	Lisp-2	Lisp-1
Booleans	NIL / anything else	NIL / T	`#f` / `#t`
Name truncation	8 chars	Unlimited	Unlimited
TCO	No	No (in general)	Yes (required)
Exact integers	32 bits, no promotion	Bignum	Bignum
Rationals	Yes, native	Yes	Yes
Exit	`BYE`	`(QUIT)`	`(exit)`

Continues in Appendix C: Error Messages

Appendix C — Error Messages

C.1 Arithmetic Errors

Message	Cause
`ERROR: value out of range`	The result of an integer operation exceeds the 32-bit range (±2,147,483,647).
`Use real numbers (e.g. 1.0) if you need larger ranges.`	Supplementary note for integer overflow.
`ERROR: division by zero`	Attempt to divide by 0 (integer, rational, or real).

C.2 Structure Errors

Message	Cause
`ERROR: CAR requires a non-empty list`	`CAR` was applied to `NIL`.
`ERROR: CDR requires a non-empty list`	`CDR` was applied to `NIL`.

C.3 Evaluation Errors

Message	Cause
`ERROR: undefined symbol: XXXX`	A symbol with no value in the environment was referenced. The name shown is the symbol truncated to 8 characters.

C.4 Diagnosing Common Errors

"undefined symbol: FACTORIA" — You tried to call `FACTORIAL` (9 chars), which MYLISP stores as `FACTORIA`. Use the truncated name in your code.

"value out of range" — Result outside the 32-bit range (±2,147,483,647). Options: use reals (`1.0` instead of `1`), or redesign the algorithm.

"CAR requires a non-empty list" — You tried `(CAR NIL)`. Add a `(NULL list)` test before calling `CAR`.

The REPL isn't responding — You probably have unclosed parentheses. Type the missing parentheses and press ENTER. If that doesn't work, type `BYE` and restart.

Continues in Appendix D: CAS Quick Reference — Every Function

---

Appendix D — CAS Quick Reference: Every Function

> Unlike Chapter 20's usage guide — meant for someone who just wants to *use* the already-built CAS — this appendix catalogs **all 59 functions in `CAS.LSP`**, one by one, exactly as they stand in their final version after the ten chapters of Part II. It's the reference to consult to find out, at a glance, what any function in the file takes in and returns.

D.1 Representation and Constructors (Chapter 10)

Function	Input	Returns
`ISCONST?`	`(ISCONST? E)`	`T` if `E` is a numeric constant; `NIL` otherwise.
`ISVAR?`	`(ISVAR? E)`	`T` if `E` is a symbol (variable); `NIL` otherwise.
`ISSUM?`	`(ISSUM? E)`	`T` if `E` is a list whose head is `+`; `NIL` otherwise.
`ISPROD?`	`(ISPROD? E)`	`T` if `E` is a list whose head is `*`; `NIL` otherwise.
`ISPOW?`	`(ISPOW? E)`	`T` if `E` is a list whose head is `POT`; `NIL` otherwise.
`ADDEND`	`(ADDEND E)`	The first addend of a sum `E`.
`AUGEND`	`(AUGEND E)`	The second addend of a sum `E`.
`FACTOR1`	`(FACTOR1 E)`	The first factor of a product `E`.
`FACTOR2`	`(FACTOR2 E)`	The second factor of a product `E`.
`BASE`	`(BASE E)`	The base of a power `E`.
`EXPON`	`(EXPON E)`	The exponent of a power `E`.
`CERO?`	`(CERO? X)`	`T` if `X` is the constant `0`; `NIL` otherwise.
`UNO?`	`(UNO? X)`	`T` if `X` is the constant `1`; `NIL` otherwise.
`MAKESUM`	`(MAKESUM A1 A2)`	The sum `A1 + A2`, simplified at construction time (removes zeros, combines constants); if no rule applies, `(+ A1 A2)`.
`MAKEPROD`	`(MAKEPROD A1 A2)`	The product `A1 · A2`, simplified at construction time (removes zeros/ones, combines constants); if no rule applies, `(* A1 A2)`.
`MAKEPOW`	`(MAKEPOW B E)`	The power `B^E`, simplified at construction time (exponent 0 or 1, base 1, combining constants); if no rule applies, `(POT B E)`.

D.2 Substitution and Evaluation (Chapter 11)

Function	Input	Returns
`SUBST`	`(SUBST E VBL VAL)`	`E` with every occurrence of the variable `VBL` substituted by `VAL`, without evaluating.
`LIBRE?`	`(LIBRE? X)`	`T` if `X` is the `VARLIBRE` sentinel; `NIL` otherwise.
`POTENCIA`	`(POTENCIA B N)`	`B` raised to `N` (an integer), computed by recursive multiplication.
`EVALSUMA`	`(EVALSUMA E)`	The numeric value of the sum `E`, or `VARLIBRE` if some addend has a free variable.
`EVALPROD`	`(EVALPROD E)`	The numeric value of the product `E`, or `VARLIBRE` if some factor has a free variable.
`EVALPOW`	`(EVALPOW E)`	The numeric value of the power `E`, or `VARLIBRE` if the base or the exponent has a free variable.
`EVALEXPR`	`(EVALEXPR E)`	The numeric value of `E` if it has no free variables; `VARLIBRE` (with a printed warning) otherwise.

D.3 Raw Differentiation (Chapter 12)

Function	Input	Returns
`DSUMA`	`(DSUMA E VBL)`	The raw (unsimplified) derivative of a sum `E` with respect to `VBL`.
`DPROD`	`(DPROD E VBL)`	The raw derivative of a product `E` with respect to `VBL`, via the product rule.
`DPOW`	`(DPOW E VBL)`	The raw derivative of a power `E` with respect to `VBL` (constant exponent).
`DERIV`	`(DERIV E VBL)`	The raw symbolic derivative of `E` with respect to `VBL`, unsimplified.

D.4 Canonical Order and Like Terms (Chapter 14)

Function	Input	Returns
`COEF`	`(COEF E)`	The numeric coefficient of a term `E` (`1` if it has none explicit).
`LITERAL`	`(LITERAL E)`	The literal (non-numeric) part of a term `E`.
`FLATSUM`	`(FLATSUM E)`	The list of every addend in `E`, flattening any chain of nested sums.
`ASSOCADD`	`(ASSOCADD LIT CF ALIST)`	`ALIST` updated: adds `CF` to the coefficient already accumulated for `LIT`, or adds a new entry if `LIT` wasn't there.
`AGRUPA`	`(AGRUPA TERMS)`	An association list `(literal . coefficient)`, grouping the like terms in `TERMS`.
`REBUILD`	`(REBUILD ALIST)`	The sum rebuilt from the association list `ALIST`.
`SIMPSUM`	`(SIMPSUM E)`	The sum `E` simplified: like terms grouped and in canonical order.

D.5 Full-Tree Simplification (Chapter 15)

Function	Input	Returns
`SIMP`	`(SIMP E)`	`E` fully simplified, walking and simplifying every node in the tree.

D.6 Product Simplifier (Chapter 16)

Function	Input	Returns
`BASEOF`	`(BASEOF E)`	The base of `E` if it's a power; `E` itself otherwise.
`EXPOF`	`(EXPOF E)`	The exponent of `E` if it's a power; `1` otherwise.
`FLATPROD`	`(FLATPROD E)`	The list of every factor in `E`, flattening any chain of nested products.
`NUMPROD`	`(NUMPROD TERMS)`	The product of every numeric constant present in `TERMS`.
`SYMTERMS`	`(SYMTERMS TERMS)`	The terms in `TERMS` that aren't numeric constants.
`ADDBASE`	`(ADDBASE B EXP ALIST)`	`ALIST` updated: adds `EXP` to the exponent already accumulated for base `B`, or adds a new entry if `B` wasn't there.
`AGRUPAP`	`(AGRUPAP TERMS)`	An association list `(base . exponent)`, grouping the repeated factors in `TERMS`.
`REBUILDP`	`(REBUILDP ALIST)`	The product rebuilt from the association list `ALIST`.
`SIMPPROD`	`(SIMPPROD E)`	The product `E` simplified: repeated factors combined into powers and in canonical order.

D.7 The Orchestrator (Chapter 17)

Function	Input	Returns
`DERIVA`	`(DERIVA E VBL)`	The derivative of `E` with respect to `VBL`, already simplified.
`DERIVAEN`	`(DERIVAEN E VBL VAL)`	The derivative of `E` with respect to `VBL`, numerically evaluated by substituting `VBL` with `VAL`.

D.8 Stable Canonical Order (Chapter 18)

Function	Input	Returns
`TIPORDEN`	`(TIPORDEN E)`	`0` if `E` is a constant, `1` if it's a variable, `2` if it's a compound expression.
`EXPLESS?`	`(EXPLESS? E1 E2)`	`T` if `E1` should come before `E2` in canonical order; `NIL` otherwise.
`INSERTA`	`(INSERTA PAR LISTA)`	`LISTA` with the pair `PAR` inserted at its sorted position according to `EXPLESS?`.
`ORDENA`	`(ORDENA LISTA)`	`LISTA` of `(key . value)` pairs sorted according to `EXPLESS?`.

D.9 Infix Printing (Chapter 19)

Function	Input	Returns
`PRECOP`	`(PRECOP E)`	The precedence of `E`'s main operator (power `3`, product `2`, sum `1`, atom `4`).
`ENVOLVER`	`(ENVOLVER E S MINIMA)`	The string `S` wrapped in parentheses if `E`'s precedence is lower than `MINIMA`; `S` unchanged otherwise.
`NEGTERM?`	`(NEGTERM? E)`	`T` if `E` is a negative term (negative constant or product with a negative coefficient); `NIL` otherwise.
`NEGAR`	`(NEGAR E)`	`E` with its sign flipped.
`INFBASE`	`(INFBASE B)`	The text representation of a power's base `B`, with the minimum precedence of an atom.
`INFPROD`	`(INFPROD E)`	The infix text representation of a product `E` (not wrapped in parentheses).
`INFSUMA`	`(INFSUMA E)`	The infix text representation of a sum `E`, showing a subtraction when the second addend is negative.
`INFIJO`	`(INFIJO E MINIMA)`	The infix text representation of `E`, with the minimum parentheses needed given the context's `MINIMA` precedence.
`PRINTMAT`	`(PRINTMAT E)`	Displays `E` in readable mathematical notation and ends the line; doesn't return a useful value (output effect).

Continues in Appendix E: Complete Source Code of CAS.LSP

Appendix E — Complete Source Code of `CAS.LSP`

> The complete listing, as it stands after the ten chapters of Part II, in the same order it was built. Meant to be copied into a `CAS.LSP` file and loaded all at once with `(LOAD "CAS.LSP")` (or the 8-character filename your platform uses, section 8.3), without having to go chapter by chapter looking for each function. To find out what each one takes in and returns, see Appendix D.

```
; ============================================================
; Chapter 10 -- Representation and Data Abstraction
; ============================================================
; ** CLASSIFICATION PREDICATES **
(DEFUN ISCONST? (E) (NUMBERP E))
(DEFUN ISVAR? (E) (SYMBOLP E))
(DEFUN ISSUM? (E)
  (AND (LISTP E) (EQ (CAR E) '+)))
(DEFUN ISPROD? (E)
  (AND (LISTP E) (EQ (CAR E) '*)))
(DEFUN ISPOW? (E)
  (AND (LISTP E) (EQ (CAR E) 'POT)))
; ** SELECTORS **
(DEFUN ADDEND (E) (CAR (CDR E)))
(DEFUN AUGEND (E) (CAR (CDR (CDR E))))
(DEFUN FACTOR1 (E) (CAR (CDR E)))
(DEFUN FACTOR2 (E) (CAR (CDR (CDR E))))
(DEFUN BASE (E) (CAR (CDR E)))
(DEFUN EXPON (E) (CAR (CDR (CDR E))))
; ** CONSTRUCTORS (smart version, updated in Chapter 13) **
```

```
(DEFUN CERO? (X) (AND (ISCONST? X) (= X 0)))
(DEFUN UNO?  (X) (AND (ISCONST? X) (= X 1)))
(DEFUN MAKESUM (A1 A2)
  (COND ((CERO? A1) A2)
        ((CERO? A2) A1)
        ((AND (ISCONST? A1) (ISCONST? A2)) (+ A1 A2))
        (T (LIST '+ A1 A2))))
(DEFUN MAKEPROD (A1 A2)
  (COND ((OR (CERO? A1) (CERO? A2)) 0)
        ((UNO? A1) A2)
        ((UNO? A2) A1)
        ((AND (ISCONST? A1) (ISCONST? A2)) (* A1 A2))
        (T (LIST '* A1 A2))))
(DEFUN MAKEPOW (B E)
  (COND ((CERO? E) 1)
        ((UNO? E) B)
        ((UNO? B) 1)
        ((AND (ISCONST? B) (ISCONST? E)) (POTENCIA B E))
        (T (LIST 'POT B E))))
; ============================================================
; Chapter 11 -- Substitution (SUBST) and Numeric Evaluation
; ============================================================
(DEFUN SUBST (E VBL VAL)
  (COND ((ISCONST? E) E)
        ((ISVAR? E)
         (COND ((EQ E VBL) VAL)
               (T E)))
        ((ISSUM? E)
         (MAKESUM (SUBST (ADDEND E) VBL VAL)
                  (SUBST (AUGEND E) VBL VAL)))
        ((ISPROD? E)
         (MAKEPROD (SUBST (FACTOR1 E) VBL VAL)
                   (SUBST (FACTOR2 E) VBL VAL)))
        ((ISPOW? E)
         (MAKEPOW (SUBST (BASE E) VBL VAL)
                  (SUBST (EXPON E) VBL VAL)))))
(DEFUN LIBRE? (X) (EQ X 'VARLIBRE))
(DEFUN POTENCIA (B N)
  (COND ((= N 0) 1)
        (T (* B (POTENCIA B (- N 1))))))
(DEFUN EVALSUMA (E)
  (LET ((A (EVALEXPR (ADDEND E))))
    (COND ((LIBRE? A) A)
          (T (LET ((B (EVALEXPR (AUGEND E))))
               (COND ((LIBRE? B) B)
                     (T (+ A B))))))))
(DEFUN EVALPROD (E)
  (LET ((A (EVALEXPR (FACTOR1 E))))
    (COND ((LIBRE? A) A)
          (T (LET ((B (EVALEXPR (FACTOR2 E))))
               (COND ((LIBRE? B) B)
                     (T (* A B))))))))
(DEFUN EVALPOW (E)
  (LET ((B (EVALEXPR (BASE E))))
    (COND ((LIBRE? B) B)
          (T (LET ((N (EVALEXPR (EXPON E))))
               (COND ((LIBRE? N) N)
                     (T (POTENCIA B N))))))))
```

```
(DEFUN EVALEXPR (E)
  (COND ((ISCONST? E) E)
        ((ISVAR? E)
         (PROGN (PRINT "CAS: variable libre, no evaluable:")
                (PRINT E)
                'VARLIBRE))
        ((ISSUM? E) (EVALSUMA E))
        ((ISPROD? E) (EVALPROD E))
        ((ISPOW? E) (EVALPOW E))))
; ============================================================
; Chapter 12 -- Raw Symbolic Differentiation (DERIV)
; ============================================================
(DEFUN DSUMA (E VBL)
  (MAKESUM (DERIV (ADDEND E) VBL) (DERIV (AUGEND E) VBL)))
(DEFUN DPROD (E VBL)
  (MAKESUM (MAKEPROD (DERIV (FACTOR1 E) VBL) (FACTOR2 E))
           (MAKEPROD (FACTOR1 E) (DERIV (FACTOR2 E) VBL))))
(DEFUN DPOW (E VBL)
  (MAKEPROD (MAKEPROD (EXPON E) (MAKEPOW (BASE E) (- (EXPON E) 1)))
            (DERIV (BASE E) VBL)))
(DEFUN DERIV (E VBL)
  (COND ((ISCONST? E) 0)
        ((ISVAR? E) (COND ((EQ E VBL) 1) (T 0)))
        ((ISSUM? E) (DSUMA E VBL))
        ((ISPROD? E) (DPROD E VBL))
        ((ISPOW? E) (DPOW E VBL))))
; ============================================================
; Chapter 14 -- Canonical Order and Like Terms (SIMPSUM)
; ============================================================
(DEFUN COEF (E)
  (COND ((ISCONST? E) E)
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E)) (FACTOR1 E))
               ((ISCONST? (FACTOR2 E)) (FACTOR2 E))
               (T 1)))
        (T 1)))
(DEFUN LITERAL (E)
  (COND ((ISCONST? E) 1)
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E)) (FACTOR2 E))
               ((ISCONST? (FACTOR2 E)) (FACTOR1 E))
               (T E)))
        (T E)))
(DEFUN FLATSUM (E)
  (COND ((ISSUM? E)
         (APPEND (FLATSUM (ADDEND E)) (FLATSUM (AUGEND E))))
        (T (LIST E))))
(DEFUN ASSOCADD (LIT CF ALIST)
  (COND ((NULL ALIST) (LIST (CONS LIT CF)))
        ((EQUAL (CAR (CAR ALIST)) LIT)
         (CONS (CONS LIT (+ (CDR (CAR ALIST)) CF)) (CDR ALIST)))
        (T (CONS (CAR ALIST) (ASSOCADD LIT CF (CDR ALIST))))))
(DEFUN AGRUPA (TERMS)
  (COND ((NULL TERMS) NIL)
        (T (ASSOCADD (LITERAL (CAR TERMS))
                     (COEF (CAR TERMS))
                     (AGRUPA (CDR TERMS))))))
(DEFUN REBUILD (ALIST)
```

```
  (COND ((NULL ALIST) 0)
        (T (MAKESUM (MAKEPROD (CDR (CAR ALIST)) (CAR (CAR ALIST)))
                    (REBUILD (CDR ALIST))))))
(DEFUN SIMPSUM (E) (REBUILD (ORDENA (AGRUPA (FLATSUM E)))))
; ============================================================
; Chapter 15 -- Full-Tree Simplification (SIMP)
; ============================================================
(DEFUN SIMP (E)
  (COND ((ISCONST? E) E)
        ((ISVAR? E) E)
        ((ISSUM? E)
         (SIMPSUM (MAKESUM (SIMP (ADDEND E)) (SIMP (AUGEND E)))))
        ((ISPROD? E)
         (SIMPPROD (MAKEPROD (SIMP (FACTOR1 E)) (SIMP (FACTOR2 E)))))
        ((ISPOW? E)
         (MAKEPOW (SIMP (BASE E)) (SIMP (EXPON E))))))
; ============================================================
; Chapter 16 -- Product Simplifier (SIMPPROD)
; ============================================================
(DEFUN BASEOF (E) (COND ((ISPOW? E) (BASE E)) (T E)))
(DEFUN EXPOF  (E) (COND ((ISPOW? E) (EXPON E)) (T 1)))
(DEFUN FLATPROD (E)
  (COND ((ISPROD? E)
         (APPEND (FLATPROD (FACTOR1 E)) (FLATPROD (FACTOR2 E))))
        (T (LIST E))))
(DEFUN NUMPROD (TERMS)
  (COND ((NULL TERMS) 1)
        ((ISCONST? (CAR TERMS)) (* (CAR TERMS) (NUMPROD (CDR TERMS))))
        (T (NUMPROD (CDR TERMS)))))
(DEFUN SYMTERMS (TERMS)
  (COND ((NULL TERMS) NIL)
        ((ISCONST? (CAR TERMS)) (SYMTERMS (CDR TERMS)))
        (T (CONS (CAR TERMS) (SYMTERMS (CDR TERMS))))))
(DEFUN ADDBASE (B EXP ALIST)
  (COND ((NULL ALIST) (LIST (CONS B EXP)))
        ((EQUAL (CAR (CAR ALIST)) B)
         (CONS (CONS B (+ (CDR (CAR ALIST)) EXP)) (CDR ALIST)))
        (T (CONS (CAR ALIST) (ADDBASE B EXP (CDR ALIST))))))
(DEFUN AGRUPAP (TERMS)
  (COND ((NULL TERMS) NIL)
        (T (ADDBASE (BASEOF (CAR TERMS))
                    (EXPOF (CAR TERMS))
                    (AGRUPAP (CDR TERMS))))))
(DEFUN REBUILDP (ALIST)
  (COND ((NULL ALIST) 1)
        (T (MAKEPROD (MAKEPOW (CAR (CAR ALIST)) (CDR (CAR ALIST)))
                     (REBUILDP (CDR ALIST))))))
(DEFUN SIMPPROD (E)
  (LET ((TERMS (FLATPROD E)))
    (MAKEPROD (NUMPROD TERMS)
              (REBUILDP (ORDENA (AGRUPAP (SYMTERMS TERMS)))))))
; ============================================================
; Chapter 17 -- The Orchestrator (DERIVA and DERIVAEN)
; ============================================================
(DEFUN DERIVA (E VBL) (SIMP (DERIV E VBL)))
(DEFUN DERIVAEN (E VBL VAL) (EVALEXPR (SUBST (DERIVA E VBL) VBL VAL)))
; ============================================================
; Chapter 18 -- Stable Canonical Order (EXPLESS?)
```

```
; ============================================================
(DEFUN TIPORDEN (E)
  (COND ((ISCONST? E) 0)
        ((ISVAR? E) 1)
        (T 2)))
(DEFUN EXPLESS? (E1 E2)
  (COND ((< (TIPORDEN E1) (TIPORDEN E2)) T)
        ((> (TIPORDEN E1) (TIPORDEN E2)) NIL)
        ((ISCONST? E1) (< E1 E2))
        ((ISVAR? E1) (STR< E1 E2))
        (T NIL)))
(DEFUN INSERTA (PAR LISTA)
  (COND ((NULL LISTA) (LIST PAR))
        ((EXPLESS? (CAR PAR) (CAR (CAR LISTA))) (CONS PAR LISTA))
        (T (CONS (CAR LISTA) (INSERTA PAR (CDR LISTA))))))
(DEFUN ORDENA (LISTA)
  (COND ((NULL LISTA) NIL)
        (T (INSERTA (CAR LISTA) (ORDENA (CDR LISTA))))))
; ============================================================
; Chapter 19 -- Infix Printing (PRINTMAT)
; ============================================================
(DEFUN PRECOP (E)
  (COND ((ISPOW? E) 3)
        ((ISPROD? E) 2)
        ((ISSUM? E) 1)
        (T 4)))
(DEFUN ENVOLVER (E S MINIMA)
  (COND ((< (PRECOP E) MINIMA) (STRCAT "(" (STRCAT S ")")))
        (T S)))
(DEFUN NEGTERM? (E)
  (COND ((ISCONST? E) (< E 0))
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E)) (< (FACTOR1 E) 0))
               ((ISCONST? (FACTOR2 E)) (< (FACTOR2 E) 0))
               (T NIL)))
        (T NIL)))
(DEFUN NEGAR (E)
  (COND ((ISCONST? E) (- E))
        ((ISPROD? E)
         (COND ((ISCONST? (FACTOR1 E))
                (MAKEPROD (- (FACTOR1 E)) (FACTOR2 E)))
               ((ISCONST? (FACTOR2 E))
                (MAKEPROD (FACTOR1 E) (- (FACTOR2 E))))
               (T E)))
        (T E)))
(DEFUN INFBASE (B) (INFIJO B 4))
(DEFUN INFPROD (E)
  (COND ((ISCONST? (FACTOR1 E))
         (COND ((= (FACTOR1 E) -1) (STRCAT "-" (INFIJO (FACTOR2 E) 2)))
               (T (STRCAT (FACTOR1 E) (INFIJO (FACTOR2 E) 2)))))
        ((ISCONST? (FACTOR2 E))
         (COND ((= (FACTOR2 E) -1) (STRCAT "-" (INFIJO (FACTOR1 E) 2)))
               (T (STRCAT (FACTOR2 E) (INFIJO (FACTOR1 E) 2)))))
        (T (STRCAT (INFIJO (FACTOR1 E) 2)
                   (STRCAT "*" (INFIJO (FACTOR2 E) 2))))))
(DEFUN INFSUMA (E)
  (COND ((NEGTERM? (AUGEND E))
         (STRCAT (INFIJO (ADDEND E) 1)
```

```
                  (STRCAT " - " (INFIJO (NEGAR (AUGEND E)) 1))))
        (T (STRCAT (INFIJO (ADDEND E) 1)
                   (STRCAT " + " (INFIJO (AUGEND E) 1))))))
(DEFUN INFIJO (E MINIMA)
  (COND ((ISCONST? E) (STRCAT E ""))
        ((ISVAR? E) (STRCAT E ""))
        ((ISPOW? E)
         (ENVOLVER E
                   (STRCAT (INFBASE (BASE E))
                           (STRCAT "^" (INFIJO (EXPON E) 3)))
                   MINIMA))
        ((ISPROD? E) (ENVOLVER E (INFPROD E) MINIMA))
        ((ISSUM? E) (ENVOLVER E (INFSUMA E) MINIMA))))
(DEFUN PRINTMAT (E) (PROGN (DISPLAY (INFIJO E 0)) (NEWLINE)))
```